Mälardalen University
Faculty of Engineering and Health Sciences
Department of Computer Science and Engineering
Västerås, Sweden

Thesis for the Degree of Bachelor in Computer Science, 15 hp

Application of Agentic AI and Large Language Models in Industrial Requirements Engineering

Iman Ismael
iil23002@student.mdu.se

Examiner: Jacob Axelsson
Mälardalen University, Västerås, Sweden

Supervisors: Kristian Sandström
Mälardalen University, Västerås, Sweden

Company supervisor: Christer Norström
Norström&Wretås AB, Sweden

June 3, 2026

Abstract

Industrial Requirements Engineering (RE) in safety-critical sectors moves through several manual document tiers before reaching code. Each tier transition is detailed, formal work that takes time and reviewer attention to maintain traceability. The research literature shows that very few published Artificial Intelligence (AI) and Large Language Model (LLM) systems for RE have reached production use. This thesis explores how AI and LLMs can support industrial Requirements Engineering, in partnership with Westermo Network Technologies AB. Westermo is a Swedish manufacturer of industrial data communications equipment for mission-critical sectors such as rail, energy, and water infrastructure.

Design Science Research (DSR) is used as the methodology. Qualitative discussions with senior Westermo staff pointed to the manual step from the Market Requirement Specification (MRS) to the Functional Specification (FS) as the place where AI support would help most. In response, a four-agent pipeline is proposed. It is built around a shared Retrieval-Augmented Generation (RAG) knowledge base and two mandatory Human-in-the-Loop (HITL) checkpoints. A working prototype runs the pipeline end-to-end on three deployment backends. Three evaluation strands are used: a side-by-side comparison of five LLMs, a fault-injection test, and an expert review with two Likert forms and free-text comments.

A model running locally came within one percentage point of a large cloud model on semantic similarity, and still produced a complete and fully traced FS. Defects that show up in the text are caught in every run on the cloud model, while missing content is caught in only one run out of five. The expert reviewers rated the conceptual design at a median of $4/5$ *and gave the symmetric review setup the maximum score.*

The novel multi-agent Requirements Engineering design shows that confidentiality and the human-oversight rules of the EU AI Act can be met end-to-end without losing output quality. Placing a dedicated Review Agent after every generative phase is a viable extension of existing multi-agent RE frameworks.

Table of Contents

Abstract **1**

1 Introduction **4**

2 Background **5**
2.1 Industrial Context . . . 5
2.1.1 Market Requirements Specification . . . 6
2.1.2 Functional Specification . . . 6
2.1.3 Software Requirements . . . 6
2.2 Artificial Intelligence and Large Language Models . . . 6
2.3 Multi-Agent Systems and Agentic AI . . . 6
2.4 Human-In-The-Loop . . . 6
2.5 Retrieval-Augmented Generation . . . 7
2.6 Requirements Engineering . . . 7
2.7 IEC 62443-4-2 . . . 7
2.8 On-Prem and Cloud AI Deployment . . . 7

3 Related Work **8**
3.1 Multi-Agent Frameworks . . . 8
3.2 Retrieval-Augmented Generation . . . 8
3.3 Ambiguity Detection . . . 8
3.4 On-Prem AI . . . 9
3.5 AI-Assisted Code Generation . . . 9

4 Problem Formulation **10**
4.1 Research Questions . . . 10
4.2 Scope and Limitations . . . 10

5 Method **11**
5.1 Literature Study (RQ1) . . . 11
5.2 Qualitative Conversations (RQ2) . . . 11
5.3 Design Science Research (RQ3) . . . 11
5.3.1 Artifact Design . . . 12
5.3.2 Routing and Control Flow . . . 12
5.3.3 Prototyping . . . 12
5.3.4 RAG Architecture . . . 13
5.4 Evaluation (RQ4) . . . 13
5.4.1 Expert Evaluation . . . 13
5.4.2 Comparative Model Evaluation . . . 14
5.4.3 Semantic Similarity Scoring . . . 15
5.4.4 Fault-Injection Testing . . . 15

6 Ethical and Societal Considerations **16**

7 Design and Implementation **17**
7.1 Conceptual Design . . . 17
7.1.1 Symmetric Review Architecture . . . 17
7.1.2 Pipeline Components . . . 19
7.1.3 Theoretical Framework . . . 20
7.2 Prototype Implementation . . . 21
7.3 Experiment Setup . . . 22
7.3.1 Comparative Model Evaluation . . . 22
7.3.2 Fault-Injection Testing . . . 23
7.3.3 Expert Evaluation . . . 23

8 Results **24**
8.1 Findings from Qualitative Conversations 24
8.2 Expert Evaluation 25
8.2.1 Conceptual Design Evaluation 25
8.2.2 Functional Specification (prototype output) Evaluation 26
8.3 Comparative Model Evaluation 26
8.3.1 Comparison Table 26
8.3.2 Quality Analysis 27
8.3.3 Security and Confidentiality Considerations 27
8.4 Fault-Injection Testing 27
8.4.1 Injected Faults 27
8.4.2 Detection Results 28
8.5 Threats to Validity 28

9 Discussion **30**
9.1 RQ1 — State of the Art in AI/LLM for Industrial RE 30
9.2 RQ2 — Westermo's Specific MRS-to-FS Needs 30
9.3 RQ3 — A Four-Agent MRS-to-FS Pipeline 30
9.4 RQ4 — Strengths, Limitations, and Risks 31
9.5 Generalizability Beyond Westermo 32
9.6 Synthesis 32

10 Conclusions **33**

11 Future Work **34**

References **36**

Appendix A Agent System Prompts **37**

Appendix B RAG Knowledge Base **40**

Appendix C Orchestrator Pseudocode **42**

Appendix D Reference MRS — Storm Control **44**

Appendix E Fault-Injected MRS **47**

Appendix F Generated Functional Specification (Claude Opus 4.7) **50**

1 Introduction

Software development in safety-critical industries depends on long *Requirements Engineering (RE)* chains. Customer-needs move through several manual document tiers before they reach implementation code. Industrial standards demand an unbroken traceability chain from each requirement to code and tests. Each tier transition is detailed and time-consuming. Work invested in the early tiers pays back five to ten times over compared to revisions made later [1].

Artificial Intelligence (AI) and *Large Language Models (LLMs)* have now matured enough to support this work. This thesis looks at how to apply them concretely, in partnership with Westermo Network Technologies AB. Westermo is a Swedish manufacturer of industrial data communications equipment for mission-critical sectors. Westermo's three-tier requirements chain is the industrial case used to ground and evaluate the proposed AI support. The tiers are: *Market Requirement Specification (MRS)*, *Functional Specification (FS)*, and Software Requirements, with manual transitions between them.

Large Language Models and Multi-Agent Systems are now good enough to help with this work. Several multi-agent frameworks for Requirements Engineering have appeared in the last few years. MARE divides the RE process into four tasks, carried out by five role-based agents [2]. Sami et al. tackle user-story prioritization in Agile work using role-based LLM agents [3]. ReqRAG, deployed in an industrial collaboration with Alstom, uses *Retrieval-Augmented Generation (RAG)* to answer engineers' questions across release notes, manuals, and system architecture documents [4]. Even so, systematic reviews find that only a small share of the published systems have reached real production use [1]. Two main obstacles explain the gap. First, most frameworks use cloud-hosted models that do not fit the confidentiality rules of regulated industries. Second, few are built to meet the human-oversight rules that the EU AI Act sets for high-risk AI [5].

How AI and LLMs can support industrial Requirements Engineering is the focus of this thesis. The method is Design Science Research [6], which fits here since the main output is a designed artifact. Discussions with Westermo's leading personnel pointed to the manual MRS-to-FS step as the place where AI could support most. Errors caught in this step cannot spread further down the chain. The design is a four-agent pipeline built around a shared RAG knowledge base, with two mandatory Human-in-the-Loop checkpoints. A working prototype runs the pipeline end-to-end on three backends from the same prompts.

Evaluation uses three strands: a side-by-side study of five LLMs, a fault-injection test, and an expert review with two Likert forms. There are three main contributions. First, a novel four-agent MRS-to-FS pipeline that places a dedicated Review Agent after every generative phase. Second, a symmetric, severity-tagged review setup with two mandatory Human-in-the-Loop checkpoints. Third, an empirical demonstration that the same pipeline runs end-to-end on three deployment backends.

Chapters included: Background (Chapter 2), Related Work (Chapter 3), Problem Formulation (Chapter 4), Method (Chapter 5), Ethics (Chapter 6), Approach (Chapter 7), Results (Chapter 8), Discussion (Chapter 9), Conclusion (Chapter 10), and Future Work (Chapter 11).

2 Background

In this chapter the general industrial problem that motivates the work is presented. It walks through the terms and concepts needed to follow the contribution. Westermo Network Technologies AB is the partner organization, and fits as the concrete case.

2.1 Industrial Context

Safety-critical industries that develop embedded software depend on long *Requirements Engineering (RE)* chains. Customer-needs are captured at a business level, then progressively refined through several document tiers before they reach implementation code. Layered requirements chains of this kind are common across the safety-critical software industry. They are reflected in widely used development lifecycle models such as the V-model. Requirements are written at multiple abstraction levels and verified against matching test artifacts on the way back up. The transitions between tiers are formal work that has to keep traceability intact end-to-end.

Westermo Network Technologies AB is the industrial partner for this work and provides the concrete case used throughout the thesis. Westermo is a Swedish company founded in 1975, and part of Ependion [7]. The company designs and manufactures industrial data communications equipment. Westermo serves sectors where network downtime carries real safety and operational consequences [7].

Westermo's software development process uses a three-tier requirements chain, see Figure 1 . At the top sits the *Market Requirement Specification (MRS)*, owned by the *Product Manager (PM)*, which captures what customers need at a business level. Below it sits the *Functional Specification (FS)*, owned by the *Product Owner (PO)*, which turns those needs into system functions. At the bottom sit the *Software Requirements (SR)*, which set the exact behavior the implementation has to meet. Industrial standards add one more requirement: requirements have to be traceable down to code and test.

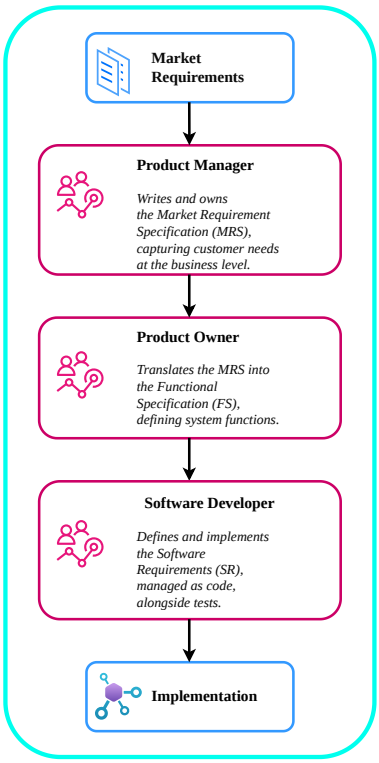


Figure 1: Westermo's three-tier requirements chain, MRS–FS–SR

The transitions between these tiers are done manually. The needs analysis in Chapter 4 picked out the MRS-to-FS step as the area where *Artificial Intelligence (AI)* support would help most.

2.1.1 Market Requirements Specification

A Market Requirements Specification is owned and written by the Product Manager, whose job is to capture market needs and customer demand at the business level. Focus is to solve the problem and why solving it has market value. As the first artifact in the requirements chain, an MRS says what the market needs but not how the technical solution should look. In practice, the document often holds unstructured material, such as notes from customer meetings or raw feedback. These documents have to be interpreted before development can move on.

2.1.2 Functional Specification

A Functional Specification is owned by *Research and Development (R&D)* and written by the Product Owner. The PO's job is to turn the needs in the Market Requirements Specification into precise system behaviors. Today this is done manually as part of a formal, reviewed workflow. The work is reliable but time-consuming, and no automated trace is left back to the original market needs.

2.1.3 Software Requirements

The Software Requirements sit at the most detailed level of the requirements hierarchy, where the requirements meet the code. They are managed as code. Requirement documents are version-controlled and tagged together with the code and test repositories, so that traceability is kept for every software release.

2.2 Artificial Intelligence and Large Language Models

Artificial Intelligence systems are machine-based systems that can act with some degree of autonomy. Rather than following fixed programming rules, they take input data and produce outputs, and they may adjust their behavior as new input comes in. What sets AI apart from regular software is the ability to act in ways that fixed program logic cannot [5]. The AI sub-field most relevant to this thesis is *Generative AI (GenAI)*, where the system produces new content rather than classifying or scoring existing data [1].

One category of GenAI systems is built to handle a wide range of tasks and use cases [5]. *Large Language Models (LLMs)* are a well-known type of GenAI whose main output is text. Because they are trained on large amounts of data, they can produce text that fits the context [8]. LLM size is commonly reported as the number of trainable parameters: "8B" means eight billion parameters, "70B" means seventy billion. Larger parameter counts generally lead to better output quality on natural-language tasks. In *Requirements Engineering (RE)*, LLMs can take on tasks that used to need a human expert, such as spotting when a sentence is ambiguous [1].

2.3 Multi-Agent Systems and Agentic AI

Multi-Agent Systems (MAS) go beyond what a single Large Language Model can do. Each agent is given its own specialized role, much like the way work is split up inside a human development team. In Requirements Engineering, the agents can be set up to play roles such as Product Manager, Product Owner, and Developer, with each agent responsible for one stage of the analysis [3].

When the agents work through a sequence of tasks without a human stepping in between each one, the system is called *Agentic AI*. Instead of reacting to a single prompt, the workflow is split into sub-tasks. One agent produces something, another reviews it, and a third puts together the final result [3].

2.4 Human-In-The-Loop

Human-In-The-Loop (HITL) is the idea that human judgment stays a real part of an AI-assisted workflow. In a Multi-Agent System this matters because the agents have known weaknesses. They can respond out of context and are sensitive to how the prompt is phrased. That is why human experts should stay inside the agent loop rather than be replaced by it [3].

2.5 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) has three stages: retrieval, augmentation, and generation. It works alongside an LLM by mixing the model's own text-generation ability with knowledge pulled from an outside database. Rather than relying only on what was learned during training, the model adds the retrieved content while it generates, which cuts down on out-of-date or missing information [9]. In implementation, the outside source is a collection of pre-computed vectors. Each source document is split into smaller passages. Each passage is converted into a high-dimensional representation that captures its meaning. At query time, the same conversion is applied to the question. The passages whose representations lie closest to the query are returned as context for the LLM [9].

2.6 Requirements Engineering

Requirements Engineering covers the work of eliciting, specifying, and maintaining Software Requirements over the lifetime of a project. RE sets the ground on which every later development decision is built [1]. Doing this work badly is costly. Work invested in the early tiers is estimated to be five to ten times less costly than equivalent revisions made after implementation. The literature attributes more than a third of enterprise software project failures to requirements-related issues [1].

2.7 IEC 62443-4-2

IEC 62443-4-2:2019 [10] is the part of the IEC 62443 series of international standards that specifies technical cybersecurity requirements. The standard organizes its requirements around seven *Foundational Requirements (FRs)* [10]: FR1 Identification and Authentication Control, FR2 Use Control, FR3 System Integrity, FR4 Data Confidentiality, FR5 Restricted Data Flow, FR6 Timely Response to Events, and FR7 Resource Availability. Each FR is realized through a set of *Component Requirements (CR)*, and some carry optional requirement enhancements. Together, the CRs and requirement enhancements determine the security capability level a component is able to claim. The prototype in this thesis uses a guide grounded in this framework as one of its RAG knowledge-base documents (Appendix B). Agents tag the security requirements they generate using the corresponding FR identifiers (e.g. `[FR1-IAC]`, `[FR2-UC]`) and CR numbers (e.g. CR 2.8 *Auditable events*, CR 1.1 *Human user identification and authentication*).

2.8 On-Prem and Cloud AI Deployment

There are two main ways to deploy a Large Language Model. Each one has its own trade-offs around what the model can do and how confidential the data stays. These trade-offs matter for industrial adoption [8].

Cloud-hosted frontier models are large LLMs served through external APIs or web interfaces. They are trained on huge amounts of data and perform well on hard Natural Language tasks [8]. The downside is that using them means sending input data to outside servers. This is a confidentiality risk wherever non-disclosure agreements apply. A well-known example is Claude (Anthropic).

Open-source models deployed locally (*on-prem AI*) are public models that can be run on local hardware so that input and intermediate state stay on the local machine, without sending data externally. Tools, such as Ollama, have made local deployment practical on ordinary consumer machines. Platforms, such as Hugging Face, offer hosted inference as a middle option that avoids local hardware limits without locking you into a commercial API. Examples of open-weight models are Phi-3 (Microsoft), Mistral, and Llama 3 (Meta). The main drawback of running a model locally is that the available hardware sets a ceiling on the model size. In practice this means lower output quality than the largest cloud models [8].

3 Related Work

The use of *Large Language Models (LLMs)* in *Requirements Engineering (RE)* has moved fast. Recent work has shifted to multi-agent frameworks that try to cover full documentation workflows from end-to-end [3, 1].

Industrial adoption has not kept up. The systematic review by Cheng et al. finds that only a small share of the surveyed studies, around 1.3 %, have reached production. The great majority are still at an early development stage [1].

3.1 Multi-Agent Frameworks

Recent work argues that *Multi-Agent Systems (MAS)* do better than single-model setups, because each agent can specialize on a narrow part of the task [3]. Two of the most directly relevant frameworks are MARE [2] and the pipeline by Sami et al. [3].

MARE aims to automate the full RE process. It splits the work into four tasks: elicitation, modeling, verification, and specification. These tasks are carried out by five role-based agents (Stakeholders, Collector, Modeler, Checker, and Documenter) that take turns through a shared workspace [2].

Sami et al. focus on Agile development. Their pipeline produces and prioritizes user stories with LLM agents playing different roles (*Product Owner (PO)*, *Quality Assurance (QA)*, Developer, and Manager) [3]. One interesting finding in their study is that GPT-3.5 was the stronger of the two backbones on the semantic-similarity metric they used, not GPT-4o [3]. The implication is that bigger models do not automatically give higher output quality once they are placed inside an agentic pipeline.

A weakness shared by both frameworks is that quality control is left to the agents alone. MARE's Checker agent runs without any required human checkpoint between phases [2], and the Sami et al. pipeline has no human gate either [3]. Both frameworks are also evaluated primarily on cloud-hosted frontier models [2, 3].

Industrial AI use in the European Union is governed by the EU AI Act, which requires transparency, traceability, and human oversight for any system classified as high-risk [5]. Systems used in safety-critical industrial settings fall inside that scope, including any system whose outputs feed into product compliance documentation. The Act calls for real human supervision rather than fully autonomous decision-making.

The multi-agent frameworks for Requirements Engineering described above do not meet the Act's requirement for human checkpoints at critical decision points [2, 3].

3.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) has become a common way to ground LLM-generated content and cut down on hallucinations. It pairs the model's own knowledge with evidence pulled in from an outside source [9].

ReqRAG is built by Ibtasham et al. in an industrial setting. It is a RAG-based question-answering chatbot that helps engineers find information across release notes, manuals, and system architecture documents [4]. In their human evaluation, around seven out of ten generated answers were judged good enough by the industrial practitioners involved [4].

ReqRAG functions as a search-and-respond tool, pulling passages to answer user questions. Transforming one specification artifact into another is outside its scope. Using RAG to ground the generation of new specification documents from existing ones is still relatively unexplored in industrial Requirements Engineering.

3.3 Ambiguity Detection

Bashir et al. ran an industrial study together at Westermo Network Technologies AB and Alstom Rail AB [11]. They used Large Language Models with in-context learning to spot and explain linguistic ambiguities in Natural Language requirements. They treat the task as binary classification (ambiguous vs. unambiguous), pairing each prediction with a short explanation [11]. Across three

industrial datasets, going from a zero-shot prompt to ten-shot prompt improved performance by roughly twenty percentage points on average [11].

The approach flags ambiguous requirements for human review, a *Human-In-The-Loop (HITL)* handoff that detects but does not resolve. That leaves room for downstream tools that pair detection with proposal generation, which is the direction this thesis takes.

3.4 On-Prem AI

A strong cloud-first pattern runs through both major systematic reviews of LLMs in RE. Cheng et al. find that GPT-family models account for the bulk of the surveyed implementations. The GPT-4 series at around 37 %, the GPT-3.5 series at around a quarter [1]. Zadenoori et al. see the same pattern across 74 primary studies from 2023–2024. GPT-family make up around 90 % of the field at the headline level. It dropped to about 77 % in the detailed breakdown, with LLaMA as the most common open alternative [12]. If an organization's NDA covers the specification artifacts themselves, this cloud-first pattern is a real problem. The documents end up on third-party servers, which is a risk that purely cloud-based studies never have to think about.

Zadenoori et al. flag this as an open gap. They note that smaller organizations rarely have the hardware to run very large models in-house. They argue that the field stands to gain from compact models that can be hosted on-premises, or from comparable solutions that keep data inside the organization [12].

Two recent toolkits show that running LLMs locally is technically possible. **OnPrem.LLM** [13] is a privacy-aware document intelligence toolkit, built around local model orchestration. It is designed for organizations that work under strict data-privacy and compliance rules. **Hive** [14] is a distributed inference framework. It uses Ollama as the local LLM runtime, and lets several nodes share the computing load without exposing the inference to public infrastructure.

These toolkits give a technical base for local deployment. However, they do not say how an organization should structure a multi-agent Requirements Engineering pipeline so that confidentiality is kept end-to-end across every stage.

3.5 AI-Assisted Code Generation

A separate but related line of work asks how LLMs can turn well-specified requirements directly into implementation code. Wei argues that carefully written requirements are the foundation that makes Artificial Intelligence-assisted code generation work in the first place [15]. He shows a tailored LLM that step by step refines requirements into design, tests, and implementation code. That makes careful upstream specification work a research and engineering priority in its own right, whether it is done by hand or with AI support.

4 Problem Formulation

Long *Requirements Engineering (RE)* chains are normal in safety-critical industries. Each manual step between document tiers is formal work that takes time and reviewer attention. The further downstream changes have to be made, the more rework they require. On top of that, industrial standards require that requirements can be traced all the way down to code and test.

Artificial Intelligence (AI) and *Large Language Models (LLMs)* are now reaching the point where they can help with this work, but two industrial constraints get in the way. Most of the published systems run on cloud-hosted models, which clashes with industrial confidentiality rules. Only a few of them meet the human-oversight bar that the EU AI Act sets for high-risk industrial AI. This thesis tries to bridge those two realities in a real industrial setting, in partnership with Westermo Network Technologies AB.

One overarching research problem follows: *how can AI and LLMs be applied to support industrial Requirements Engineering, end-to-end, while keeping data confidentiality and the human-oversight requirements of the EU AI Act?* It is broken down into the four research questions below.

4.1 Research Questions

- *RQ1*: What is the state of the art in applying AI and LLMs to support Requirements Engineering in the industrial product development process?
- *RQ2*: What are the specific needs and challenges at Westermo regarding the MRS-to-FS transition in their product development process?
- *RQ3*: How can an AI-based support system be designed to meet these needs, in support of the leading technical roles?
- *RQ4*: What are the strengths, limitations, and risks associated with the proposed conceptual design and its prototype?

The four research questions correspond, in order, to: a literature study, a set of qualitative conversations, a *Design Science Research (DSR)* design-and-build cycle, and an expert evaluation combined with a fault-injection and comparative-model study. The full methodology is described in Chapter 5.

4.2 Scope and Limitations

The work is bounded in four ways. First, the findings are tied to Westermo's specific situation as the industrial case used to ground the design. This limits how directly the empirical results carry over to other organizations. The underlying pattern, a multi-tier requirements chain with manual transitions, is however common across safety-critical software development. Second, a full-scale, production-ready implementation falls outside the scope of this work. What is delivered is a Design Science Research demonstrator. Third, a NDA is in place between the author and Westermo. Confidential material is kept out of the public report and is handled under that agreement. Fourth, better AI models or tools may have appeared during or after the completion of this work, as the field is developing quickly. Comparative-model results in particular should be read against the backends available at the time of writing.

5 Method

Each research question is answered through a different method. A two-phase literature study and a set of qualitative conversations with senior Westermo staff together identify the gap that *Artificial Intelligence (AI)* can usefully fill. The gap is then turned into a concrete artifact through *Design Science Research (DSR)*, and the resulting design and its prototype are tested through a three-strand evaluation. Westermo provides the industrial setting that grounds the design and gives it a place to be evaluated.

5.1 Literature Study (RQ1)

The literature study ran in two phases, one on each side of the qualitative conversations. Phase one came before the needs analysis and took a broad look at AI and *Large Language Model (LLM)*. Focus was on the use across the industrial product-development life-cycle. The findings showed that most tools today sit in the design phase, with little reaching the upstream requirements artifacts. Phase two came after the needs analysis had picked out *Requirements Engineering (RE)* as the gap at Westermo Network Technologies AB. This narrowed the focus down to LLM and *Multi-Agent System (MAS)* use in RE.

Sources from 2024 to 2026 were given priority. Since the field moves faster than the normal peer-review cycle, non-peer-reviewed material was also used. Since the EU AI Act shapes the design directly, it is included as a primary source even though it is not a research paper.

5.2 Qualitative Conversations (RQ2)

Qualitative conversations were held with people in the leading technical roles to find out what the specific challenges at Westermo were. Participants were the CTO, Software Development Manager, and Project Manager. The format was semi-structured. Each session was framed around a small set of guiding topics but left room for back-and-forth conversations and follow-up questions. This format was used in industrial software-engineering interview studies [16]. Since the gap itself was not known in advance, this format was a good fit here. Each session was documented in internal meeting notes.

Requirements Engineering came out of these conversations as the place where AI support would help most, in particular the manual job of turning *Market Requirement Specification (MRS)* content into the *Functional Specification (FS)*.

5.3 Design Science Research (RQ3)

Design Science Research (DSR) methodology was used to design the conceptual AI-based support system. It follows the six-step process of Peffers et al. [6]. DSR comes from information-systems research and is centered on building and evaluating artifacts that solve real-world problems. A designed artifact is the main output of this thesis, which aligns with this approach. The artifact is grounded both in the existing literature and in the surfaced organizational needs.

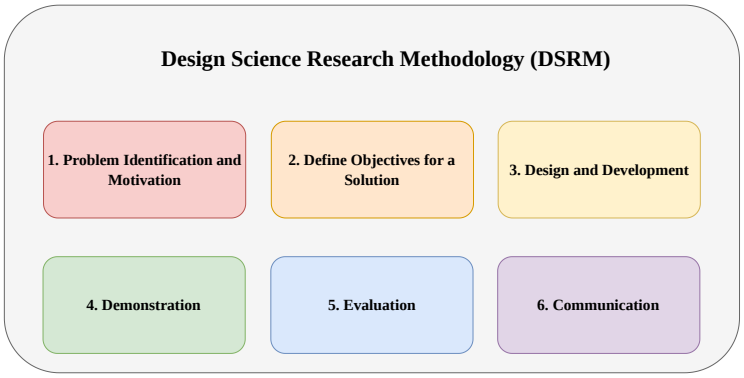


Figure 2: The Design Science Research Methodology

Summarized in Figure 2 is the DSR process used in this work. Qualitative conversations with Westermo leading staff served two purposes. Firstly, they identified the problem and motivated the work (Activity 1). Secondly, they set the goals for a solution (Activity 2). Design and development (Activity 3) produced the conceptual architecture and the working prototype that puts it into practice. The artifact was shown to Westermo staff (Activity 4) and evaluated through Likert-scale feedback (Activity 5). Communication of the results (Activity 6) happens through this thesis.

5.3.1 Artifact Design

Mapping the human roles that own the MRS and the FS, namely the *Product Manager (PM)* and the *Product Owner (PO)*, served as the starting point of the design process. One AI agent was then assigned to mirror each role. Review Agents were added after the literature study showed that dedicated quality-checking agents improve the output of multi-agent Requirements Engineering pipelines [2, 3]. Putting a Review Agent after each generative phase, instead of a single end-of-pipeline check as in MARE [2], is one of the contributions of this thesis. Real human oversight is required by the EU AI Act for high-risk industrial systems [5]. Mandatory *Human-In-The-Loop (HITL)* checkpoints were therefore built in. At each checkpoint, the Product Manager and the Product Owner can review what the agent produced, add information, give feedback, or approve it and continue.

First, early sketches were drawn on paper. Then they were turned into a conceptual architecture in draw.io. Finally, domain experts at Westermo and the supervisors reviewed it before the design was implemented.

5.3.2 Routing and Control Flow

Two ways of orchestrating the agents were considered. Model-driven routing, where the Large Language Model decides which agent runs next, and deterministic routing, where the sequence is fixed in code. The second was chosen, with the pipeline written in Python and each agent's output fed into the next agent in a fixed order. The reason is traceability, because in a safety-critical industrial setting, every state change has to be auditable after the fact. A model-driven router's choices depend on the LLM's run-by-run behavior and are therefore hard to reconstruct. A deterministic Python control flow gives the same path every run and can be inspected like any other code.

5.3.3 Prototyping

A production-ready system is outside the scope of this work. Instead, a working prototype was built to test the logical flow of the design and to cover the Design Science Research demonstration step (Activity 4). *Software Engineering (SE)* research is increasingly using LLMs to help with technical implementation work [15]. A similar approach was used here. Code was generated with the help of two LLMs, Claude and ChatGPT. Each generated file was read through and adjusted by hand, including prompt-tuning, gate logic, control-flow corrections, and bug fixes, so the author understands the codebase in full and is responsible for its current state. That keeps the focus of the study on the artifact itself. At the same time it shows that the multi-agent pipeline can be run in practice on more than one backend.

The author defined the four-agent architecture, the HITL checkpoints, the deterministic routing, each agent's system prompt and output format, and the contents of the RAG knowledge base. The LLM turned those design choices into working code, which was then reviewed and refined step by step until the pipeline behaved as intended.

Three working prototypes are implemented, as shown in Table 1.

Table 1: Prototype deployment configurations evaluated in this thesis.

Prototype	Provider	Models	Data flow
Local	Ollama (local)	Phi-3, Llama 2, Llama 3, Mistral	All input and intermediate state stay on the local machine.
API-based	Hugging Face Inference API	Gemma 3	Hybrid between local and frontier. Input sent to Hugging Face.
Cloud-hosted	Anthropic API	Claude Opus 4.7	Input sent to a third-party inference provider.

Sources and versions. Local runtime: Ollama[1], pulling Phi-3 3.8B (`phi3`[2]), Llama 2 7B (`llama2`[3]), Llama 3 8B (`llama3`[4]), and Mistral 7B (`mistral`[5]). API-based: Hugging Face Inference API[6] serving Gemma 3 27B Instruct-Tuned (`google/gemma-3-27b-it`[7]). Cloud-hosted: Anthropic API[8] via the official `anthropic` Python SDK ($\geq$0.40) serving Claude Opus 4.7 (model ID `claude-opus-4-7`). Orchestration layer: Python 3, LlamaIndex (`llama-index-core` $\geq$0.10[9]) for the RAG layer, `python-docx` $\geq$1.1 for document I/O, and `sentence-transformers` $\geq$2.5 (model `all-MiniLM-L6-v2`) for the semantic-similarity scoring described in Section 5.4.3.

5.3.4 RAG Architecture

A two-tier setup is used, combining system prompts with RAG retrieval. Each agent's system prompt carries its role, its output format, its behavior rules, and the relevant document template (see Appendix A for the prompts). MRS Agent and MRS-Review Agent use MRS template. FS Agent and FS-Review Agent use FS template. These templates are hard-coded into the prompts. The shared retrieval index holds a cybersecurity guide for industrial software product development, and a generic software-process reference.

5.4 Evaluation (RQ4)

5.4.1 Expert Evaluation

A multi-agent pipeline, presented as a Design Science Research artifact, is the output of this thesis. The artifact is a conceptual design and a working prototype that puts it into use. Each output answers a different evaluation question, so two Likert forms are used, one for each. Both forms use the same five-point scale (1 = strongly disagree, 5 = strongly agree), and both mix closed Likert items with free-text comments.

Conceptual design evaluation. The first form looks at the architectural design of the multi-agent pipeline on its own. Whether this is the right thing to build is the question asked. Participants rate the nine design dimensions listed in Table 2.

The nine were chosen to give coverage of the design at three levels and to map onto RQ4 without leaving any major decision unrated. Four dimensions explore the architectural choices that distinguish this pipeline from existing multi-agent RE frameworks: Role separation, Review symmetry, HITL placement, and RAG knowledge base. Three explore the quality and control mechanisms layered on top: Severity-tagged feedback, Loop limit, and Anti-fabrication policy. The last two, Industrial fit and Practical deployment, explore the situational appropriateness of the design relative to a real engineering organization.

An overall recommendation dimension closes the rated section. Four free-text questions cover strongest aspects, weakest aspects, missing components, and industrial deployment concerns.

[1] https://ollama.com
[2] https://ollama.com/library/phi3
[3] https://ollama.com/library/llama2
[4] https://ollama.com/library/llama3
[5] https://ollama.com/library/mistral
[6] https://huggingface.co/docs/api-inference
[7] https://huggingface.co/google/gemma-3-27b-it
[8] https://docs.anthropic.com
[9] https://docs.llamaindex.ai

Table 2: Conceptual design evaluation dimensions rated on a five-point Likert scale.

Dimension	**Statement**
Role separation	Whether splitting the work into four agents (MRS analysis, MRS review, FS generation, FS review) is a sensible decomposition.
Review symmetry	Whether having a dedicated Review Agent for both MRS and FS improves quality and catches errors earlier.
HITL placement	Whether the two HITL checkpoints are placed at the right points in the workflow.
RAG knowledge base	Whether sharing a RAG knowledge base across all agents is a reasonable design choice.
Severity-tagged feedback	Whether categorizing review findings as Critical, Medium or Low is useful.
Loop limit	Whether limiting the retry loop to three attempts before escalating to a human is appropriate.
Anti-fabrication policy	Whether marking unknown content with `[TBD]`/`[ASSUMED]` rather than inventing values is essential for industrial use.
Industrial fit	Whether the overall design fits how Westermo actually develops MRS and FS documents.
Practical deployment	Whether the architecture is realistic to deploy in a real engineering team.

Functional Specification (prototype output) evaluation. The second form looks at an FS produced by the prototype (see Appendix F), taking that output as the demonstration deliverable of the DSR artifact. Whether the prototype produces documents of industrially useful quality is the question asked. Participants rate the five quality dimensions listed in Table 3.

The five are drawn from established Requirements Engineering quality criteria. A single reviewer can therefore judge the FS without needing deep familiarity with the agent pipeline itself. The dimensions are Completeness, Correctness, Clarity, Implementability, and Security coverage (IEC 62443-4-2 obligations addressed with traceable *Functional Requirements (FRs)*).

Table 3: Functional Specification (prototype output) evaluation dimensions rated on a five-point Likert scale.

Dimension	**Statement**
Completeness	Whether the FS covers all sections and requirements expected of a Westermo FS.
Correctness	Whether the technical content is accurate and consistent with the source MRS.
Clarity	Whether a software engineer could implement the requirements without further clarification.
Implementability	Whether the specification is realistic to implement on Westermo hardware/software within reasonable effort.
Security coverage	Whether all relevant IEC 62443-4-2 obligations are addressed with appropriate FRs and traceability tags.

An overall suitable-for-release dimension closes the rated section. Four free-text questions cover strengths, weaknesses, comparison to a human-written FS, and other comments.

5.4.2 Comparative Model Evaluation

The five models were not chosen to rank them against each other. Differences in size and architecture make a head-to-head ranking uninformative on its own. It is unsurprising that a ∼3.8B-parameter model produces a thinner FS than a frontier cloud model. The selection criterion was instead deployment coverage.

The five models are tested on the same task, producing an FS from a reference MRS. One model is reached through a commercial cloud API, and four run locally through Ollama. Hugging Face Inference API was also part of the plan, but had to be dropped from the analysis after the requests kept failing. Each model gets the same input, the same prompts, the same RAG context, and one generation attempt. Resulting documents are measured on document length, the count

of FRs, how many of the twelve template sections are present, and the semantic similarity to the source MRS.

5.4.3 Semantic Similarity Scoring

Semantic similarity is computed between the generated Functional Specification and the source Market Requirements Specification, so that the comparison can be put into numbers. The embeddings come from the `all-MiniLM-L6-v2` model. This follows the approach of Sami et al. [3], who applied the same metric to compare agent outputs in a multi-agent Requirements Engineering pipeline. Using a comparable metric makes the results of this study easy to relate back to that earlier work.

While the metric captures topical overlap between vocabularies, it says nothing about factual correctness. A document that uses the right words in the wrong combinations can still score high. For that reason, it is reported next to the structural-completeness and FR-count metrics rather than read on its own.

5.4.4 Fault-Injection Testing

A faulty version of the reference MRS is constructed by introducing ten deliberate errors. The ten faults span the five categories listed in Table 4: ambiguity, unmeasurable requirement, gap, contradiction, and security gap.

Table 4: The five fault categories from which the ten injected faults are drawn.

Category	**Fault type**
Ambiguity	Replacing a concrete value with vague terminology.
Unmeasurable requirements	Replacing a measurable parameter with an unmeasurable phrase.
Gap	Removing a required statement (e.g. a default-value declaration).
Contradiction	Inserting a statement that conflicts with an existing parameter.
Security gap	Removing references to specific security controls.

Table 13 enumerates the exact set of injected faults. Appendix E reproduces the fault-injected input document itself. The faulty document is passed through the full pipeline using Claude Opus 4.7 as the underlying model. Each run is evaluated on the two dimensions listed in Table 5.

Table 5: Evaluation dimensions for each fault-injection run.

Metric	**Description**
Detection rate	The proportion of the ten injected faults flagged by the Review Agents. A fault is counted as detected when a Review Agent's output explicitly references the requirement identifier or content affected by the fault.
Semantic similarity score	Expected to degrade on the faulty input relative to the perfect input, providing an indirect measure of input-quality sensitivity.

The fault-injection experiment is repeated across five independent runs on the same fault-injected MRS, holding all configuration parameters constant.

6 Ethical and Societal Considerations

The qualitative conversations were open and exploratory, and were written up as internal meeting notes. No personal or sensitive data was collected. No individual opinion can be tied to a named person in the report. Likert ratings and free-text comments are reported as group totals. Any quotes are tied to a role rather than to a name. Participants can decline or withdraw at any point, with no consequences.

A non-disclosure agreement between the author and Westermo Network Technologies AB governs how confidential material is handled. The *Market Requirement Specification (MRS)* and *Functional Specification (FS)* templates used in this thesis are made up. They follow the structure of the real Westermo templates, but without copying any confidential content. The same is true for the example MRS, the generic software-process document, and the security-guidelines document used as input to the prototype.

Use of *Artificial Intelligence (AI)* is stated openly throughout the report. Prototype code was generated with the help of two *Large Language Models (LLMs)*, Claude and ChatGPT, used alternately during development. Its role within the Design Science Research methodology is to demonstrate the artifact (Activity 4). After generation, the author read through and adjusted the generated code by hand where needed. The author therefore understands the codebase in full and is responsible for its current state. All AI-written specification content passes through the mandatory *Human-In-The-Loop (HITL)* checkpoints before it counts as a valid artifact in the requirements chain.

The proposed pipeline is designed for use in a high-risk industrial context as defined by the EU AI Act [5]. The mandatory HITL checkpoints and the review checks are direct responses to the Act's requirements.

Automating the MRS-to-FS transition can reduce the manual work in *Requirements Engineering (RE)*. The design aims to support the *Product Manager (PM)* and *Product Owner (PO)*. The HITL checkpoints assume that human experts stay accountable for the resulting specifications.

The prototype supports both cloud and local models. Local use is built into the design from the start, which takes away the risk that private specifications are sent to outside providers. That is a real societal benefit on top of the efficiency picture, for industries that work under strict confidentiality and security rules.

LLM outputs can contain made-up facts or biases which spread further if they are not caught early. Fault-injection results show that defects which appear in the text are caught reliably, while missing content is not. Human-In-The-Loop checkpoints are crucial here, as they minimize the risk when a competent human expert is in the loop.

Two more societal angles matter here. The first is environmental. Training a frontier LLM uses electricity and water at a scale that small inference calls do not approach. Jegham et al. benchmarked 30 commercial LLMs [17]. The most energy-intensive ones use more than 29 Wh per long prompt, about 65 times more than the most efficient. At 700 million queries per day, even a 0.42 Wh short query adds up to enough drinking water for around 1.2 million people.

Second, automating drafting work creates a de-skilling risk for the engineers who used to write the artifacts by hand [18]. Product Manager and Product Owner are not only writers. They are also reviewers, decision-makers, and accountability holders. Writing a precise FS is the same skill that lets a reviewer catch a bad FS. The long-term consequence is a workforce that no longer notices when the AI gets it wrong. Mandatory HITL checkpoints in this design are a partial countermeasure, but only if the Human-In-The-Loop is given real authority.

7 Design and Implementation

This chapter describes the conceptual design of a multi-agent pipeline that turns a *Market Requirement Specification (MRS)* into a *Functional Specification (FS)*. It then covers the prototype that puts the design into practice for the *Design Science Research (DSR)* demonstration step.

7.1 Conceptual Design

The design proposed here is a novel four-agent pipeline that mirrors Westermo Network Technologies AB's own three-tier requirements process, shown in Figure 1, with one specialized *Artificial Intelligence (AI)* role per stage. The pipeline uses a four-agent layout with two *Human-In-The-Loop (HITL)* checkpoints, shown in Figure 3. Each agent has its own clear responsibility, and all four share access to the same *Retrieval-Augmented Generation (RAG)* knowledge base.

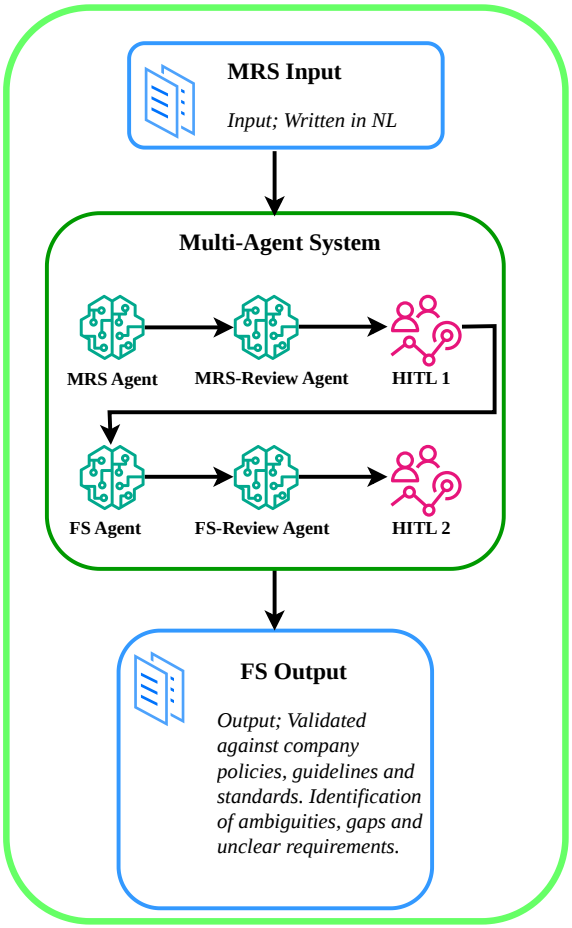


Figure 3: High-level conceptual architecture of the proposed *Multi-Agent System (MAS)* pipeline.

7.1.1 Symmetric Review Architecture

A core structural choice in this pipeline is where review happens. Earlier multi-agent *Requirements Engineering (RE)* frameworks tend to put review at the very end as a single block. MARE concentrates the verification into a single Checker Agent [2], and the Sami et al. pipeline relies on a *Quality Assurance (QA)* Agent at the end of the user-story generation chain [3]. Review is cheap compared to letting a bad artifact propagate further down the chain. A dedicated Review Agent is therefore placed after each generative phase, not only at the end. The resulting flow is: MRS Agent → MRS-Review Agent → HITL 1 → FS Agent → FS-Review Agent → HITL 2 (Figure 4).

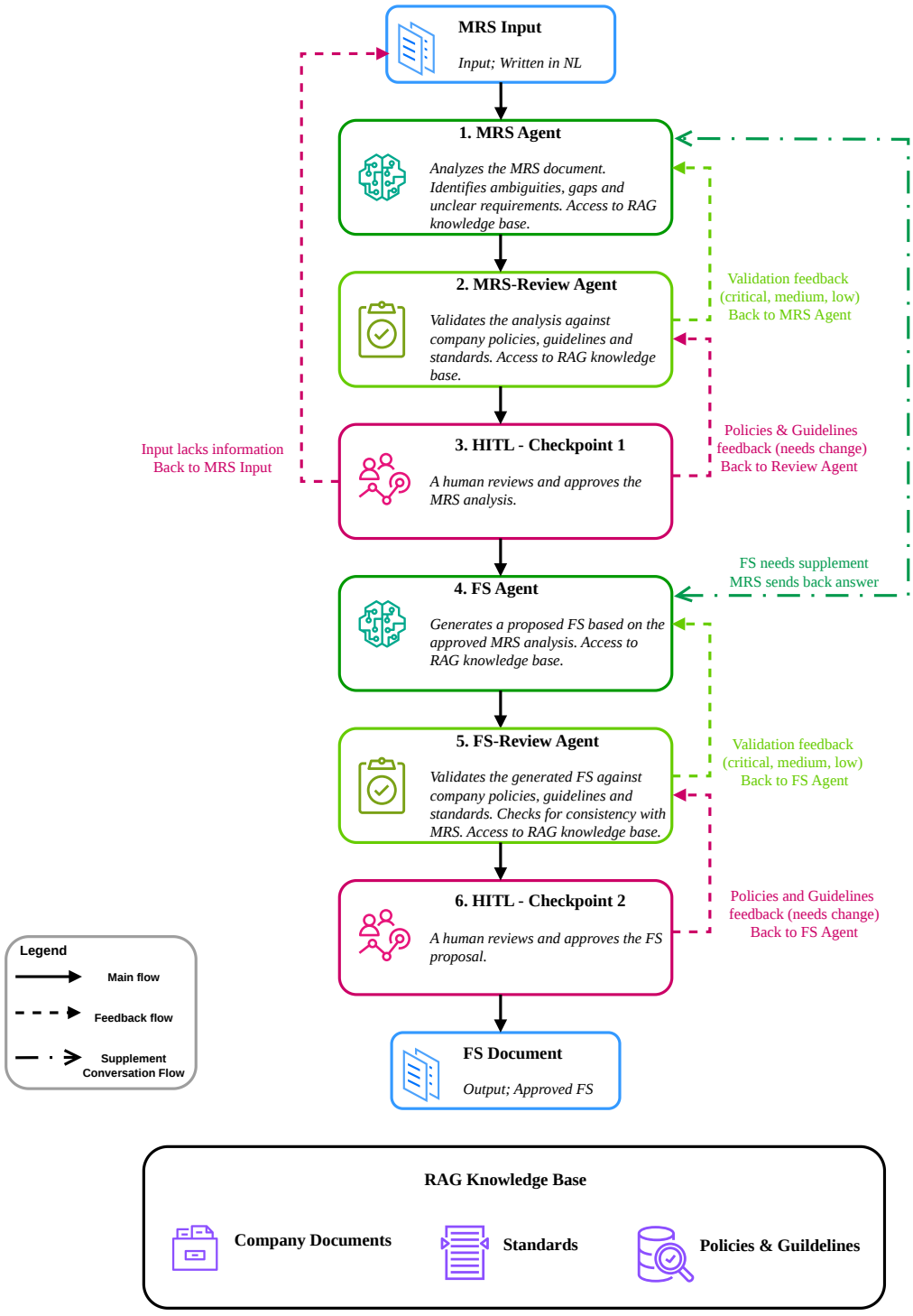


Figure 4: Detailed view of the symmetric review architecture.

Symmetry helps in two ways. Each phase's output is validated before it becomes the input of the next. That stops low-quality artifacts from carrying further down the pipeline, and keeps the traceability chain (Requirement → Code → Test) that industrial safety standards require. Human authority is also used twice on the way from MRS to FS, instead of only at a single end-of-pipeline gate. That matches what the EU AI Act asks for in the way of real human supervision [5].

7.1.2 Pipeline Components

Agents. Each of the four agents takes over one part of the manual handoff between the top two tiers of Westermo's requirements chain, shown in Figure 1. Each agent's task described below is based on the resulting flow seen in Figure 4.

The **MRS Agent** reads the incoming MRS and produces three artifacts. First, an analysis document. It lists every problem found in the input with each tagged Critical, Medium, or Low. The MRS Agent's prompt specifies five problem categories to look for: ambiguities, gaps, contradictions, unmeasurable requirements, and security gaps. Second, a corrected and complete MRS, based on the template hard-coded into the agent's system prompt. Ambiguous values are replaced with concrete ones where possible. Gaps in the document structure are filled. Contradictions are reconciled. Information that is genuinely missing is marked `[TBD]` or `[ASSUMED]` rather than fabricated. Third, the agent writes log entries to the shared session log. Security findings are grounded in IEC 62443-4-2 content from the shared RAG knowledge base.

The **MRS-Review Agent** validates the analysis and the corrected MRS independently. It checks them against the hard-coded MRS template, the company policies, and IEC 62443-4-2 content from the knowledge base. Each finding is tagged Critical, Medium, or Low. The result is written to the shared session log. If issues are found, the work goes back to the MRS Agent for revision. Otherwise it moves on to the first HITL checkpoint.

The **FS Agent** produces a full FS from the approved corrected MRS. It follows the twelve sections of the FS template hard-coded into its system prompt. Security requirements are grounded in IEC 62443-4-2 content from the knowledge base. The FS Agent also writes log entries to the shared session log.

The **FS-Review Agent** reviews the generated FS in the same way MRS-Review Agent does. It checks that every FS requirement traces back to the approved corrected MRS. It also checks that the FS structure matches the hard-coded template in the agent's system prompt, and that security obligations are met against IEC 62443-4-2 content. Findings and log entries are written to the shared session log.

Table 6 summarizes the four agents and the actions each one carries out, in the same style as MARE's role definition table [2].

Table 6: Role definition of the four agents in the proposed pipeline.

Agent	Role	Goal	Action
MRS Agent	An experienced market requirements analyst and editor with domain knowledge of industrial networking and embedded communication equipment	Produce an analysis of every problem in the input MRS, plus a corrected, template-complete MRS where the problems are resolved or marked `[TBD]`/`[ASSUMED]`	a. *analyze* b. *rewrite* c. *supplement*
MRS-Review Agent	An experienced requirements reviewer for industrial networking products	Validate the analysis and the corrected MRS against the MRS template, the company policies, and IEC 62443-4-2 content. Tag findings by severity	a. *review*
FS Agent	An experienced functional specification writer for embedded networking systems	Generate a complete FS from the approved corrected MRS, following the twelve template sections. Refine on review feedback	a. *generate* b. *patch*
FS-Review Agent	An experienced FS reviewer for embedded networking systems	Run six gates (structure, requirement quality, configuration, behavior, security, assumptions) over the generated FS. Tag findings by severity	a. *review*

Each agent owns one phase, so no single agent has to carry analysis, review, and synthesis at the same time. Pairing one reviewer with each generator is the part that goes beyond MARE's single-Checker line-up [2].

HITL checkpoints. Two HITL checkpoints give explicit human oversight. At HITL 1, a domain expert, such as *Product Manager (PM)*, reviews the analysis and the corrected MRS. Four options are available: approve the work, reject it with feedback, edit the source MRS and restart the pipeline, or end the run. At HITL 2, an expert, such as *Product Owner (PO)*, reviews the generated FS, with similar options mentioned for HITL1. Both checkpoints stop agents from making unsupervised decisions on critical content. Each HITL decision is recorded in the session log.

Output artifacts. Every agent writes its work to disk so the full run can be reconstructed later. Six artifact files are produced per run, listed in Table 7. Every agent also writes structured log entries to a single per-session log file. Each entry carries a timestamp, the agent name, the event type, and a free-form detail payload. The session log binds every agent's work, every HITL decision, and every artifact write into one chronological record.

Table 7: Output artifacts produced by each agent.

Agent	Artifact (file)	Content
MRS Agent	`mrs_analysis.md`	Structured analysis listing every problem in the input MRS (ambiguities, gaps, contradictions, unmeasurable requirements, security gaps), each tagged Critical, Medium or Low.
	`mrs_corrected.md`	Corrected, template-complete MRS with problems resolved. Missing information is marked `[TBD]` or `[ASSUMED]` rather than fabricated.
MRS Review Agent	Review verdict	`[APPROVED]` (pipeline continues to HITL 1) or a list of severity-tagged findings sent back to the MRS Agent for revision.
FS Agent	`fs_document.md`	Generated FS across the twelve template sections, with traceability tags linking each FR back to the corrected MRS. One version per refinement iteration.
FS Review Agent	`fs_review_feedback.md`	Critical and Medium findings from Gates 1–6 (structure, requirement quality, configuration, behavior, security, assumptions) that trigger FS refinement.
	`fs_review_advisory.md`	Low-severity findings, passed through with an audit note rather than triggering refinement.

RAG knowledge base. All four agents share the same retrieval index (see Appendix B), which holds two documents. One is a cybersecurity guide grounded in IEC 62443-4-2. It gives the agents the seven Foundational Requirements and the Component Requirements they tag findings against. The other is a generic software-process reference. It covers MRS and FS quality criteria, MoSCoW prioritization (MUST/SHOULD/COULD), and common defect categories. It was written for this thesis following the structure of the company's real process documents, without confidential content. Document templates are hard-coded into each agent's system prompt, so they are always present in the context window. Retrieval against the index runs locally, and retrieved content is then included in the prompt sent to the underlying LLM.

7.1.3 Theoretical Framework

The design rests on the *Multi-Agent System (MAS) Requirements Engineering (RE)* (MARE) paradigm [2]. A traditionally manual process can be broken down into distinct cognitive tasks, each one a good fit for a specialized software agent. MARE calls out five core architectural properties such a system is expected to have. They are listed in Table 8: role specialization, iterative refinement, a shared knowledge base, quality gates, and human supervision.

Table 8: Core architectural properties of a MARE-style system.

Property	**Description**
Role specialization	Each agent owns one phase, avoiding single-agent overload.
Iterative refinement	Agents revise their output based on feedback from other agents.
Shared knowledge base	A common store of standards and prior specifications, used by every agent to prevent drift.
Quality gates	Explicit validation between phases stops low-quality output from carrying downstream.
Human supervision	Human checkpoints keep control over critical decisions.

Each of these MARE properties has a direct realization in the present design. Role specialization is realized through the four distinct agents. Iterative refinement runs as a bounded loop with up to three retries before human handover. The shared knowledge base is a single RAG store holding the cybersecurity guide (IEC 62443-4-1/4-2) and a generic software-process reference, queried by all four agents. Quality gates are realized as the six FS gates (structure, requirement quality, configuration, behavioral model, security, assumptions). Human supervision is realized as two HITL checkpoints, one after MRS and one after FS. At each, the options are to approve, reject with feedback, edit and restart, or end the run.

Table 9: Mapping of MARE architectural properties to the pipeline.

MARE property	**Realization in this pipeline**
Role specialization	Four distinct agents: MRS Agent (analyze + rewrite), MRS Review Agent (validate), FS Agent (write FS), FS Review Agent (validate).
Iterative refinement	Review agents send feedback back to their generators. Up to three retries before human handover.
Shared knowledge base	A single RAG store holding the cybersecurity guide (IEC 62443-4-1/4-2) and a generic software-process reference, queried by all four agents. Templates are hard-coded into system prompts, not retrieved.
Quality gates	Six FS gates: structure, requirement quality, configuration, behavioral model, security, assumptions.
Human supervision	Two HITL checkpoints (after MRS, after FS). Options: approve, reject with feedback, edit source MRS and restart, or end the run.

The design follows the MARE paradigm closely. The two extensions on top of it are severity-tagged review output and a Review Agent after every generative phase. Each review finding is tagged Critical, Medium, or Low. Blocking issues kick off automatic refinement, advisory findings pass through with a note in the audit log. This keeps MARE's iterative-refinement idea alive while stopping endless review loops on cosmetic issues.

7.2 Prototype Implementation

A working prototype was built to show that the conceptual design above can be put into practice. The prototype provides the executable evidence base that the comparative model study, the fault-injection study, and the expert evaluation rely on. The code was generated with the help of two LLMs, Claude and ChatGPT, used alternately as suited the task. Every generated file was then read through, reviewed, and adjusted by hand where needed. This included prompt-tuning, gate logic, control-flow corrections, and bug fixes.

Pipeline orchestration. The prototype follows the four-agent flow shown in Figure 4, where each generative agent is followed by a Review Agent and a Human-In-The-Loop checkpoint. A bounded refinement loop runs between each generator–reviewer pair. It retries up to three times before any remaining `[CRITICAL]` finding is sent on to the human checkpoint. Sometimes the FS Agent finds that the approved corrected MRS is missing information that it cannot reasonably assume. In that case, a supplement-conversation flow sends the gap back to the MRS Agent and re-runs FS generation. All artifacts are written to disk along with a structured session log to which every agent contributes. The full control flow is given as pseudocode in Appendix C.

Deployment configurations. Without any code changes, the prototype runs against the three deployment backends listed in Table 1. Exactly the same agent prompts and RAG context are used in all three setups. As a result, the comparative model study is a fair comparison of model capability. Built once with a local embedding model and reloaded on later runs, the shared RAG knowledge base lets embedding and retrieval always run locally.

Test artifacts. Two MRS documents are used as input across all the experiments. `MRS_perfect.docx` (Appendix D) is a reference MRS that describes a network storm control feature in full and follows the company MRS template.

Storm control is a Layer-2 networking feature that monitors the rate of broadcast, multicast, or unknown-unicast traffic arriving on each switch port. When the rate exceeds a configured threshold (e.g. frames-per-second), the switch acts to limit the impact. One misbehaving device cannot then stall the wider network. Storm control was chosen as the test case for two reasons. First, it covers the full breadth of what a FS has to address: configurable parameters, measurable behavior, error and recovery paths. Second, it is well-scoped to one feature and one tier transition. That keeps the comparative-model and fault-injection studies tractable.

The document is written by the author and follows the company MRS template, with confidential customer detail kept out per the NDA. `MRS_perfect_with_faults.docx` (Appendix E) is a copy of the same document with ten deliberate faults injected.

7.3 Experiment Setup

This section describes each experiment. All three experiments share the same four-agent pipeline, the same agent prompts (Appendix A), the same RAG knowledge base (Appendix B), and the same deterministic orchestration logic (Appendix C). What differs between experiments is the input MRS, the LLM backend, the number of repetitions, and the metrics computed on the output.

7.3.1 Comparative Model Evaluation

Purpose is to measure how the choice of LLM backend affects the quality of the generated FS, holding everything else constant. Five LLM backends are configured: Claude Opus 4.7 through the Anthropic cloud API, and four local backends through Ollama (Llama 3 8B, Llama 2 7B, Mistral 7B, Phi-3 3.8B). Each backend is plugged into the same Python orchestrator with no code changes between runs. The same four agent prompts (Appendix A) are sent to each backend. The RAG knowledge base is built once with a local embedding model and reloaded for each run. Embedding and retrieval always run locally regardless of which inference backend is used.

Input is `MRS_perfect.docx` (Appendix D), the storm-control reference MRS. For each backend, one complete pipeline run is executed. The refinement loop is left enabled with a cap of three retries. Each run produces one FS artifact and a session log.

Outputs are five Functional Specifications, one per backend. The Claude Opus 4.7 output is included in Appendix F as the demonstration deliverable. The four local-backend outputs are summarized in Table 12.

For each generated FS the following metrics are computed:

- Document length (word count)
- Number of *Functional Requirements (FRs)* (FR-XXX entries)
- Number of template sections present out of the twelve defined in the FS template
- Traceability rate: percentage of FRs carrying a valid `[MRS-XXX]` or `[FRn-XXX]` tag
- Count of MRS Requirement Inventory items referenced
- Count of IEC 62443-4-2 Foundational Requirements addressed
- Semantic similarity between the FS and the source MRS, computed with `all-MiniLM-L6-v2` embeddings

7.3.2 Fault-Injection Testing

Purpose is to measure how reliably the Review Agents detect deliberately injected defects in the input MRS. A single backend, Claude Opus 4.7, is used for all runs. The same four agent prompts, the same RAG knowledge base, and the same orchestration logic are used as in the comparative study. Each run is independent, a fresh session log and fresh agent state per run.

Input is `MRS_perfect_with_faults.docx` (Appendix E), a copy of `MRS_perfect.docx` with ten deliberate faults injected. The faults span all five categories defined in Table 4: ambiguity, unmeasurable requirement, gap, contradiction, and security gap. The exact set of injected faults is enumerated in Table 13. Faulty MRS is passed through the full pipeline. The run is repeated *five times* on the same fault-injected MRS with all configuration parameters held constant.

Following metrics are computed:

- Detection rate per run: number of injected faults flagged out of ten
- Detection rate per fault category, aggregated across the five runs
- Semantic similarity score on the faulty input, compared to the same metric on `MRS_perfect.docx` as a sensitivity check

A fault is counted as detected when the MRS-Review Agent's output explicitly references the requirement identifier or content affected by the injected fault.

7.3.3 Expert Evaluation

Purpose is to get qualitative judgment from Westermo staff on whether the conceptual pipeline design is the right thing to build, and whether the prototype's output is industrially useful.

Two Likert forms are used, both on the same five-point scale (1 = strongly disagree, 5 = strongly agree). Both forms mix closed Likert items with free-text comments.

Input to reviewers:

- Conceptual design form: the proposed four-agent architecture, Figures 4
- Functional Specification form: the FS produced by Claude Opus 4.7 in the comparative model study (Appendix F), and the input MRS (Appendix D)

The forms were distributed to two Westermo reviewers: a Project Manager and a Software Development Manager. Reviewers completed the forms independently. Likert ratings and free-text comments were collected without follow-up clarification.

8 Results

This chapter goes through the results and findings of this thesis: qualitative findings that motivated the design, the expert evaluation, the side-by-side comparison of five *Large Language Model (LLM)* backends, the fault-injection experiment, and the threats to validity.

8.1 Findings from Qualitative Conversations

The conversations with Westermo Network Technologies AB's leading technical roles pointed to one area as the place where *Artificial Intelligence (AI)* support would help most, *Requirements Engineering (RE)*. Within RE, the three-tier requirements chain has two manual steps that came up as concrete bottlenecks.

MRS-to-FS. Today, moving from the *Market Requirement Specification (MRS)* to the *Functional Specification (FS)* is a manual step, governed by a formal review process that ensures the resulting FS is correct. It is reliable but time-consuming, and the writing effort scales with each new feature. There is no automated trace tying a market requirement to its functional counterpart in the FS. AI support is sought here as a way to speed up the drafting work without giving up the formal review that keeps the output correct.

FS-to-SR. A similar challenge shows up at the next step in the chain. Complex *Functional Requirements (FRs)* have to be broken down into atomic, testable Software Requirements. The task is detailed and time-consuming, and depends heavily on individual expertise. The FS is the only source to check against, so each requirement needs careful review by hand.

These two findings shaped the design of the artifact. The primary focus was put on the MRS-to-FS handoff for two reasons. It sits earliest in the chain, so any defect caught there cannot spread further. And both endpoints (MRS, FS) are template-bound documents, which is what the artifact needs to be grounded. Figure 5 sets the current manual workflow side by side with the AI-augmented pipeline proposed in this thesis.

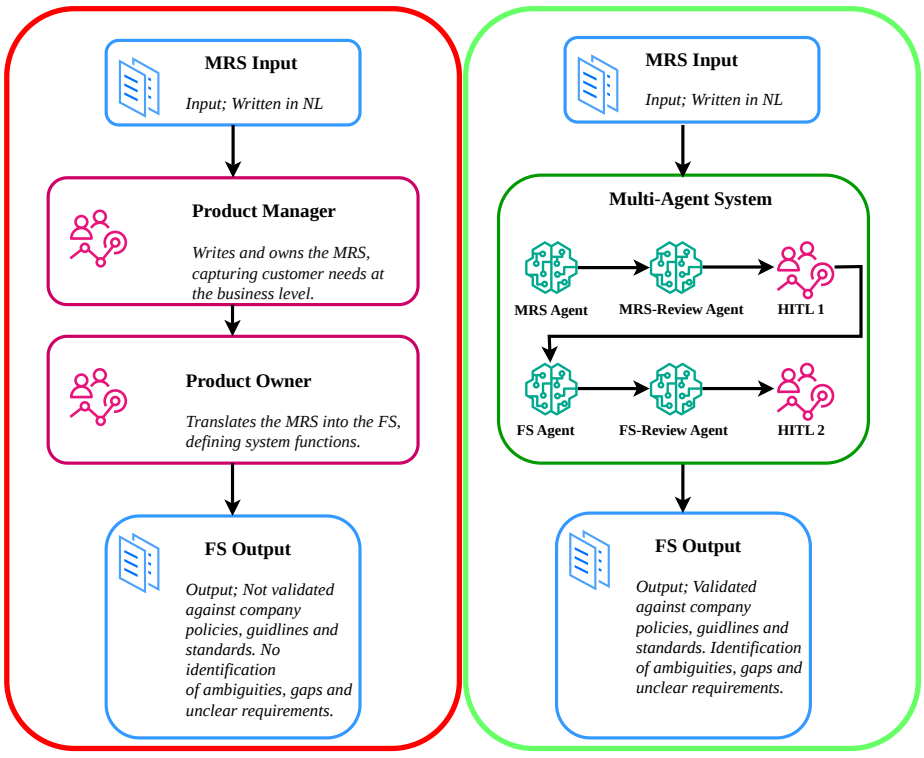


Figure 5: Left shows the current manual MRS-to-FS workflow. Right shows the *Multi-Agent System (MAS)* pipeline proposed in this thesis.

8.2 Expert Evaluation

Two Likert forms were used: the first for the conceptual design of the multi-agent pipeline, and the second for an FS produced by the prototype as the *Design Science Research (DSR)* demonstration deliverable of the artifact.

8.2.1 Conceptual Design Evaluation

The conceptual design was rated by two reviewers at Westermo: a Project Manager and a Software Development Manager. Ratings are on a five-point Likert scale (1 = Strongly disagree, 5 = Strongly agree). Table 10 lists the rating per dimension from both reviewers.

Table 10: Conceptual design evaluation. Mgr = Manager, SW Dev. = Software Development

Dimension	**Project Mgr**	**SW Dev. Mgr**	**Median**
Role separation	4	5	4.5
Review symmetry	5	5	5
HITL placement	3	5	4
RAG knowledge base	3.5	5	4.25
Severity-tagged feedback	5	3	4
Loop limit (3 retries)	4	4	4
Anti-fabrication policy	5	4	4.5
Industrial fit	—	5	5
Practical deployment	4	3	3.5
Overall recommendation	**4**	**3**	**3.5**

The Project Manager deferred Industrial fit to the Software Development Manager and is therefore marked "—" on that row. The Software Development Manager rated all nine dimensions.

Both reviewers gave the highest score (5) on Review symmetry. They also rated Loop limit at 4, Role separation high (4 and 5), and *Human-In-The-Loop (HITL)* placement and *Retrieval-Augmented Generation (RAG)* knowledge base on the positive side overall. Both want the design taken forward as a proof-of-concept and iterated on internally.

Severity-tagged feedback (Project Manager: 5 vs. Software Development Manager: 3) and Practical deployment (4 vs. 3) show the clearest differences. On severity tagging, the Project Manager emphasized it as necessary in practice to avoid endless review loops on low-priority findings; the Software Development Manager rated it neutrally without elaboration. On HITL placement, the Project Manager rated it at 3, arguing for a Human-In-The-Loop at every step rather than only at two checkpoints; the Software Development Manager rated the same dimension at 5.

Free-text synthesis. The Software Development Manager identified the four-agent workflow as the strongest aspect, since it follows Westermo's existing process. The design was judged as worth carrying forward and would keep it for further evaluation, with the next focus being on improving the input data and the resulting output quality. On the generated FS itself, it was found not yet specific enough or fully correct to use as a base for development, a matter of input data and context, not the design itself.

The Project Manager added a complementary view. Noted was that the design depends on many interdependent technical decisions (RAG, model choice, etc.) that risk analysis paralysis, and suggested a simple "good enough" threshold to avoid it. Two missing components were flagged: an audit log that lets the team trace which agent took which decision when, and guard-rails against toxic or off-policy output. Also noted was that the human roles around the pipeline (requester, developer, system architect) have different views of what a good requirement is. These roles sit at different points in the requirement lifecycle, which the current design does not differentiate. Deployment concerns were data confidentiality, regulatory compliance that may require humans in specific roles, and ownership of the AI tooling once it is in production.

8.2.2 Functional Specification (prototype output) Evaluation

The FS rated here is the Claude Opus 4.7 output from the comparative model study. The reviewer is a Software Development Manager at Westermo. Ratings are on the same five-point Likert scale (1 = Strongly disagree, 5 = Strongly agree). Table 11 lists the rating per dimension.

Table 11: FS (prototype output) evaluation.

Dimension	Rating	Comment
Completeness	4	The FS covers the expected sections; nothing essential is missing.
Correctness	3	The technical content is partly accurate, partly off; the reviewer attributes the inaccuracies to limited internal data being available to the agents.
Clarity	2	Requirements are too general and not specific enough to be implemented without further clarification.
Implementability	2	Not realistic to implement on Westermo hardware in its current form without further work on the inputs and agent instructions.
Security coverage	2	The reviewer notes that this is not actually expected to be covered by the FS at Westermo, security is managed in other requirements and processes.
Suitable for release	**2**	Not ready to be released to an implementation team in its current state; needs further internal iteration first.

Free-text synthesis. FS was described as containing too much general text and not being specific enough to function as a requirement document on its own. The reviewer attributed this to the input data available to the agents rather than to the pipeline design. Agents were operating without the depth of Westermo-internal context that any first FS draft would normally rely on. With richer input data and more specific agent instructions, the reviewer expected the same pipeline to produce a much more useful draft. Despite the current limitations, the result was described as "a really good and valuable proof-of-concept" and "something we can use as a base for further work". Recommended next steps are to enrich the input data and put the pipeline in front of real Developers and Product Owners as the Human-In-The-Loop.

Interpreting the FS ratings. A Suitable-for-release rating of 2 has to be read against how the question was framed: can this FS be given to a team to work with? The follow-up made clear that a higher rating was not reasonable at this stage, because the agents were never given the depth of internal Westermo context that a release-ready FS would need. Overall research outcome was judged as a successful proof-of-concept and a valuable input to Westermo's own further work. This is consistent with the high ratings a Project Manager gave on the conceptual design.

8.3 Comparative Model Evaluation

The same FS-generation task, producing a FS from `MRS_perfect.docx`, was executed across five models under identical input, prompts, and RAG context.

8.3.1 Comparison Table

Table 12: Comparison of five LLMs on the same FS task. Sim. = Semantic similarity.

Model	Hosting	Words	FRs	Sections	Traced	[MRS]	IEC	Sim.
Claude Opus 4.7	Cloud (Anthropic)	9 988	31	12/12	100%	26	5	0.731
Llama 3 (8B)	Local (Ollama)	2 216	14	12/12	100%	9	3	0.721
Mistral (7B)	Local (Ollama)	2 788	0	12/12	—	17	5	0.752
Llama 2 (7B)	Local (Ollama)	1 807	4	10/12	0%	0	0	0.431
Phi-3 (3.8B)	Local (Ollama)	5 765	0	3/12	—	0	0	0.073

Table 12 shows that the choice of backend matters less than whether the backend is large enough to follow the template at all. Claude Opus 4.7 produced the deepest specification: 31 functional requirements across all twelve template sections, every one traced back to the MRS Requirement Inventory. Llama 3 8B, running fully on local hardware, closed the gap to a single percentage point on semantic similarity (0.721 vs. 0.731), and also covered all twelve sections with full traceability. The three smaller local models did not reach this baseline. Mistral 7B and Phi-3 3.8B returned zero FRs in the expected `FR-XXX` format. Llama 2 7B produced four FRs with no traceability tags at all. Semantic similarity alone is misleading here. Mistral has the highest similarity in the study (0.752) but no usable requirements, so the metric has to be read alongside the structural columns rather than on its own.

8.3.2 Quality Analysis

Claude Opus 4.7 produced the deepest specification, with 31 atomic functional requirements, full template coverage, and 100% traceability between the FRs and the MRS Requirement Inventory.

Llama 3 (8B) was close behind on the structural metrics, 12/12 sections, 100% traced FRs, and a semantic similarity of 0.721. Even though it has about a tenth of Opus's parameter count, Llama 3 still produced a structurally complete document.

Mistral (7B) reached the highest semantic similarity score in the study, 0.752, but returned no requirements in the FR-XXX format expected by the template. What it produced was requirement-like prose without numbering, which would be rejected by the FS-Review Agent's traceability check. Semantic similarity on its own is not a good enough quality metric. A document can resemble its source in vocabulary while breaking the structural rules.

Llama 2 (7B) produced only four FRs, none of them with traceability tags (similarity 0.431).

Phi-3 (3.8B) collapsed almost completely. It included three sections, no FRs, and a similarity score of 0.073. This is a sign of context rot. The model lost track of the long, structured instructions early in the generation flow. It wrote about unrelated subjects by the time it reached the heaviest section. Small parameter counts and shorter effective context windows make this failure mode more likely.

8.3.3 Security and Confidentiality Considerations

Cloud-hosted models (Claude Opus 4.7, and the dropped Hugging Face models) need the full MRS to be sent to a third-party inference provider. These providers usually promise not to train on submitted data, but the document still leaves the organization. *Local models* (Llama, Mistral, Phi-3 via Ollama) keep all input and output inside the local machine. No MRS ever leaves the organization. This lines up directly with the confidentiality rules of industrial customers.

If specification depth is the priority, Claude Opus 4.7 is the strongest backbone in the study (31 FRs, full traceability, 9 988-word output). The cost is that every MRS that goes through it has to be sent to Anthropic. If keeping data confidential on a standard workstation matters more, Llama 3 (8B) is the local backbone to use. It produced a template-complete, fully-traced FS with semantic similarity almost on par with the cloud baseline (0.721 vs. 0.731), without any document ever leaving the organization. The smaller local backbones (Llama 2 7B, Phi-3 3.8B) are not usable for end-to-end FS generation in this pipeline. They may still suit narrower sub-tasks such as keyword extraction or summarization, where format compliance and depth matter less.

8.4 Fault-Injection Testing

The fault-injection study tests how reliably the Review Agents flag deliberately introduced defects. Ten faults are injected into the reference MRS, spanning the five categories defined in Table 4: ambiguity, unmeasurable, gap, contradiction, and security gap. Section 8.4.1 lists the exact ten faults. Section 8.4.2 reports detection across five pipeline runs.

8.4.1 Injected Faults

Table 13 lists the ten faults. All ten are present at the same time in a single file used as input for the fault-injection experiment, which is run on Claude Opus 4.7.

Table 13: The ten injected faults organized by category and location in the source document.

ID	Category	Location	Injected fault
F01	Ambiguity	[MRS-002]	"threshold 500 (fps)" → "an appropriate threshold"
F02	Unmeasurable	[MRS-001]	"measured in frames per second (fps)" → "with appropriate monitoring"
F03	Gap	[MRS-007]	Entire default-value statement (300 s, 0 = manual re-enable) deleted
F04	Contradiction	[MRS-006]	Added: "shall automatically recover within 10 seconds", which contradicts the persistence parameter
F05	Security Gap	Security Requirements	CR 2.08 (audit logging) removed
F06	Security Gap	Security Requirements	CR 1.01 / 1.03 (authentication) removed
F07	Contradiction	[MRS-005]	"shall" → "could", but still listed under MUST Requirements
F08	Ambiguity	Functional Overview	"300 seconds" default → "reasonable value"
F09	Gap	Known Issue	Entire #17845 paragraph (disabled-port enable bug) removed
F10	Contradiction	[MRS-009]	"shall not affect forwarding on unaffected ports" → "may affect forwarding on adjacent ports"

The ten faults are unevenly distributed across the five categories. Contradiction has the most representatives (F04, F07, F10), ambiguity and gap have two each (F01/F08 and F03/F09), unmeasurable has one (F02), and security gap has two (F05, F06). They are also placed at different points in the MRS structure. Six fall in numbered Functional Requirements (`[MRS-001]` through `[MRS-009]`), two in the Security Requirements section, one in the Functional Overview, and one in a Known Issue paragraph. The detection task is therefore not biased toward a single section of the document. Two Gap faults differ in kind. F03 removes a single in-line default-value statement that the MRS template requires. F09 removes an entire Known Issue paragraph that does not reappear anywhere else in the document.

8.4.2 Detection Results

Table 14: Fault detection matrix across five Claude Opus 4.7 pipeline runs on the same fault-injected MRS. Each cell shows whether a given fault (F01–F10) was detected (✓) or missed (×) in that run.

Run	F01	F02	F03	F04	F05	F06	F07	F08	F09	F10	Total
R1	✓	✓	✓	✓	×	×	✓	✓	✓	✓	8/10
R2	✓	✓	✓	✓	×	×	✓	✓	×	✓	7/10
R3	✓	✓	✓	✓	✓	✓	✓	✓	×	✓	9/10
R4	✓	✓	✓	✓	×	✓	✓	✓	×	✓	8/10
R5	✓	✓	✓	✓	✓	✓	✓	✓	×	✓	9/10
Detection rate	100%	100%	100%	100%	40%	60%	100%	100%	20%	100%	82%

Across five independent runs of the Opus 4.7 pipeline against the same fault-injected MRS, the system reached an average detection rate of 8.2 out of 10 (82 %). Seven of the ten faults were caught in all five runs (100 %): every ambiguity (F01, F08), the unmeasurable requirement (F02), every contradiction (F04, F07, F10), and the pattern-based gap (F03). The two security gaps fared worse. F05 removed an audit-logging *Component Requirement (CR)* and was caught in 2 of 5 runs (40 %). F06 removed two authentication CR and was caught in 3 of 5 runs (60 %). The absence-only gap F09, where the Known Issue paragraph was removed, was caught in only 1 of 5 runs (20 %). It is the single weakest result in the matrix. Pattern matching against the text is reliable, while reasoning about what is missing from the document needs an explicit Human-In-The-Loop.

8.5 Threats to Validity

The four validity aspects from Runeson et al.'s case-study framework [19] are used to structure the discussion: construct, internal, external, and reliability.

Construct validity. Semantic similarity is the most exposed metric. It captures topical overlap between the FS and MRS vocabulary, not factual correctness. A high-similarity FS can still hold wrong values. Mistral 7B (0.752 similarity, zero usable FRs) shows the limit. The metric is therefore reported alongside the structural columns, not on its own. Detection rate also leaves room for interpretation.

A fault counts as detected when the Review Agent's output references the requirement identifier or content affected by the injected fault. The metric does not separate a generic flag from a precise diagnosis.

The Likert dimensions on the FS form (Completeness, Correctness, Clarity, Implementability, Security coverage) are not weighted equally by every reviewer either. Security coverage rated 2 reflects question framing as much as a real gap, since at Westermo security obligations sit outside the FS.

Internal validity. LLM output is non-deterministic. The same MRS sent to the same backend with the same prompts can produce slightly different FS content between runs. The five-run fault-injection study reduces but does not remove that exposure. The refinement loop is a second source of noise. Each generator-reviewer pair retries up to three times, so a fault that is missed on iteration 1 but caught on iteration 3 looks like a successful detection rather than a recovery. The session log preserves which iteration the detection came from, so a reader can reconstruct the path.

External validity. The empirical results come from one feature (storm control), one MRS document, one company, and ten injected faults. The detection rates and similarity scores are tied to that combination and should not be read as population estimates.

The two Likert reviewers work for Westermo and have institutional incentive to be constructive. A more critical rater in a different organization might score lower.

Safety-critical domains rely on different standards, which would change the RAG knowledge base. The five LLM backends tested are those available at the time of writing. Models released after this work may shift the cloud-versus-local trade-off in either direction.

Reliability. The pipeline is deterministically routed and reproducible in principle. The agent prompts are reproduced in Appendix A, the RAG knowledge base in Appendix B, and the orchestration logic as pseudocode in Appendix C. The per-session log file works as an audit trail in the sense of Runeson et al.'s improving-validity guidance [19]: every agent call, HITL decision, refinement iteration, and artifact write is timestamped and tagged, so a later reader can reconstruct the chain of evidence. A second researcher could in principle re-run the same experiments.

9 Discussion

9.1 RQ1 — State of the Art in AI/LLM for Industrial RE

RQ1: What is the state of the art in applying AI and LLMs to support Requirements Engineering in the industrial product development process?

RQ1 is answered through the two-phase literature study described in Section 5.1. Research in this field has moved from single-prompt experiments to multi-agent frameworks. Directly comparable work is MARE [2] and the role-based pipeline of Sami et al. [3]. Nearby contributions cover retrieval-augmented question answering [4] and ambiguity classification [11]. Two recent *Systematic Literature Reviews (SLRs)* [1, 12] show that only a tiny share of published *Requirements Engineering (RE) Large Language Model (LLM)* studies have reached real use, and that almost all rely on cloud-hosted GPT-family backbones. Gaps emerging from this review are regulated human oversight, confidentiality of organizational data, and the absence of frameworks targeting the *Market Requirement Specification (MRS)* to *Functional Specification (FS)* step. Together, these gaps set the design goals for this work.

9.2 RQ2 — Westermo's Specific MRS-to-FS Needs

RQ2: What are the specific needs and challenges at Westermo regarding the MRS-to-FS transition in their product development process?

RQ2 is answered through the qualitative conversations described in Section 5.2. The conversations brought out two manual handoffs in the three-tier chain as the concrete bottlenecks: MRS-to-FS, and FS-to-*Software Requirements (SR)*. MRS-to-FS was chosen as the primary target for two reasons. Firstly, it sits earliest in the chain, so any defect caught there cannot spread further. Secondly, both endpoints are template-bound documents, which is what a structured generative approach needs to be grounded. FS-to-SR has the same shape, so it is a natural follow-up rather than a separate problem. These findings are tied to this one organization, but three-tier requirements chains with manual handoffs are common in safety-critical industries.

9.3 RQ3 — A Four-Agent MRS-to-FS Pipeline

RQ3: How can an AI-based support system be designed to meet these needs, in support of the leading technical roles?

RQ3 is answered through the *Design Science Research (DSR)* cycle described in Section 5.3. The artifact is a four-agent pipeline, backed by a shared *Retrieval-Augmented Generation (RAG)* knowledge base. Two mandatory *Human-in-the-Loop (HITL)* checkpoints are included as well. It builds on the MARE paradigm [2], but adds symmetric review with required human authority. MARE puts all the verification in a single autonomous Checker [2], and Sami et al. rely on a single *Quality Assurance (QA)* Agent at the end of the pipeline [3]. Neither one requires human involvement. The proposed pipeline puts a Review Agent after each generative phase, and uses human authority twice. That matches what the EU AI Act [5] asks for in the way of real oversight at critical decision points.

The hybrid deployment setup addresses the cloud-first pattern that both major SLRs flag as a barrier to adoption [1, 12].

Within the Design Science Research framework of Peffers et al. [6], this thesis delivers a designed artifact: the four-agent pipeline. A working prototype demonstrates it (DSR Activity 4). Production concerns, such as how the system holds up under load and the engineering work for an industrial roll-out, are outside the scope at this stage. Prototype code was generated with the help of two LLMs, with the author serving as architect and reviewer. After generation, the author read through the code, made manual adjustments where needed, and signed off on its current state. This is reported openly, and it does not change the conclusions about how the pipeline behaves. The underlying pattern is simple: turn a less structured document into a more structured one, guided by template, policy, and standard constraints, with human authority preserved. The same pattern fits more than Requirements Engineering. Legal contract drafting and clinical-note structuring work the same way.

9.4 RQ4 — Strengths, Limitations, and Risks

RQ4: What are the strengths, limitations, and risks associated with the proposed conceptual design and its prototype?

RQ4 is answered through three complementary evaluation strands: the expert review, the comparative model study, and the fault-injection experiment described in. The three strands are discussed below in the same order.

Model dependence. The comparative study shows one narrow gap: Claude Opus 4.7 (cloud, semantic similarity 0.731, 31 *Functional Requirements (FRs)*) sits one point above Llama 3 8B (local, 0.721, 14 FRs). Both produced a structurally complete, fully traceable FS. Phi-3, Llama 2, and Mistral did not follow the FS template at all. Mistral 7B is the cautionary case in the set. It scored the highest semantic similarity in the study (0.752), yet produced zero requirements in the expected `FR-XXX` format. Semantic similarity alone, picked up from Sami et al. [3], is necessary but nowhere near enough as a quality signal for generated specifications. Structural gates catch the failure mode that similarity alone hides. The cloud-versus-local gap therefore narrows to a single percentage point on semantic similarity. With the right structural gates around it, an on-prem model can get close to a frontier cloud model.

Fault detection. Across the ten injected faults, the pipeline reached an average detection rate of 8.2/10 (82 %). Defect classes split sharply: 100 % on pattern-based defects (ambiguity, contradiction, unmeasurability), 40–60 % on security gaps, and 20 % on the absence-based fault. This split is not random.

Three faults illustrate different failure modes: F05, F06, and F09. **F05** (40 %) was the deliberate removal of the *Critical Requirements (CR)* `CR 2.08` audit-logging requirement from the Security Requirements section. In principle the Review Agent could flag the missing CR via its IEC 62443-4-2 reference in RAG. But the absence has no textual fingerprint. Only an experienced reviewer who knows that `CR 2.08` normally belongs in this category recognizes it as missing. **F06** (60 %) was the removal of authentication requirements (`CR 1.01` / `1.03`). Detection here was slightly higher because authentication has more cross-references elsewhere in the MRS. The Review Agent sometimes reconstructs the gap from downstream mentions. In the two runs where it missed, the Agent fell back on a generic "Section appears complete" verdict. **F09** (20 %) is the hardest case in the set: an entire `Known Issue` paragraph was removed. It leaves no trace anywhere else in the document.

Pattern-based defects leave a visible trace in the text that the Review Agent can match against. Absence-based defects ask the agent to know what content should be present. A single pass over the document cannot supply that outside knowledge. This is where human involvement matters most. F09 is the clearest evidence in the study that an AI-only pipeline cannot be trusted to catch what is missing from a document. A domain expert who knows the product and customer history can spot missing-information cases that no review agent will ever find on its own. Human-In-The-Loop checkpoints already give such an expert a place to act, but F09 shows that this human step is not optional.

Choosing the LLM backend is not just about output quality. Large cloud models produce the deepest specifications but need every MRS to leave the organization. Local models keep all data inside, but output quality is restricted to what Llama 3 8B can produce. Claude Opus 4.7 is measurably better on absolute quality. Llama 3 8B is strong on structural completeness and full traceability. Hugging Face Inference API was dropped from the comparison because the requests kept failing.

Expert evaluation. Two Westermo reviewers rated the design: the Project Manager and the Software Development Manager. On the conceptual design, both gave Review symmetry the maximum (5) and rated Role separation high (4 and 5). The biggest disagreements were Severity-tagged feedback (5 vs. 3) and HITL placement (3 vs. 5). On the latter, the Project Manager wanted a Human-In-The-Loop at every step, while the Software Development Manager rated the proposed two checkpoints as well-placed.

Two items the Project Manager flagged as missing are already in the prototype. The per-session log file is the audit log described. Every agent call, HITL decision, and artifact write is timestamped and tagged, so a model or vendor can be swapped out after the fact. An anti-fabrication policy (`[TBD]` / `[ASSUMED]`, rated 5 by the same reviewer) covers part of the guard-rails role by stopping the agent from inventing values. Role differentiation remains a real gap and is left to future work.

The Software Development Manager rated the generated FS: Completeness 4, Correctness 3, Clarity 2, Implementability 2, Suitable-for-release 2. Follow-up made it clear that the FS was not expected to be release-ready in its current form. Agents lacked the depth of internal Westermo context that a release-ready FS would need. Results were still called a valuable proof-of-concept and a useful base for further work. Next steps were more internal data and HITL feedback. The Security-coverage 2 reflects question framing rather than a real gap: at Westermo, security obligations are managed in separate processes, not in the FS.

9.5 Generalizability Beyond Westermo

The empirical results were produced from one feature, one MRS document, and one company. The architectural and regulatory parts of the contribution travel further than that.

What likely generalizes. Multi-tier requirements chains with manual transitions between tiers are common across safety-critical software development. Automotive, rail, and energy infrastructure use layered models that resemble Westermo's MRS–FS–SR setup, often grounded in the V-model. Any organization with a Product Manager, Product Owner, and Developer split is a candidate adopter of the pipeline. The four-agent layout, the symmetric review setup, and the two HITL checkpoints are not tied to any specific standard or document template. Adapting the pipeline to a different organization would mean swapping the templates hard-coded into the agent prompts and the standard referenced in the RAG knowledge base, while keeping the orchestration logic intact. The cloud-versus-local confidentiality trade-off applies wherever specifications are covered by an NDA or trade-secret rules. The EU AI Act applies to all high-risk AI use in the EU, so the regulatory motivation for two HITL checkpoints holds across organizations in scope. The absence-detection gap visible in the F09 result is also organization-independent. It is a property of single-pass review against a document with no template-mandated section, so any review agent that operates on the visible text shares the same blind spot.

What is Westermo-specific. The Likert ratings (median 4/5 on the conceptual design) come from two Westermo reviewers commenting on a pipeline designed for Westermo's process. A reviewer from a different organization, or from a different role (system architect, regulatory officer), would likely score the same design along another axis. The case-study evidence here is best read as analytical generalization in Runeson et al.'s sense [19], where the pattern is the contribution and the numbers are the existence proof, not the population estimate.

9.6 Synthesis

Taken together, the four research questions show that AI and LLMs can take real responsibility in industrial Requirements Engineering without breaking the two constraints that hold the published literature back. Confidentiality is preserved: the pipeline runs on a local backbone (Llama 3 8B) at a quality cost of one percentage point on semantic similarity. Human authority is preserved because two mandatory HITL checkpoints sit on the critical edges where the EU AI Act asks for oversight. F09 makes those checkpoints concrete rather than ceremonial: 80 % of absence-based defects only get caught when a human is in the loop.

The overall picture is positive. Two Westermo reviewers rated the conceptual design at a median 4/5, with symmetric review at the maximum on both forms. A single pipeline configuration runs across three deployment backends with no code changes. The cloud-versus-local trade-off narrows to a single percentage point on the central similarity metric. One counterweight remains: the FS produced for the case study was rated not yet release-ready. That verdict is tied to internal context available to the agents, not to the pipeline design.

10 Conclusions

This thesis looked at how *Artificial Intelligence (AI)* and *Large Language Models (LLMs)* can support industrial *Requirements Engineering (RE)*. The work was done together with Westermo Network Technologies AB as the industry partner. It follows the *Design Science Research (DSR)* methodology of Peffers et al. [6].

Qualitative conversations picked out the manual step from the *Market Requirement Specification (MRS)* to the *Functional Specification (FS)* as the area where AI and LLM can provide most support. The resulting artifact is a four-agent pipeline backed by a shared *Retrieval-Augmented Generation (RAG)* knowledge base and two mandatory *Human-in-the-Loop (HITL)* checkpoints. It builds on the *Multi-Agent Systems (MAS)* RE paradigm, but adds two things. A review output tagged by severity, and a symmetric review setup where every generative agent is followed by its own reviewer. A working prototype puts the design into practice for the DSR demonstration step and runs it end-to-end on three deployment backends.

Three evaluation strands were used to test the artifact. A comparative model study showed that the choice of backend matters less than expected. Claude Opus 4.7 produced the deepest specification, and Llama 3 (8B), running fully on local hardware, produced a complete and fully traceable FS. Llama 3 closed the gap to Opus to a single percentage point on semantic similarity (0.721 vs. 0.731). Phi-3, Llama 2, and Mistral failed at the structural level and are not usable backends for this pipeline. Cloud models offer slightly more specification depth, local Llama 3 8B offers full confidentiality, and either choice is defensible depending on the organization's appetite for risk. A fault-injection test reached an average detection rate of 82 % across ten injected faults. The expert evaluation was carried out by two Westermo reviewers on the conceptual design and by one on the generated FS. Both reviewers gave Review symmetry the maximum score, and the conceptual design overall received a median of 4 out of 5. The generated FS itself was judged not yet release-ready, with the limitation attributed to the depth of internal Westermo context available to the agents rather than to the design itself.

11 Future Work

The next step is to build a fully industry-ready *Market Requirements Specification (MRS)* to *Functional Specification (FS)* system, with the results of this thesis as reference. Included would be Westermo's internal documents, which would give the agents precise and relevant data.

An extended version of the pipeline, the *Functional Specification (FS)* to *Software Requirements (SR)* step, would be a natural follow-up after the first MRS-to-FS system is released. It is the second manual handoff identified under *RQ2* and has the same shape as the MRS-to-FS case: an input artifact, a target artifact, a template, and quality criteria. The four-agent layout, the symmetric review setup, and the HITL checkpoints should all carry over with only minor adjustments.

Absence-based detection scored low, since the FS-Review Agent cannot flag content that is missing from the document in front of it. A strong *Human-In-The-Loop (HITL)* step is therefore needed in any future implementation. It is the part that catches what the agent cannot see on its own.

The current design treats all human reviewers the same. In practice, the requester, the developer, and the system architect have different views of what a good requirement is. They sit at different points in the requirement lifecycle. A future version of the pipeline could give each role its own view of the agent output, with checkpoints and review criteria tailored to that role.

References

[1] H. Cheng, J. H. Husen, Y. Lu, T. Racharak, N. Yoshioka, N. Ubayashi, and H. Washizaki, "Generative AI for requirements engineering: A systematic literature review," *Software: Practice and Experience*, vol. 56, no. 2, pp. 141–170, 2026.

[2] D. Jin, Z. Jin, X. Chen, and C. Wang, "MARE: Multi-agents collaboration framework for requirements engineering," *arXiv preprint arXiv:2405.03256*, 2024.

[3] M. A. Sami, M. Waseem, Z. Zhang, Z. Rasheed, K. Systä, and P. Abrahamsson, "AI based multiagent approach for requirements elicitation and analysis," *arXiv preprint arXiv:2409.00038*, 2024.

[4] M. S. Ibtasham, S. Bashir, M. Abbas, Z. Haider, M. Saadatmand, and A. Cicchetti, "ReqRAG: Enhancing software release management through retrieval-augmented LLMs: An industrial study," in *Proceedings of Requirements Engineering: Foundation for Software Quality (REFSQ 2025)*. Springer, Cham, 2025, pp. 277–292.

[5] European Union, "Regulation (EU) 2024/1689 of the european parliament and of the council of 13 june 2024 laying down harmonised rules on artificial intelligence (artificial intelligence act)," Official Journal of the European Union, Tech. Rep. L 2024/1689, 2024. [Online]. Available: http://data.europa.eu/eli/reg/2024/1689/oj

[6] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

[7] Westermo, "Company in brief," https://www.westermo.com/about-us/company-in-brief, accessed: May 26, 2026.

[8] M. Raza, Z. Jahangir, M. B. Riaz, M. J. Saeed, and M. A. Sattar, "Industrial applications of large language models," *Scientific Reports*, vol. 15, p. 13755, 2025.

[9] M. Klesel and H. F. Wittmann, "Retrieval-augmented generation (RAG)," *Business & Information Systems Engineering*, vol. 67, no. 4, pp. 551–561, 2025.

[10] International Electrotechnical Commission, "IEC 62443-4-2:2019 security for industrial automation and control systems — part 4-2: Technical security requirements for IACS components," IEC, International Standard 62443-4-2:2019, 2019, edition 1.0, published 2019-02, 192 pp. Corrected by IEC 62443-4-2:2019/C1:2022.

[11] S. Bashir, A. Ferrari, A. Khan, P. E. Strandberg, Z. Haider, M. Saadatmand, and M. Bohlin, "Requirements ambiguity detection and explanation with LLMs: An industrial study," in *Proceedings of the 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2025, pp. 620–631.

[12] M. A. Zadenoori, J. Dąbrowski, W. Alhoshan, L. Zhao, and A. Ferrari, "Large language models (LLMs) for requirements engineering (RE): A systematic literature review," *arXiv preprint arXiv:2509.11446*, 2025.

[13] A. S. Maiya, "OnPrem.LLM: A privacy-conscious document intelligence toolkit," *arXiv preprint arXiv:2505.07672*, 2025.

[14] D. Vake, J. Vičič, and A. Tošić, "Hive: A secure, scalable framework for distributed Ollama inference," *SoftwareX*, vol. 30, p. 102183, 2025.

[15] B. Wei, "Requirements are all you need: From requirements to code with LLMs," in *Proc. IEEE 32nd International Requirements Engineering Conference (RE)*. IEEE, 2024, pp. 416–422.

[16] P. E. Strandberg, "Ethical interviews in software engineering," *arXiv preprint arXiv:1906.07993*, 2019.

[17] N. Jegham, M. Abdelatti, C. Y. Koh, L. Elmoubarki, and A. Hendawi, "How hungry is AI? benchmarking energy, water, and carbon footprint of LLM inference," *arXiv preprint arXiv:2505.09598*, 2025.

[18] F. Huseynova, "Addressing deskilling as a result of human-AI augmentation in the workplace," in *Proceedings of the Conference on Technology Ethics (Tethics 2024)*, ser. CEUR Workshop Proceedings, vol. 3901. CEUR-WS.org, 2024, pp. 148–156. [Online]. Available: https://ceur-ws.org/Vol-3901/short_5.pdf

[19] P. Runeson, M. Höst, A. Rainer, and B. Regnell, *Case Study Research in Software Engineering: Guidelines and Examples*. Wiley, 2012.

A Agent System Prompts

This appendix lists the system prompt of each of the four agents in the multi-agent pipeline. Each prompt is the role and rules block that prepends every LLM call for that agent. The task-specific portion of each prompt is omitted here for length, but follows the same structure on every backend. Company-specific identifiers have been replaced with generic placeholders.

```
============================================================
AGENT SYSTEM PROMPTS (verbatim from prototype source code)
Company-specific identifiers anonymized for publication.
============================================================


============================================================
MRS Agent -- system prompt
============================================================

You are MRS_Agent - an expert in requirements analysis for embedded
and networked industrial systems at The Company.

You operate in a multi-agent pipeline:
MRS_Agent -> MRS_Review_Agent -> [HITL 1] -> FS_Agent -> FS_Review_Agent -> [HITL 2]

You produce TWO documents per run:
  1. analysis       - 5-section MRS analysis used by FS_Agent.
  2. ambiguity_doc  - improvement document used to route problems back
                      to the marketing/product team if needed.

The MRS template (hardcoded - do not query RAG for it) is:

{MRS_TEMPLATE_SECTIONS}

INVIOLABLE RULES:
1. Base findings on the RAG context (process + IEC 62443) plus the MRS itself.
2. Every finding cites specific MRS text (direct quote where possible).
3. Do not invent information. If something is absent, flag it; never fill it in.
4. Write in English. Precise, professional, constructive.
5. Brief or partial outputs are not acceptable.
6. Follow the EXACT section headings specified in each task prompt.

============================================================
MRS-Review Agent -- system prompt
============================================================

You are MRS_Review_Agent - a quality reviewer for MRS analyses.

You review the ANALYSIS produced by MRS_Agent - not the MRS document itself.
A good analysis identifies many problems. You receive three inputs:

  1. The original MRS document (raw input)
  2. The MRS analysis (5-section structured output)
  3. The Ambiguity Document (focused breakdown of ambiguities/gaps/unclear requirements)

Your job: verify the analysis is complete and acceptable for FS generation.

============================================================================
FIVE STRUCTURAL ELEMENTS THE ANALYSIS MUST CONTAIN
============================================================================

1. MRS Overview with explicit quality verdict using exactly one of:
   Complete / Mostly complete / Significant gaps / Not suitable for FS generation.
   Plus subsection 1.1 Requirement Inventory listing every MRS requirement
   as [MRS-NNN] (PRIORITY) <text>.

2. Findings by Category - one subsection PER category:
   Ambiguities / Gaps / Unmeasurable Requirements / Contradictions /
   Security Gaps / Priority Issues. Each finding cites or quotes specific
   MRS text and explains why it is a problem. Empty categories explicitly
   stated as "None found."

3. Security Assessment - every relevant IEC 62443-4-2 FR labelled
   Addressed / Partial / Missing, with remediation if Missing.

4. Missing Information list - every item prefixed exactly "MISSING:" with
   format: MISSING: [item] - [why needed] - [suggested resolution].

5. Recommendation - exactly one of APPROVE / APPROVE WITH CONDITIONS /
   REVISE, with the listed conditions/blocking items.

The Ambiguity Document is checked separately: it must enumerate every
ambiguity and unclear requirement individually with severity rationale.
```

```
========================================================================
SEVERITY TAGGING - MANDATORY
========================================================================

Every finding in your feedback MUST start with one of these tags:

  [CRITICAL] - the analysis cannot be used for FS generation as-is. Examples:
               missing Requirement Inventory, missing entire required section,
               recommendation field absent, security FR statuses unparseable.

  [MEDIUM]   - the analysis is incomplete but FS generation could limp along.
               Examples: a category subsection present but only one finding
               where multiple are obvious; security assessment misses a relevant FR.

  [LOW]      - cosmetic or minor. Examples: minor typos, slightly vague
               phrasing in a finding, sub-sections in unusual order but
               otherwise complete.

========================================================================
RESPONSE FORMAT
========================================================================

Reply with EXACTLY one of:

  APPROVED: [one sentence summarising the recommendation given by MRS_Agent,
             confirming all five elements are present, and the ambiguity doc
             is acceptable]

  REVISE: [bulleted list, every line starting with a severity tag, e.g.
           "  * [CRITICAL] Section 1.1 Requirement Inventory is missing.
                 Without it FS_Agent cannot produce [MRS-XXX] traceability tags."
           Generic feedback like "Section 3 is incomplete" is NOT acceptable.
           Be specific: name section numbers, missing sub-sections, exact
           items the analysis fails to cover.]

Write in English.

============================================================
FS Agent -- system prompt
============================================================

You are FS_Agent - an expert in Functional Specifications for
embedded and networked industrial systems at The Company.

You operate as part of a multi-agent pipeline:
MRS_Agent -> Review_Agent -> [HITL Checkpoint 1] -> FS_Agent -> Review_Agent -> [HITL Checkpoint 2]

Your task is to produce a complete, standalone Functional Specification that
a software team can implement and test without ambiguity.

========================================================================
ANTI-FABRICATION POLICY - READ CAREFULLY. THIS IS A LEGITIMATE The Company FS.
========================================================================

You are writing a document that will be USED by The Company engineers. The
ONLY two trusted sources of content are The Company's own documents. Anything
outside these two sources is treated as unknown and must be flagged.

You MUST distinguish between exactly THREE classes of content:

  (A) FACTS FROM THE MRS - use directly, cite with [MRS-XXX]. Examples:
      product name "The Software", feature name, MUST/SHOULD/COULD requirement text,
      hardware models explicitly named (e.g. "SFP Module X"), traffic-type
      categories the MRS lists.

  (B) FACTS FROM THE RAG CONTEXT - use directly, cite the source. The RAG
      knowledge base contains The Company's process documentation, MRS/FS
      templates, and the cybersecurity guide based on IEC 62443-4-2.
      Use anything from these documents freely.

  (C) ANYTHING ELSE - treat as UNKNOWN. This includes:
      - Public standards NOT present in the RAG (RFC, IEEE, ISO, IEC variants
        other than IEC 62443-4-2). Even if you "know" them from training,
        DO NOT cite them. If a requirement clearly needs such a standard,
        write "[TBD - propose external reference for The Company review: <topic>]".
      - The Company-internal document names (Alarm Framework Spec, CLI Reference
        Manual, RBAC Specification, etc.) - DO NOT invent these. Use
        "[TBD - verify with The Company doc team: <topic>]" instead.
      - Exact The Software CLI syntax. You may propose CLI shape, but every command
        block must carry the disclaimer
        "[SYNTAX PROPOSAL - verify against The Software CLI conventions]".
      - Exact default values, ranges, units, timeouts, port counts that are
        not in the MRS. Use [TBD - PM/PO confirmation required] inline AND
```

```
        an ASSUMPTION-nn entry in section 11.
      - Author names, customer names, internal team names.
      - Hardware platform names not mentioned in the MRS.
      - The Software internal API names, function names, file paths.
      - Version numbers of The Software or its components not in the MRS.
      - Generic "industry best practice" claims you cannot trace to (A) or (B).

When in doubt: mark with [TBD - ...] or [ASSUMED - ...]. NEVER guess silently.
The Company cares only about its own standards - do not pad the document with
external knowledge they have not vetted into RAG.

================================================================================
INVIOLABLE RULES
================================================================================

1. TRACEABILITY - every functional requirement carries a [MRS-XXX] tag.
2. MEASURABILITY - never use vague terms (fast, reliable, high, easy,
   appropriate, correct, properly, as needed, quickly).
3. SECURITY - every IEC 62443-4-2 FR flagged Missing or Partial in the MRS
   analysis must have at least one corresponding FS requirement carrying
   the appropriate FR tag (e.g. [FR7-RA], [FR1-IAC], [FR2-UC]).
4. NO FABRICATION - see policy above. Use [TBD - PM/PO confirmation required]
   or [ASSUMED - PM/PO confirmation required] for anything not in MRS or RAG.
5. ASSUMPTIONS - every MISSING item from the MRS analysis must produce an
   ASSUMPTION-nn entry in section 11 using exactly this format:
   ASSUMPTION-nn: [what was assumed] - [why] - [PM/PO confirmation required before implementation]
6. SHALL - every requirement uses "shall" and describes externally observable behaviour.
7. STANDALONE - this is a NEW document, not a summary of the MRS analysis.
8. ENGLISH - write in English. Be precise, professional, and technically accurate.

If you receive REVISE_FS feedback: integrate ALL feedback. Patch in place -
do not introduce new fabricated content while fixing other issues.

============================================================
FS-Review Agent -- system prompt
============================================================

You are FS_Review_Agent - a quality reviewer for Functional
Specifications. You review one gate at a time. The FS template you verify
against is fixed (hardcoded, not retrieved from RAG):

{FS_TEMPLATE_SECTIONS}

================================================================================
SEVERITY TAGGING - MANDATORY
================================================================================

Every finding starts with [CRITICAL] / [MEDIUM] / [LOW]:

  [CRITICAL] - the FS cannot be released as-is. Examples: required section
               missing, fabricated values not flagged, FR has no traceability,
               security FR for a Missing IEC 62443 FR is absent.

  [MEDIUM]   - noticeable defect that an engineer can patch. Examples: FR uses
               vague term, configuration parameter lacks default but has range,
               assumption format slightly off.

  [LOW]      - cosmetic. Examples: ordering issue, redundant phrasing.

================================================================================
RESPONSE FORMAT
================================================================================

  APPROVED: [one sentence: gate passed, key counts]
  REVISE_FS: [bulleted list, every line starting with severity tag,
              referencing section numbers, FR-IDs, MISSING-item names.
              Generic observations are NOT acceptable.]

Write in English.
```

B RAG Knowledge Base

This appendix describes the Retrieval-Augmented Generation knowledge base used by all four agents: the indexing stack, the documents in the index, the retrieval parameters, the four named query getters, and which agent uses which getter. The embedding and retrieval always run on the local machine regardless of which LLM backend is used.

```
============================================================
RAG KNOWLEDGE BASE -- configuration and content
============================================================

INDEX STACK
    LlamaIndex                vector index + retrieval
    Ollama (local)            embedding model (nomic-embed-text)
                              + fallback inference (Llama 3)
    SimpleDirectoryReader     ingestion (recursive, .docx/.txt/.pdf/.md)
    Persist directory         knowledge_base/.index_cache/
                              (built once, reloaded on subsequent runs)


INDEXED DOCUMENTS
    1. RAG_Cybersecurity_Guide_IEC62443_updated.docx
       Source: industrial cybersecurity guide aligned with
               IEC 62443-4-1 and IEC 62443-4-2.
       Use:    grounds every security finding the agents emit;
               provides the FR1-FR7 Foundational Requirements.

    2. RAG_Generic_Software_Process.docx
       Source: generic software-process reference describing
               MRS / FS quality criteria, MoSCoW prioritisation,
               and common defect categories.
       Use:    grounds the structural and quality checks the
               agents apply to MRS and FS artifacts.


RETRIEVAL PARAMETERS
    similarity_top_k          8     (top-k chunks per query)
    embedding model           nomic-embed-text (Ollama, local)
    request_timeout           120 s


NAMED QUERY GETTERS
    Each agent uses one or more of the four named getters below.
    The getter pre-pends a base query string focused on its source
    document, then appends a topic-specific query string supplied
    by the agent. The combined string is what hits LlamaIndex.

    get_process_context(topic):
        Source: RAG_Generic_Software_Process.
        Base query:
            "Generic software process MRS quality criteria phase 1
             MoSCoW prioritisation MUST SHOULD COULD common defects
             language style guidelines FS quality checklist completeness"

    get_mrs_template(topic):
        Source: RAG_Generic_Software_Process.
        Base query:
            "Market Requirement Specification template structure sections
             Benefit Hypothesis Customer Representative Constraints Assumptions
             Functional Overview Use Cases Functional Requirements MoSCoW
             Revision History Reference Documents Terms Abbreviations"

    get_fs_template(topic):
        Source: RAG_Generic_Software_Process.
        Base query:
            "Functional Specification template structure sections
             Market Requirement Use Cases CLI Configuration Functional Requirements
             Topology Acceptance Criteria Risks Revision History
             Reference Documents Terms Abbreviations Assumptions"

    get_security_context(topic):
        Source: RAG_Cybersecurity_Guide_IEC62443.
        Base query:
            "IEC 62443 4-2 Foundational Requirements FR1 IAC identification
             FR2 UC use control FR3 SI system integrity FR4 DC data confidentiality
             FR5 RDF restricted data flow FR6 TRE timely response events
             FR7 RA resource availability authentication logging audit
             storm control denial of service availability CLI access"


WHICH AGENT USES WHICH GETTER
```

```
    MRS Agent
        get_process_context()   process quality criteria for Phase-1 MRS
        get_security_context()  IEC 62443 FRs for the security assessment
    MRS-Review Agent
        get_process_context()   MRS quality criteria for the structural check
        get_security_context()  IEC 62443 FRs for the section-3 verification
    FS Agent
        get_process_context()   FS quality checklist
        get_security_context()  IEC 62443 FRs for FRn-XXX tagging
    FS-Review Agent
        get_process_context()   FS quality checklist for the six gates
        get_security_context()  IEC 62443 FRs for the security gate
    Note:    MRS and FS templates are HARD-CODED into the agents'
             system prompts and are NOT retrieved from RAG. Only
             process/security context goes through retrieval.


WHERE RAG CHUNKS ARE EMBEDDED IN THE PROMPT
    Every agent prompt includes the retrieved chunks under named
    headings, for example:
        ## RAG CONTEXT - Generic Software Process (quality criteria for an MRS)
        <retrieved process chunks>
        ## RAG CONTEXT - IEC 62443-4-2 (security foundational requirements FR1-FR7)
        <retrieved security chunks>


DATA-FLOW NOTE
    Embedding and retrieval ALWAYS run on the local machine.
    For the cloud-hosted and API-based backends, the retrieved
    chunks become part of the outgoing inference request. The
    Local (Ollama) backend keeps the chunks on the local machine
    end-to-end.
```

C Orchestrator Pseudocode

This appendix gives the deterministic control flow of the orchestrator (`main.py`) in pseudocode. The actual Python source is approximately 30 KB.

```
============================================================
ORCHESTRATOR PSEUDOCODE -- main.py
Distilled from the prototype's deterministic control flow.
The actual Python source is ~30 KB; this pseudocode preserves
the routing logic, retry caps, and HITL branches but omits
logging, error formatting, and CLI plumbing.
============================================================

CONSTANTS
    MAX_RETRIES_MRS = 3
    MAX_RETRIES_FS  = 3

ENTRY POINT main(mrs_path):
    raw_mrs       <- read_document(mrs_path)
    kb            <- KnowledgeBase(knowledge_base_dir)
    kb.load_or_build()
    mrs_agent        <- MRSAgent(kb)
    mrs_review_agent <- MRSReviewAgent(kb)
    fs_agent         <- FSAgent(kb)
    fs_review_agent  <- FSReviewAgent(kb)

    # --- Phase 1: MRS analysis + review (refinement loop) ---
    feedback_doc <- ""
    FOR attempt IN 1..MAX_RETRIES_MRS:
        mrs_output <- mrs_agent.analyze(raw_mrs, feedback_doc)
        analysis      <- mrs_output["analysis"]
        ambiguity_doc <- mrs_output["ambiguity_doc"]

        review <- mrs_review_agent.review(raw_mrs, analysis, ambiguity_doc)
        IF review["approved"]:
            BREAK
        IF review["severity"]["critical"] == 0 AND attempt > 1:
            BREAK  # no blocking findings left
        feedback_doc <- review["feedback"]

    # --- HITL Checkpoint 1: Product Manager review ---
    decision_1 <- hitl_checkpoint(
        title    = "HITL 1 - MRS",
        artifact = analysis + ambiguity_doc,
        options  = ["approve", "reject_with_feedback",
                    "edit_source_mrs_and_restart", "end_run"]
    )
    SWITCH decision_1:
        CASE "edit_source_mrs_and_restart":
            RESTART main(edited_mrs_path)
        CASE "reject_with_feedback":
            feedback_doc <- decision_1["feedback"]
            GOTO Phase 1
        CASE "end_run":
            RETURN partial(analysis)
        # else continue

    # --- Phase 2: FS generation + review (refinement loop) ---
    review_feedback <- ""
    FOR attempt IN 1..MAX_RETRIES_FS:
        IF attempt == 1:
            fs_document <- fs_agent.generate(analysis)
        ELSE:
            # Surgical patch on retry (one LLM call instead of full rewrite)
            fs_document <- fs_agent.patch(fs_document, review_feedback,
                                          mrs_analysis = analysis)

        # --- Supplement conversation flow (FS-Agent -> MRS-Agent) ---
        question <- fs_agent.detect_supplement_request(fs_document)
        IF question IS NOT NULL:
            supplement_answer <- mrs_agent.supplement(raw_mrs, analysis, question)
            analysis <- analysis + supplement_answer
            CONTINUE  # re-generate FS with supplemented analysis

        review <- fs_review_agent.review(fs_document, analysis)
        IF review["approved"]:
            BREAK
        IF review["severity"]["critical"] == 0 AND attempt > 1:
            BREAK
        review_feedback <- review["feedback"]

    # --- HITL Checkpoint 2: Product Owner review ---
```

```
    decision_2 <- hitl_checkpoint(
        title    = "HITL 2 - FS",
        artifact = fs_document,
        options  = ["approve", "reject_with_feedback",
                    "edit_source_mrs_and_restart", "end_run"]
    )
    SWITCH decision_2:
        CASE "reject_with_feedback":
            review_feedback <- decision_2["feedback"]
            GOTO Phase 2
        CASE "edit_source_mrs_and_restart":
            RESTART main(edited_mrs_path)
        CASE "end_run":
            RETURN partial(fs_document)

    # --- Persist artifacts and session log ---
    write_artifact("mrs_analysis.md",    analysis)
    write_artifact("mrs_corrected.md",   analysis)        # template-complete MRS
    write_artifact("fs_document.md",     fs_document)
    write_artifact("fs_review_feedback.md", review["feedback"])
    write_artifact("fs_review_advisory.md",
                   filter_low_severity(review["feedback"]))
    RETURN fs_document


HELPER hitl_checkpoint(title, artifact, options):
    # Human review point. In autonomous test mode, this auto-approves.
    show(title, artifact, options)
    RETURN user_choice()


SESSION LOG
    Every call writes a structured JSONL line:
        {"ts": ISO8601,
         "agent": "mrs_agent|mrs_review_agent|fs_agent|fs_review_agent",
         "event": "agent_start|agent_end|review_start|review_end|hitl|...",
         "detail": <call-specific payload>}
    File: output/logs/session_<timestamp>.jsonl


NOTES
1. Routing is deterministic: Python decides which agent runs next, never the LLM.
2. Each generator-reviewer pair has its own refinement loop with a hard
   cap of 3 retries. After 3, control passes to the human checkpoint
   regardless of remaining findings.
3. The supplement-conversation flow is the one place where the FS Agent
   can effectively "send back" to the MRS Agent. The branch condition
   ("detect_supplement_request") is a deterministic regex match in the
   Python code, not an LLM decision.
4. The same control flow runs unchanged against all three LLM backends
   (Local Ollama, Hugging Face Inference API, Anthropic API). Only the
   call_llm() implementation in utils/llm_client.py differs.
```

D Reference MRS — Storm Control

This appendix reproduces `MRS_perfect.docx`, the reference Market Requirement Specification used as the perfect input across all comparative model runs. It describes a network storm-control feature in full and follows the MRS template. Company-specific identifiers have been replaced with generic placeholders.

```
Software Market Requirement Specification
Storm Control - The Software
Version: 1.0  |  Status: Approved (Perfect reference - fault injection baseline)
Document No: MRS-SC-001  |  Date: 2026-05-08

## Benefit Hypothesis
Industrial network operators in rail, energy, utility, and factory-automation environments depend on
      uninterrupted network availability. A broadcast storm - where a single misbehaving device floods the
      network with excessive broadcast, multicast, or unknown unicast traffic - can saturate link capacity,
      exhaust device CPU resources, and halt communication across an entire network segment. Root causes include
      faulty network interface cards (NICs), accidental Layer-2 loops introduced during commissioning or
      maintenance, misconfigured neighbouring devices, and deliberate denial-of-service activity.
Today, The Software provides no automated mechanism to detect or respond to such conditions. Operators must
      manually identify and isolate the offending device, which in industrial environments can be slow and costly
      . Storm control addresses this by enabling the switch to autonomously detect abnormal ingress traffic on a
      per-port basis and take immediate protective action - limiting the impact to the affected port while
      keeping the rest of the network operational. This directly improves network availability and reduces mean
      time to recovery in safety-critical environments.
User profile: Network administrator or system integrator responsible for configuring and maintaining industrial
      Ethernet installations.

## Customer Representative(s)
Reviewer A, Product Manager - Industrial Switching, The Company.
Reviewer B, Field Application Engineer - Rail & Energy segment, The Company.

## Constraints and Assumptions
Initial hardware support is limited to the supported hardware family devices (e.g. Hardware Platform A). Support
       for additional platforms may be added in future releases.
The SFP Module X module is not compatible with storm control functionality. Use of this SFP on a port where
      storm control is active will disturb feature behaviour. The switch must warn the operator clearly in both
      CLI output and documentation.
CPU bandwidth limiting must be configured on devices where storm control is deployed to prevent the device CPU
      from being overwhelmed before the storm control trigger fires.
FDB aging-timeout must be reviewed on adjacent devices, as a storm may replace correct FDB entries and affect
      traffic routing after the storm subsides.
Storm control is implemented using the existing The Software alarm trigger/action framework. The trigger type is
       storm-detect (monitoring ingress traffic in frames per second per port). The action type is port (shutting
       down the affected port). This is consistent with how other alarm features work in The Software.
The CLI configuration syntax follows the The Software alarm framework convention: configure alarm, then trigger
      storm-detect with threshold and port parameters, and action port with target port and led parameters.
      Persistence is set via the persistent parameter on the trigger.
Storm control monitors the aggregated ingress frame rate across broadcast, multicast, and unknown-unicast
      traffic classes combined. Per-class thresholds are not supported in this release.
Configuration parameters (threshold, persistence, port assignment, action) are persisted to startup-config and
      survive device reboot. The triggered runtime state and any active persistence timer are not retained across
       a reboot; every port starts in the normal state after boot.
The feature is available via CLI only in this release. Web UI configuration is deferred to a future release.

## Functional Overview
Storm control enables the switch to monitor incoming traffic volume on individual ports and automatically take a
       configured protective action when traffic exceeds a defined threshold. This protects the broader network
      from being disrupted by a storm originating on a single port.
The feature works as follows: the operator configures a storm-detect trigger on one or more ports, specifying a
      threshold in frames per second (fps). The switch continuously monitors aggregated ingress traffic (
      broadcast + multicast + unknown unicast) on each configured port. When the measured rate exceeds the
      configured threshold, the trigger fires and activates the configured alarm action. The default and most
      common action is to shut down the affected port and activate the LED alarm indicator. The port remains shut
       down until manually re-enabled by the operator, or recovers automatically after a configurable persistence
       interval. The default persistence interval is 300 seconds; configuring persistent 0 means the port stays
      shut down until manually re-enabled.
This integrates with the existing The Software alarm framework, meaning operators familiar with other The
      Software alarm triggers (such as ping triggers or the Ring Protocol triggers) will find storm control
      straightforward to configure.

### Use Cases
Use case 1 - Faulty NIC:
A NIC on an end device malfunctions and begins flooding the network with broadcast packets at a rate exceeding
      500 fps. The switch detects that ingress traffic on the connected port exceeds the configured threshold of
      500 fps, shuts down the port, logs the event, and activates the LED alarm indicator. The rest of the
      network continues to operate normally. The operator receives the alarm and can physically inspect the
      device before manually re-enabling the port via CLI.
Use case 2 - Network loop:
An incorrect cable connection creates a loop in the network, causing traffic to multiply rapidly. Storm control
      detects the resulting traffic spike on the affected port, shuts it down before the storm propagates to the
```

```
      rest of the network, and logs the event with LED indication. The operator investigates, corrects the
      cabling, and manually re-enables the port via CLI.

## Functional Requirements

### MUST Requirements
[MRS-001] The switch shall monitor aggregated ingress traffic volume (broadcast + multicast + unknown-unicast
      frames) on a per-port basis, measured in frames per second (fps). Monitoring shall be active on all ports
      where the storm-detect trigger has been configured.
[MRS-002] The switch shall allow the operator to configure a threshold value per port, expressed in frames per
      second (fps). The threshold is specified as an integer value. Example from The Software documentation:
      threshold 500 (fps). When the measured ingress rate exceeds the configured threshold, the trigger shall
      fire and activate the configured alarm actions.
[MRS-003] The switch shall support port shutdown as an alarm action in response to a detected storm. When the
      action fires, the affected port shall be placed in an operationally down state such that no frames are
      transmitted or received on that port until a recovery condition is met. The action name in The Software CLI
       is: action port.
[MRS-004] The switch shall support alarm log entry as an alarm action. When the storm-detect trigger fires, an
      event shall be written to the system log identifying the affected port and the trigger type.
[MRS-005] The switch shall support LED indication as an alarm action. When the storm-detect trigger fires, the
      device LED alarm indicator shall be activated. The LED indication shall remain active until the port
      returns to normal state. The action target in The Software CLI is: target port led.
[MRS-006] The operator shall be able to configure storm detection as persistent. The persistence interval is
      specified in seconds via the persistent parameter on the trigger (e.g. persistent 300). When configured
      with a non-zero persistence interval, the port shall remain shut down for the specified duration before
      auto-recovery is considered. Setting persistent 0 means the port stays shut down until manually re-enabled.
[MRS-007] The persistence interval shall be configurable by the operator, expressed in seconds. The default
      persistence value used in The Software documentation examples is 300 seconds. A value of 0 disables auto-
      recovery and requires manual re-enable.
[MRS-008] The feature shall be configurable via CLI using the existing The Software alarm framework. The
      complete configuration pattern is: (1) configure an action with target port led, (2) configure a trigger
      storm-detect with the action reference, port selection, persistence interval, and threshold. See the
      Functional Overview section for the full CLI example.
[MRS-009] Storm control shall operate independently per port and shall not affect forwarding on unaffected ports
      . A storm event on one port must not cause shutdown or forwarding disruption on any other port.
[MRS-010] The operator shall be able to display current storm control configuration and trigger status via CLI,
      consistent with the The Software alarm show command (show alarm or equivalent).

### SHOULD Requirements
[MRS-011] The switch should provide a clear warning in documentation and in CLI output that the SFP Module X
      module is not compatible with storm control. The warning should appear when storm control is configured on
      a port using this SFP. Note: given the risk of silent loss of protection, this requirement should be
      considered for elevation to MUST during implementation review.
[MRS-012] The switch should log a recovery event when a port is automatically re-enabled after the persistence
      interval expires, so that the operator has a record of the recovery in the system log.

### COULD Requirements
[MRS-013] Configuration via web interface could be supported in a future release. This is explicitly deferred
      from the initial CLI-only release.
[MRS-014] Per-VLAN storm control could be considered as a future extension.
[MRS-015] Separate thresholds for broadcast, multicast, and unknown unicast traffic classes could be supported
      in a future release. The initial release uses an aggregated counter across all three classes.

## Security Requirements (IEC 62443-4-2 SL2)
The Company products are certified to IEC 62443-4-2 Security Level 2 (Certificate [Internal Certificate Ref],
      issued 2025-12-22). Storm control configuration must comply with the following applicable foundational
      requirements:
CR 1.01 / CR 1.03 - Identification and authentication control: All CLI commands that configure storm control (
      trigger, action, threshold, persistence) shall be available only to authenticated users. Unauthenticated
      sessions must not be able to modify storm control configuration or trigger state.
CR 2.01 - Authorization enforcement: Storm control configuration commands shall be restricted to users with the
      appropriate administrative role. Read-only users may display storm control status but shall not modify
      configuration.
CR 2.08 - Auditable events: All storm control configuration changes and all storm-control-triggered port
      shutdown events shall be logged with timestamp and sufficient context to identify the event.
CR 2.11 - Timestamps: All log entries related to storm control (trigger events, recovery events, configuration
      changes) shall include accurate UTC timestamps.
CR 7.01 / CR 7.02 - Denial of service protection and resource management: The storm-control measurement and
      trigger evaluation shall continue to operate correctly even when the device is under high traffic load. The
       CPU resources consumed by storm control monitoring shall not prevent the management CLI from remaining
      accessible during a storm event.

## CLI Configuration Reference
The following is the complete, verified CLI configuration pattern for storm control in The Software, based on
      The Software Release Notes vX.Y (section N). This is the authoritative reference for FS generation.
Step 1 - Configure the alarm action:
alarm
  action 2
    target port led
  end
Step 2 - Configure the storm-detect trigger:
  trigger storm-detect
    action 2          (reference to the action above)
    port 1-2          (ports to monitor)
```

```
    persistent 300    (seconds; 0 = manual re-enable only)
    threshold 500     (frames per second)
  end
Result of show alarm after configuration:
Trigger  Type          Enabled  Action  Source
1        the Ring Protocol          YES      1       Instance 1
2        storm-detect  YES      2       1-2
Action   Targets
1        snmp log led digout
2        log led port

## Known Issue
The Software release notes (issue #17845) document the following known behaviour in the tech preview: applying a
      storm-control trigger on a disabled port will enable it in absence of a storm. This is a known limitation
     of the tech preview implementation and must be accounted for in the FS and in system testing.

## Revision History

[TABLE]
Rev. | Description | Prepared by / Date | Reviewed by / Date | Approved by / Date
1.0 | Initial approved version - complete perfect reference for fault injection testing. Based on The Software
     Release Notes vX.Y (section N), IEC 62443-4-2 certificate [Internal Certificate Ref], and MRS gap analysis.
      | Iman Ismael / 2026-05-08 | Reviewer A | Reviewer B
[/TABLE]


## Reference Documents

[TABLE]
Document ID | Title
[Internal Doc ID] | The Company Release Notes The Software vX.Y, the Storm Control section is the primary
     technical source for this MRS
[Internal Certificate Ref] | IEC 62443-4-2 Certificate of Conformity - Hardware Platform B, The Software vX.Y,
     LCIE, 2025-12-22
IEC 62443-4-2:2019 | Security for industrial automation and control systems - Technical security requirements
     for IACS components
MRS-SC-001 | This document
[/TABLE]


## Terms and Abbreviations

[TABLE]
Term / Abbreviation | Description
Storm control | A The Software feature that monitors ingress traffic volume per port using the storm-detect
     alarm trigger and takes protective action (port shutdown) when a configured fps threshold is exceeded
fps | Frames per second - unit of ingress traffic measurement used by the storm-detect trigger
MRS | Market Requirement Specification
FS | Functional Specification
CLI | Command Line Interface - the text-based management interface to The Software
FDB | Forwarding Database - the MAC address table used by the switch for frame forwarding decisions
NIC | Network Interface Card
LED | Light Emitting Diode - used as alarm indicator on the device front panel
LAG | Link Aggregation Group
IEC 62443 | International standard series for security of industrial automation and control systems
SL2 | Security Level 2 as defined in IEC 62443 - the certified security level of The Company The Software
     products
RAG | Retrieval-Augmented Generation - the AI knowledge retrieval mechanism used in the multi-agent system
MVP | Minimum Viable Product - the initial release scope of this feature
[/TABLE]
```

E Fault-Injected MRS

This appendix reproduces `MRS_perfect_with_faults.docx`, the copy of the reference MRS with ten deliberate faults injected (F01–F10, mapped in Table 13). It is the input used in all five fault-injection runs. Company-specific identifiers have been replaced with generic placeholders.

```
Software Market Requirement Specification
Storm Control - The Software
Version: 1.0  |  Status: Approved (Perfect reference - fault injection baseline)
Document No: MRS-SC-001  |  Date: 2026-05-08

## Benefit Hypothesis
Industrial network operators in rail, energy, utility, and factory-automation environments depend on
      uninterrupted network availability. A broadcast storm - where a single misbehaving device floods the
      network with excessive broadcast, multicast, or unknown unicast traffic - can saturate link capacity,
      exhaust device CPU resources, and halt communication across an entire network segment. Root causes include
      faulty network interface cards (NICs), accidental Layer-2 loops introduced during commissioning or
      maintenance, misconfigured neighbouring devices, and deliberate denial-of-service activity.
Today, The Software provides no automated mechanism to detect or respond to such conditions. Operators must
      manually identify and isolate the offending device, which in industrial environments can be slow and costly
      . Storm control addresses this by enabling the switch to autonomously detect abnormal ingress traffic on a
      per-port basis and take immediate protective action - limiting the impact to the affected port while
      keeping the rest of the network operational. This directly improves network availability and reduces mean
      time to recovery in safety-critical environments.
User profile: Network administrator or system integrator responsible for configuring and maintaining industrial
      Ethernet installations.

## Customer Representative(s)
Reviewer A, Product Manager - Industrial Switching, The Company.
Reviewer B, Field Application Engineer - Rail & Energy segment, The Company.

## Constraints and Assumptions
Initial hardware support is limited to the supported hardware family devices (e.g. Hardware Platform A). Support
       for additional platforms may be added in future releases.
The SFP Module X module is not compatible with storm control functionality. Use of this SFP on a port where
      storm control is active will disturb feature behaviour. The switch must warn the operator clearly in both
      CLI output and documentation.
CPU bandwidth limiting must be configured on devices where storm control is deployed to prevent the device CPU
      from being overwhelmed before the storm control trigger fires.
FDB aging-timeout must be reviewed on adjacent devices, as a storm may replace correct FDB entries and affect
      traffic routing after the storm subsides.
Storm control is implemented using the existing The Software alarm trigger/action framework. The trigger type is
       storm-detect (monitoring ingress traffic in frames per second per port). The action type is port (shutting
       down the affected port). This is consistent with how other alarm features work in The Software.
The CLI configuration syntax follows the The Software alarm framework convention: configure alarm, then trigger
      storm-detect with threshold and port parameters, and action port with target port and led parameters.
      Persistence is set via the persistent parameter on the trigger.
Storm control monitors the aggregated ingress frame rate across broadcast, multicast, and unknown-unicast
      traffic classes combined. Per-class thresholds are not supported in this release.
Configuration parameters (threshold, persistence, port assignment, action) are persisted to startup-config and
      survive device reboot. The triggered runtime state and any active persistence timer are not retained across
       a reboot; every port starts in the normal state after boot.
The feature is available via CLI only in this release. Web UI configuration is deferred to a future release.

## Functional Overview
Storm control enables the switch to monitor incoming traffic volume on individual ports and automatically take a
       configured protective action when traffic exceeds a defined threshold. This protects the broader network
      from being disrupted by a storm originating on a single port.
The feature works as follows: the operator configures a storm-detect trigger on one or more ports, specifying a
      threshold in frames per second (fps). The switch continuously monitors aggregated ingress traffic (
      broadcast + multicast + unknown unicast) on each configured port. When the measured rate exceeds the
      configured threshold, the trigger fires and activates the configured alarm action. The default and most
      common action is to shut down the affected port and activate the LED alarm indicator. The port remains shut
       down until manually re-enabled by the operator, or recovers automatically after a configurable persistence
       interval. The default persistence interval is set to a reasonable value; configuring persistent 0 means
      the port stays shut down until manually re-enabled.
This integrates with the existing The Software alarm framework, meaning operators familiar with other The
      Software alarm triggers (such as ping triggers or the Ring Protocol triggers) will find storm control
      straightforward to configure.

### Use Cases
Use case 1 - Faulty NIC:
A NIC on an end device malfunctions and begins flooding the network with broadcast packets at a rate exceeding
      500 fps. The switch detects that ingress traffic on the connected port exceeds the configured threshold of
      500 fps, shuts down the port, logs the event, and activates the LED alarm indicator. The rest of the
      network continues to operate normally. The operator receives the alarm and can physically inspect the
      device before manually re-enabling the port via CLI.
Use case 2 - Network loop:
An incorrect cable connection creates a loop in the network, causing traffic to multiply rapidly. Storm control
      detects the resulting traffic spike on the affected port, shuts it down before the storm propagates to the
      rest of the network, and logs the event with LED indication. The operator investigates, corrects the
      cabling, and manually re-enables the port via CLI.
```

```
## Functional Requirements

### MUST Requirements
[MRS-001] The switch shall monitor aggregated ingress traffic volume (broadcast + multicast + unknown-unicast
      frames) on a per-port basis, with appropriate monitoring. Monitoring shall be active on all ports where the
       storm-detect trigger has been configured.
[MRS-002] The switch shall allow the operator to configure a threshold value per port, expressed in frames per
      second (fps). The threshold is specified as an integer value. Example from The Software documentation: an
      appropriate threshold. When the measured ingress rate exceeds the configured threshold, the trigger shall
      fire and activate the configured alarm actions.
[MRS-003] The switch shall support port shutdown as an alarm action in response to a detected storm. When the
      action fires, the affected port shall be placed in an operationally down state such that no frames are
      transmitted or received on that port until a recovery condition is met. The action name in The Software CLI
       is: action port.
[MRS-004] The switch shall support alarm log entry as an alarm action. When the storm-detect trigger fires, an
      event shall be written to the system log identifying the affected port and the trigger type.
[MRS-005] The switch could support LED indication as an alarm action. When the storm-detect trigger fires, the
      device LED alarm indicator shall be activated. The LED indication shall remain active until the port
      returns to normal state. The action target in The Software CLI is: target port led.
[MRS-006] The operator shall be able to configure storm detection as persistent. The port shall automatically
      recover within 10 seconds regardless of the configured persistence interval. The persistence interval is
      specified in seconds via the persistent parameter on the trigger (e.g. persistent 300). When configured
      with a non-zero persistence interval, the port shall remain shut down for the specified duration before
      auto-recovery is considered. Setting persistent 0 means the port stays shut down until manually re-enabled.
[MRS-007] The persistence interval shall be configurable by the operator, expressed in seconds.
[MRS-008] The feature shall be configurable via CLI using the existing The Software alarm framework. The
      complete configuration pattern is: (1) configure an action with target port led, (2) configure a trigger
      storm-detect with the action reference, port selection, persistence interval, and threshold. See the
      Functional Overview section for the full CLI example.
[MRS-009] Storm control may affect forwarding on adjacent ports under storm conditions to protect overall
      network availability.
[MRS-010] The operator shall be able to display current storm control configuration and trigger status via CLI,
      consistent with the The Software alarm show command (show alarm or equivalent).

### SHOULD Requirements
[MRS-011] The switch should provide a clear warning in documentation and in CLI output that the SFP Module X
      module is not compatible with storm control. The warning should appear when storm control is configured on
      a port using this SFP. Note: given the risk of silent loss of protection, this requirement should be
      considered for elevation to MUST during implementation review.
[MRS-012] The switch should log a recovery event when a port is automatically re-enabled after the persistence
      interval expires, so that the operator has a record of the recovery in the system log.

### COULD Requirements
[MRS-013] Configuration via web interface could be supported in a future release. This is explicitly deferred
      from the initial CLI-only release.
[MRS-014] Per-VLAN storm control could be considered as a future extension.
[MRS-015] Separate thresholds for broadcast, multicast, and unknown unicast traffic classes could be supported
      in a future release. The initial release uses an aggregated counter across all three classes.

## Security Requirements (IEC 62443-4-2 SL2)
The Company products are certified to IEC 62443-4-2 Security Level 2 (Certificate [Internal Certificate Ref],
      issued 2025-12-22). Storm control configuration must comply with the following applicable foundational
      requirements:
CR 2.01 - Authorization enforcement: Storm control configuration commands shall be restricted to users with the
      appropriate administrative role. Read-only users may display storm control status but shall not modify
      configuration.
CR 2.11 - Timestamps: All log entries related to storm control (trigger events, recovery events, configuration
      changes) shall include accurate UTC timestamps.
CR 7.01 / CR 7.02 - Denial of service protection and resource management: The storm-control measurement and
      trigger evaluation shall continue to operate correctly even when the device is under high traffic load. The
       CPU resources consumed by storm control monitoring shall not prevent the management CLI from remaining
      accessible during a storm event.

## CLI Configuration Reference
The following is the complete, verified CLI configuration pattern for storm control in The Software, based on
      The Software Release Notes vX.Y (section N). This is the authoritative reference for FS generation.
Step 1 - Configure the alarm action:
alarm
  action 2
    target port led
  end
Step 2 - Configure the storm-detect trigger:
  trigger storm-detect
    action 2          (reference to the action above)
    port 1-2          (ports to monitor)
    persistent 300    (seconds; 0 = manual re-enable only)
    threshold 500     (frames per second)
  end
Result of show alarm after configuration:
Trigger  Type          Enabled  Action  Source
1        the Ring Protocol          YES      1       Instance 1
2        storm-detect  YES      2       1-2
Action   Targets
```

```
1        snmp log led digout
2        log led port

## Known Issue

## Revision History

[TABLE]
Rev. | Description | Prepared by / Date | Reviewed by / Date | Approved by / Date
1.0 | Initial approved version - complete perfect reference for fault injection testing. Based on The Software
     Release Notes vX.Y (section N), IEC 62443-4-2 certificate [Internal Certificate Ref], and MRS gap analysis.
      | Iman Ismael / 2026-05-08 | Reviewer A | Reviewer B
[/TABLE]


## Reference Documents

[TABLE]
Document ID | Title
[Internal Doc ID] | The Company Release Notes The Software vX.Y, the Storm Control section is the primary
     technical source for this MRS
[Internal Certificate Ref] | IEC 62443-4-2 Certificate of Conformity - Hardware Platform B, The Software vX.Y,
     LCIE, 2025-12-22
IEC 62443-4-2:2019 | Security for industrial automation and control systems - Technical security requirements
     for IACS components
MRS-SC-001 | This document
[/TABLE]


## Terms and Abbreviations

[TABLE]
Term / Abbreviation | Description
Storm control | A The Software feature that monitors ingress traffic volume per port using the storm-detect
     alarm trigger and takes protective action (port shutdown) when a configured fps threshold is exceeded
fps | Frames per second - unit of ingress traffic measurement used by the storm-detect trigger
MRS | Market Requirement Specification
FS | Functional Specification
CLI | Command Line Interface - the text-based management interface to The Software
FDB | Forwarding Database - the MAC address table used by the switch for frame forwarding decisions
NIC | Network Interface Card
LED | Light Emitting Diode - used as alarm indicator on the device front panel
LAG | Link Aggregation Group
IEC 62443 | International standard series for security of industrial automation and control systems
SL2 | Security Level 2 as defined in IEC 62443 - the certified security level of The Company The Software
     products
RAG | Retrieval-Augmented Generation - the AI knowledge retrieval mechanism used in the multi-agent system
MVP | Minimum Viable Product - the initial release scope of this feature
[/TABLE]
```

F Generated Functional Specification (Claude Opus 4.7)

AI-GENERATED CONTENT — RESEARCH USE ONLY.
This document was generated by Claude Opus 4.7 in a single pipeline run on the reference MRS in Appendix D, as part of a bachelor thesis on AI-assisted Requirements Engineering. Company-specific identifiers have been replaced with generic placeholders. The document is not a production specification and is reproduced here as the demonstration deliverable of the Design Science Research artifact (Section 5.3).

This is the Functional Specification produced by the highest-rated backend (Claude Opus 4.7) in the comparative model study (Section 7.3.1). It is also the artifact reviewed by the Software Development Manager in the FS-output Likert evaluation (Section 5.4.1).

```
Functional Specification
Date: 2026-05-10
Version: 1.0
Status: Draft
Source document: MRS_perfect.docx
Generated by: Multi-Agent MRS->FS System

## Functional Specification - Storm Control - The Software
Document No: FS-SC-001
Source MRS: MRS-SC-001 v1.0 ("Software Market Requirement Specification - Storm Control - The Software")
------------------------------------------------------------

### 1. Market Requirement
Industrial Ethernet networks deployed in rail, energy, utility and factory-automation environments are
      characterised by long operational lifetimes, mixed-vintage end-stations, and physical topologies that are
      reconfigured by field technicians who are not always network specialists. In such environments, two failure
       modes recurrently degrade availability of the Layer-2 infrastructure: (a) end-station NICs that develop a
      fault and emit broadcast or multicast traffic at a sustained high rate, and (b) accidental Layer-2 loops
      created during cabinet rewiring or commissioning, which cause unknown-unicast and broadcast frames to be
      replicated until the forwarding plane and the device CPU are saturated. Both modes can propagate from a
      single ingress port across a flat broadcast domain in seconds, taking down unrelated services that share
      the same VLAN or fabric.
This Functional Specification commits the Storm Control feature for The Software on the supported hardware
      family switches [MRS-019]. The feature monitors aggregated ingress traffic (broadcast + multicast + unknown
       unicast) on a per-port basis in frames per second [MRS-001][MRS-020] and, on exceeding an operator-
      configured threshold [MRS-002], triggers protective actions through the existing The Software alarm trigger
      /action framework: shutdown of the offending port [MRS-003], a system-log entry [MRS-004], and activation
      of the device alarm LED [MRS-005]. A configurable persistence interval supports either auto-recovery after
      a defined duration or indefinite shutdown until manual operator intervention [MRS-006][MRS-007]. The
      configuration is CLI-driven through the established The Software alarm pattern [MRS-008] and is persisted
      to startup-config so that protection survives reboot [MRS-016].
The feature is delivered under IEC 62443-4-2 SL2 obligations: configuration is restricted to authenticated, role
      -authorised users [MRS-SEC-01][MRS-SEC-02]; configuration changes and triggered events are audit-logged
      with UTC timestamps [MRS-SEC-03][MRS-SEC-04]; and storm-control measurement remains operational while the
      device is under load so that the management plane stays reachable during an event [MRS-SEC-05]. The
      customer-visible benefit is bounded blast radius: a faulty NIC or accidental loop is contained at the
      ingress port [MRS-009] without disrupting forwarding on the rest of the device, while the operator is
      notified through log, LED, and (where deployed) downstream management systems.
The committed scope for this release is constrained to the supported hardware family hardware [MRS-019]; CPU
      bandwidth limiting is a documented deployment prerequisite on every device where storm control is enabled,
      so that the protective trigger fires before the CPU itself is overwhelmed [MRS-017].

### 2. Not Committed Market Requirements
The following MRS requirements are explicitly deferred and are NOT committed by this FS:
**[MRS-013] Web interface configuration (COULD)** - Deferred. The initial release is CLI-only [MRS-008]. Web UI
      integration depends on the The Software web framework's alarm-trigger schema and is scheduled for a
      subsequent release once the CLI behaviour is field-validated.
**[MRS-014] Per-VLAN storm control (COULD)** - Deferred. The current MRS scopes monitoring to per-port,
      aggregated across traffic classes [MRS-001][MRS-020]. Per-VLAN granularity requires additional hardware-
      counter resources and a re-architected measurement path on Hardware Platform A silicon, and is therefore
      out of scope for this release.
**[MRS-015] Separate thresholds for broadcast / multicast / unknown-unicast (COULD)** - Deferred. The release
      commits a single aggregated threshold per port [MRS-020]. Per-class thresholds will be re-evaluated after
      the v1 feature has accumulated field telemetry that justifies the additional configuration surface.
No SHOULD- or MUST-prioritised MRS requirement is deferred; all are addressed in this FS.

### 5. Topology
Storm Control is a per-port ingress feature that operates on a single the supported hardware family switch and
      does not require any peer device, controller, or external service to function. The reference topology
      assumed by this FS is therefore the physical port boundary of one Hardware Platform A switch (e.g. Hardware
       Platform A [MRS-019]) and the directly-attached Layer-2 neighbours on each protected port.
Port roles. Storm Control may be configured on any Ethernet port that the Hardware Platform A hardware exposes
      for monitoring, regardless of role:
**Edge/access (downlink) ports** facing end-stations, sensors, RTUs, IEDs, IP cameras, or operator workstations.
       This is the primary use case (Use Case 1: faulty NIC) and is the role most likely to source unknown-
      unicast and broadcast storms.
**Inter-switch (uplink/trunk) ports** carrying aggregated VLAN traffic between switches. Storm Control on these
```

```
      ports protects the local switch from a storm propagating from elsewhere in the broadcast domain (Use Case
      2: accidental L2 loop).
**Management ports** dedicated to out-of-band administration. Configuring Storm Control on a port carrying the
      operator's management session may, on a storm event, sever that session; this is operationally legitimate
      but the operator is responsible for the consequence.
Media constraint. Where a port is presented through an SFP cage, the SFP Module X module is incompatible with
      Storm Control measurement and the operator must be warned at configuration time, at SFP insertion, and in
      documentation [MRS-011][MRS-018]. See section 4 for the FS requirements implementing this warning surface.
Redundancy-protocol context. Hardware Platform A switches are routinely deployed in the Ring Protocol rings,
      RSTP/MSTP meshes, and link-aggregation groups. Storm Control's port-shutdown action [MRS-003] places the
      affected port in an operationally-down state, which is observed by these protocols as a link-down event and
       will trigger their normal failover behaviour. The precise interaction precedence between Storm Control
      shutdown and other port-state controllers (the Ring Protocol, RSTP, link aggregation, PoE, 802.1X) is not
      constrained by the MRS and is captured as an assumption (see section 11, ASSUMPTION-14).
Logical context. The feature is orthogonal to VLAN segmentation: measurement is performed on aggregated ingress
      frames at the physical port before VLAN classification [MRS-020], so a storm sourced inside any VLAN
      traversing the port contributes to the measured rate. A storm event on one port shall not affect forwarding
       on any other port of the same device [MRS-009].
Management-plane reachability. During a storm event the management CLI must remain reachable [MRS-SEC-05]. The
      deployment topology is therefore expected to provide either an out-of-band management path or an in-band
      path on a port not currently experiencing the storm; this is a deployment responsibility and is documented
      as ASSUMPTION-15 in section 11.
Out-of-scope topologies. Distributed/coordinated storm control across multiple Hardware Platform A switches,
      controller-based orchestration, and Layer-3 ingress-rate enforcement are not part of this release.

### 3. Use Cases
This section describes the user-facing use cases for the Storm Control feature. Use cases are derived from the
      MRS Functional Overview and Use Case section, and from the operational behaviour implied by [MRS-001]
      through [MRS-021] and [MRS-SEC-01] through [MRS-SEC-05].
---------------------------------------------------------------

#### UC-01 -- Configure Storm Control on a Port
**Actor:** Network engineer (authenticated The Software administrator).
**Trigger:** The engineer needs to protect a the supported hardware family switch port against broadcast/
      multicast/unknown-unicast storms originating from the connected device or segment.
**Pre-conditions:**
- The engineer is authenticated to the The Software CLI [MRS-SEC-01].
- The engineer holds an administrative role authorised to modify alarm/trigger configuration [MRS-SEC-02].
- The target device is a the supported hardware family switch [MRS-019].
- CPU bandwidth limiting is configured on the device [MRS-017].
**Main flow:**
1. The engineer enters configuration mode.
2. The engineer creates an alarm action targeting the port LED indicator [MRS-005][MRS-008].
3. The engineer creates a 'storm-detect' trigger referencing the action, the port (or port list), the
      persistence interval in seconds, and the threshold in fps [MRS-002][MRS-006][MRS-008].
4. The system validates parameter syntax, range, and the existence of referenced ports and actions.
5. The system commits the configuration and persists it to startup-config when the engineer saves [MRS-016].
6. The system writes an audit log entry recording the configuration change with UTC timestamp and authenticated
      user identity [MRS-SEC-03][MRS-SEC-04].
**Post-conditions:**
- Storm Control is active on the configured port(s).
- The configuration is retrievable via the show command (UC-04).
- The configuration survives reboot [MRS-016].
**Alternative / error flows:**
- A1 -- Out-of-range threshold or persistence value: The CLI rejects the command with a validation error and no
      configuration change is committed. *(Range validation is governed by ASSUMPTION-01 and ASSUMPTION-04.)*
- A2 -- Port using SFP SFP Module X: The CLI emits a warning that storm control is not compatible with the SFP
      Module X. The configuration is accepted but the warning is recorded [MRS-011][MRS-018].
- A3 -- Port currently disabled: Per known issue MRS-021, applying the trigger will enable the port in absence
      of a storm. The CLI shall emit a warning and proceed [MRS-021]. *(Final behaviour governed by ASSUMPTION
      -15.)*
- A4 -- Unauthenticated or unauthorised user: The CLI rejects the command [MRS-SEC-01][MRS-SEC-02].
- A5 -- Invalid combination (e.g. trigger references a non-existent action, port not present on device): The CLI
       rejects the command with a descriptive validation error. *(Validation rules governed by ASSUMPTION-19.)*
---------------------------------------------------------------

#### UC-02 -- Storm Detected on a Port (Automatic Protective Action)
**Actor:** System (Storm Control measurement and alarm subsystem).
**Trigger:** The aggregated ingress frame rate on a monitored port exceeds the configured threshold for the
      configured measurement window [MRS-001][MRS-002].
**Pre-conditions:**
- A 'storm-detect' trigger and associated action are configured on the port (UC-01).
- The port is operationally up and forwarding.
**Main flow:**
1. The system samples aggregated ingress frame rate (broadcast + multicast + unknown unicast) on the monitored
      port [MRS-001][MRS-020]. *(Sampling window per ASSUMPTION-02.)*
2. The measured rate exceeds the configured threshold [MRS-002]. *(Hysteresis/debounce per ASSUMPTION-03.)*
3. The system fires the 'storm-detect' trigger and activates the configured actions:
a. The port is placed in operationally-down state (port shutdown) [MRS-003].
b. The port LED alarm indicator is activated [MRS-005].
c. A system log entry is written, identifying the affected port and trigger type, with UTC timestamp [MRS-004][
      MRS-SEC-03][MRS-SEC-04].
4. The system starts the persistence timer if a non-zero 'persistent' value is configured [MRS-006].
5. Forwarding on all other ports continues unaffected [MRS-009].
```

```
6. The management CLI remains accessible during the event [MRS-SEC-05].
**Post-conditions:**
- The affected port is operationally down; no frames are transmitted or received on it [MRS-003].
- The LED alarm indicator is active [MRS-005].
- The trigger event is durably logged [MRS-SEC-03].
**Alternative / error flows:**
- A1 -- ‘persistent 0‘ configured: No auto-recovery timer is started; the port remains down until manually re-
     enabled (UC-05) [MRS-006][MRS-007].
- A2 -- Storm still present at persistence-timer expiry: Behaviour governed by ASSUMPTION-06.
---------------------------------------------------------------

#### UC-03 -- Automatic Recovery After Persistence Interval
**Actor:** System.
**Trigger:** The persistence timer for a previously shut-down port expires [MRS-006].
**Pre-conditions:**
- The port was placed in operationally-down state by a ‘storm-detect‘ trigger (UC-02).
- The configured ‘persistent‘ value is non-zero [MRS-006].
**Main flow:**
1. The persistence timer expires.
2. The system evaluates the auto-recovery decision criteria. *(Criteria governed by ASSUMPTION-05.)*
3. The system re-enables the port to operationally-up state.
4. The system clears the LED alarm indicator for this trigger source [MRS-005]. *(LED arbitration with other
     active alarms governed by ASSUMPTION-08.)*
5. The system writes a recovery event to the system log, identifying the port and trigger, with UTC timestamp [
     MRS-012][MRS-SEC-04].
**Post-conditions:**
- The port is operationally up and forwarding (subject to other port-state controllers).
- The recovery event is logged [MRS-012].
**Alternative / error flows:**
- A1 -- Storm still present at timer expiry: Behaviour per ASSUMPTION-06 (re-trigger, remain down, or apply back
     -off).
- A2 -- Reboot occurred during persistence interval: Runtime state is not retained; the port starts in normal
     state [MRS-016].
---------------------------------------------------------------

#### UC-04 -- Display Storm Control Configuration and Status
**Actor:** Network engineer or read-only operator (authenticated The Software user).
**Trigger:** The operator wishes to inspect Storm Control configuration, current trigger state, or recent events
     .
**Pre-conditions:**
- The operator is authenticated [MRS-SEC-01].
- The operator holds at least a read-only role [MRS-SEC-02].
**Main flow:**
1. The operator issues the show command for storm control / alarms [MRS-010].
2. The system displays the configured triggers, their port assignments, thresholds, persistence intervals,
     action references, and current trigger state (armed / fired / recovering). *(Exact output schema per
     ASSUMPTION-18.)*
**Post-conditions:** No system state changes.
**Alternative / error flows:**
- A1 -- Read-only user attempts to modify configuration: Command rejected [MRS-SEC-02].
---------------------------------------------------------------

#### UC-05 -- Manual Re-Enable of a Storm-Shutdown Port
**Actor:** Network engineer (authenticated administrator).
**Trigger:** A port is in storm-shutdown state and the operator wishes to restore it manually (typically with ‘
     persistent 0‘ configured, or to override an active persistence timer).
**Pre-conditions:**
- The operator is authenticated and authorised [MRS-SEC-01][MRS-SEC-02].
- The port is in operationally-down state due to a ‘storm-detect‘ trigger.
**Main flow:**
1. The operator issues the manual re-enable command. *(Exact CLI syntax per ASSUMPTION-07.)*
2. The system clears the trigger state for the port and cancels any active persistence timer.
3. The system re-enables the port to operationally-up state.
4. The system clears the LED alarm indicator for this trigger source [MRS-005].
5. The system writes an audit log entry with UTC timestamp and user identity [MRS-SEC-03][MRS-SEC-04].
**Post-conditions:** The port is operationally up; the manual re-enable is logged.
**Alternative / error flows:**
- A1 -- Storm still present: The trigger fires again immediately upon re-enable, returning to UC-02.
---------------------------------------------------------------

#### UC-06 -- Operator Inserts Incompatible SFP (SFP Module X) on a Storm-Controlled Port
**Actor:** Field technician / network engineer.
**Trigger:** A SFP Module X module is inserted into a port that has Storm Control configured, OR Storm Control
     is configured on a port that already carries a SFP Module X.
**Pre-conditions:** The device is a the supported hardware family switch [MRS-019].
**Main flow:**
1. The system detects the SFP type at insertion or at configuration time. *(Detection points governed by
     ASSUMPTION-11.)*
2. The system emits a warning (CLI message and/or log entry) stating that the SFP Module X is not compatible
     with Storm Control [MRS-011][MRS-018]. *(Exact wording per ASSUMPTION-11.)*
**Post-conditions:** The warning is recorded; configuration is not blocked.
**Alternative / error flows:** None.
---------------------------------------------------------------
```

```
#### UC-07 -- Attacker / Unauthorised User Attempts to Disable Storm Control
**Actor:** Attacker, or any unauthenticated/unauthorised user.
**Trigger:** An attempt to modify storm-control configuration, clear an active trigger, or manually re-enable a
     storm-shutdown port without proper credentials.
**Pre-conditions:** None.
**Main flow:**
1. The user issues a configuration or state-modifying command.
2. The system rejects the command because the session is unauthenticated [MRS-SEC-01] or the user lacks the
     administrative role [MRS-SEC-02].
3. The system writes an audit log entry with UTC timestamp recording the rejected attempt [MRS-SEC-03][MRS-SEC
     -04]. *(Audit field schema per ASSUMPTION-23.)*
**Post-conditions:** Storm-control configuration and runtime state are unchanged.
**Alternative / error flows:** None.
---------------------------------------------------------------

#### UC-08 -- Device Reboot While Port is in Storm-Shutdown State
**Actor:** System.
**Trigger:** Power cycle or commanded reboot while one or more ports are in storm-shutdown state with an active
     persistence timer.
**Pre-conditions:** Storm-control configuration has been saved to startup-config [MRS-016].
**Main flow:**
1. The device reboots.
2. The system loads the persisted storm-control configuration from startup-config [MRS-016].
3. All ports start in normal (operationally-up) state; no persistence timer is restored [MRS-016].
4. Storm-control monitoring resumes per the persisted configuration.
**Post-conditions:** Configuration is intact; runtime state is reset.
**Alternative / error flows:** None.
---------------------------------------------------------------

### 3.1 CLI and Configuration
This section enumerates every configurable parameter and the CLI surface implied by the MRS. CLI command names,
     keywords, and grammar shown below are proposals modelled on the CLI fragments cited in the MRS ([MRS-003] ‘
     action port‘, [MRS-005] ‘target port led‘, [MRS-008] alarm-framework pattern, [MRS-010] ‘show alarm‘).
     Every code block carries the required disclaimer; final command syntax must be confirmed against the The
     Software CLI conventions document. [TBD - verify with The Company doc team: The Software CLI Reference
     Manual]

#### 3.1.1 Parameter Inventory

[TABLE]
# | Parameter | Description | Valid range / unit | Default | MRS source
1 | ‘threshold‘ | Aggregated ingress frame-rate threshold above which the ‘storm-detect‘ trigger fires. | [TBD -
      PM/PO confirmation required] integer fps; see ASSUMPTION-01 | [TBD - PM/PO confirmation required]; see
     ASSUMPTION-01 | [MRS-002]
2 | ‘persistent‘ | Duration the port remains shut down before auto-recovery is considered; ‘0‘ disables auto-
     recovery. | [TBD - PM/PO confirmation required] integer seconds; see ASSUMPTION-04 | 300 s (per MRS
     documentation example) [MRS-007] | [MRS-006][MRS-007]
3 | ‘port‘ (trigger scope) | Port or port-list on which the trigger monitors ingress traffic. | Valid the
     supported hardware family port identifiers; range/list semantics see ASSUMPTION-20 | none (mandatory) | [
     MRS-001][MRS-008]
4 | Action reference | Reference from ‘storm-detect‘ trigger to a previously defined action (e.g. ‘target port
     led‘, port shutdown). | Existing action identifier | none (mandatory) | [MRS-003][MRS-005][MRS-008]
5 | Action target -- port shutdown | Action that places the affected port in operationally-down state. | Boolean
      (action present / absent) | absent | [MRS-003]
6 | Action target -- LED | Action that activates the device LED alarm indicator. | Boolean (action present /
     absent) | absent | [MRS-005]
7 | Action target -- log | Action that writes a system log entry on trigger fire. | Boolean (action present /
     absent) | absent (but mandatory per [MRS-004]) | [MRS-004]
8 | Measurement window / sampling interval | Time window over which ingress fps is averaged or sampled. | [TBD -
      PM/PO confirmation required]; see ASSUMPTION-02 | [TBD - PM/PO confirmation required]; see ASSUMPTION-02 |
      [MRS-001][MRS-002]
9 | Hysteresis / debounce | Number of consecutive samples required before the trigger fires. | [TBD - PM/PO
     confirmation required]; see ASSUMPTION-03 | [TBD - PM/PO confirmation required]; see ASSUMPTION-03 |
     derived from [MRS-002]
[/TABLE]

> Note on parameters 8 and 9: These parameters are not exposed by the MRS as user-configurable. They may be
     implemented as fixed internal values rather than CLI parameters. Final exposure decision is governed by
     ASSUMPTION-02 and ASSUMPTION-03.

#### 3.1.2 Configure an Action with LED and Port-Shutdown Targets
[SYNTAX PROPOSAL - verify against The Software CLI conventions]
configure
  alarm
    action 1
      target port led
      action port
      target log
    end
  end
Notes:
‘target port led‘ is taken verbatim from [MRS-005].
‘action port‘ (port shutdown) is taken verbatim from [MRS-003].
‘target log‘ is a proposed keyword to bind the log-entry action mandated by [MRS-004]; verify exact spelling
```

```
     against the The Software alarm framework. [TBD - verify with The Company doc team: alarm-framework action
     keywords]

#### 3.1.3 Configure a ‘storm-detect‘ Trigger
[SYNTAX PROPOSAL - verify against The Software CLI conventions]
configure
  alarm
    trigger storm-detect 1
      action 1
      port 1-2
      threshold <fps>            ! integer; range per ASSUMPTION-01
      persistent <seconds>       ! integer; 0 = manual re-enable only; range per ASSUMPTION-04
    end
  end
Notes:
The ‘trigger storm-detect‘, ‘action‘, ‘port‘, ‘threshold‘, and ‘persistent‘ keywords are taken from the MRS
     configuration pattern [MRS-008] and the example values cited in [MRS-006][MRS-007].
Port-list semantics (‘1-2‘, comma-separated, ‘all‘) are governed by ASSUMPTION-20.

#### 3.1.4 Remove or Modify a Trigger
[SYNTAX PROPOSAL - verify against The Software CLI conventions]
configure
  alarm
    no trigger storm-detect 1
  end
Behaviour when a trigger is removed or modified while one of its ports is in storm-shutdown state is governed by
      ASSUMPTION-21.

#### 3.1.5 Manual Re-Enable of a Storm-Shutdown Port
The MRS references manual re-enable [MRS-006][MRS-007] but does not specify the exact command. Two candidate
     forms are proposed; the final form is governed by ASSUMPTION-07.
[SYNTAX PROPOSAL - verify against The Software CLI conventions]
! Candidate A -- clear the trigger state for a port
clear alarm storm-detect port 1
! Candidate B -- administratively re-enable the port (standard interface command)
configure
  port 1
    no shutdown
  end

#### 3.1.6 Display Storm Control Configuration and Status
The MRS specifies ‘show alarm‘ "or equivalent" [MRS-010]. The canonical command(s) and output schema are
     governed by ASSUMPTION-18.
[SYNTAX PROPOSAL - verify against The Software CLI conventions]
! Display all alarm triggers, including storm-detect
show alarm
! Display storm-detect triggers and per-port runtime state (proposed dedicated view)
show alarm storm-detect
show alarm storm-detect port 1
The output shall include, per ‘storm-detect‘ trigger: trigger ID, configured port(s), threshold (fps),
     persistence (seconds), referenced action, current state (armed / fired / recovering), and timestamp of the
     most recent trigger or recovery event [MRS-010][MRS-SEC-04]. Exact field names and column layout per
     ASSUMPTION-18.

#### 3.1.7 Persistence and Reboot Behaviour
No CLI command is required for reboot persistence beyond the standard The Software ‘copy running-config startup-
     config‘ mechanism [TBD - verify with The Company doc team: The Software save-config command]. After reboot
     all ports start in normal state and any in-flight persistence timer is discarded [MRS-016].

#### 3.1.8 SFP SFP Module X Incompatibility Warning
The CLI shall emit a human-readable warning when a ‘storm-detect‘ trigger is configured on a port equipped with
     the SFP Module X, and when a SFP Module X is inserted into a port that already has a ‘storm-detect‘ trigger
      configured [MRS-011][MRS-018]. Detection points and exact wording are governed by ASSUMPTION-11.
[SYNTAX PROPOSAL - verify against The Software CLI conventions]
! Example warning text (proposed; final wording per ASSUMPTION-11)
Warning: SFP module SFP Module X on port <N> is not compatible with Storm Control.
         Storm Control measurement on this port may be inaccurate or non-functional.

#### 3.1.9 Authorisation
All commands in sections 3.1.2 through 3.1.5 and 3.1.7 (configuration-modifying) shall require an authenticated
     session [MRS-SEC-01] with an administrative role [MRS-SEC-02]. The commands in section 3.1.6 (read-only ‘
     show‘) shall be available to read-only roles [MRS-SEC-02]. The exact role names mapped to "administrative"
     and "read-only" are governed by ASSUMPTION-22.

### 4. Functional Requirements
This section enumerates the externally observable functional requirements for the Storm Control feature. Each
     requirement carries traceability tags to the MRS Requirement Inventory (section 1.1 of the approved MRS
     analysis) and, where applicable, to IEC 62443-4-2 Foundational Requirements. Numeric thresholds, ranges,
     and behaviours that are not fixed by the MRS are recorded as ASSUMPTION-nn entries in Section 11; the FR
     text itself states a concrete behaviour.

#### 4.1 Detection
FR-001 [MRS-001] The product shall measure the aggregated ingress frame rate (broadcast + multicast + unknown
     unicast frames) on every port for which a ‘storm-detect‘ trigger is configured, expressed in frames per
```

```
      second (fps). The aggregated rate shall be computed over a 1-second sampling window [see ASSUMPTION-02].
FR-002 [MRS-001, MRS-009] The product shall maintain an independent ingress measurement per configured port,
      such that the measurement on one port does not influence the measurement, trigger evaluation, or forwarding
       state of any other port.
FR-003 [MRS-002] The product shall fire the ‘storm-detect‘ trigger for a port when the measured aggregated
      ingress rate, evaluated over the sampling window defined in FR-001, exceeds the configured threshold value
      in fps for two consecutive sampling windows [see ASSUMPTION-03].
FR-004 [MRS-002, MRS-020] The product shall evaluate the ‘storm-detect‘ trigger against the combined broadcast +
       multicast + unknown-unicast frame rate only; per-class thresholds shall not be supported in this release.
FR-005 [MRS-002] The product shall accept threshold values in the inclusive range 1 fps to 1 000 000 fps and
      shall reject any other input via the CLI parameter validator with a non-zero exit status and a descriptive
      error message [see ASSUMPTION-01].

#### 4.2 Action
FR-006 [MRS-003] When the ‘storm-detect‘ trigger fires for a port, the product shall place that port in an
      operationally down state such that no frames are transmitted or received on that port, and shall maintain
      that state until a recovery condition defined in FR-010 or FR-012 is met.
FR-007 [MRS-005] When the ‘storm-detect‘ trigger fires, the product shall activate the device alarm LED
      indication via the platform’s LED control mechanism [TBD - cross-reference The Company internal LED
      specification] and shall keep it active for the affected port until that port returns to the normal (non-
      fired) state [see ASSUMPTION-08].
FR-008 [MRS-009] A ‘storm-detect‘ action invoked on one port shall not modify the operational, administrative,
      or forwarding state of any other port on the device.

#### 4.3 Recovery
FR-009 [MRS-006, MRS-007] The product shall accept a ‘persistent‘ parameter on the ‘storm-detect‘ trigger
      expressed in seconds, in the inclusive range 0 s to 86 400 s, where the value 0 disables auto-recovery [see
       ASSUMPTION-04].
FR-010 [MRS-006] When the ‘persistent‘ parameter is non-zero, the product shall start a per-port persistence
      timer at the moment the port is shut down by the trigger, and on expiry of that timer shall re-enable the
      port (return it to the normal operational state) provided the measured aggregated ingress rate has remained
       at or below the configured threshold for the final sampling window of the persistence interval [see
      ASSUMPTION-05].
FR-011 [MRS-006] If, on expiry of the persistence timer, the measured aggregated ingress rate still exceeds the
      configured threshold, the product shall keep the port operationally down and shall restart the persistence
      timer [see ASSUMPTION-06].
FR-012 [MRS-006] When the ‘persistent‘ parameter is 0, the product shall keep the port operationally down until
      an authenticated administrative operator issues the manual re-enable command defined in section 3.1 [see
      ASSUMPTION-07].
FR-013 [MRS-016] Configuration parameters of every ‘storm-detect‘ trigger and its referenced action (port
      assignment, threshold, persistence interval, action target) shall be persisted to startup-config such that
      they survive a device reboot, while runtime trigger state and any in-flight persistence timer shall not be
      retained, and every port shall start in the normal operational state after boot.

#### 4.4 Logging
FR-014 [MRS-004, MRS-SEC-03, MRS-SEC-04] [FR2-UC] When the ‘storm-detect‘ trigger fires, the product shall write
       one event to the system log containing: UTC timestamp, event identifier ‘STORM-DETECT-FIRED‘, affected
      port identifier, configured threshold (fps), measured rate (fps), and the trigger reference [see ASSUMPTION
      -09].
FR-015 [MRS-012, MRS-SEC-04] [FR2-UC] When a port is automatically re-enabled after the persistence interval
      expires (FR-010), the product shall write one event to the system log containing: UTC timestamp, event
      identifier ‘STORM-DETECT-RECOVERED‘, affected port identifier, and the trigger reference [see ASSUMPTION
      -10].
FR-016 [MRS-SEC-03, MRS-SEC-04] [FR2-UC] Every CLI command that creates, modifies, or deletes a ‘storm-detect‘
      trigger or its referenced action shall produce one audit log entry containing: UTC timestamp, authenticated
       user identity, source of the management session, command name, and the parameter values supplied [see
      ASSUMPTION-23].

#### 4.5 Configuration
FR-017 [MRS-008] The product shall provide CLI commands to create, modify, and delete a ‘storm-detect‘ trigger
      including its port selection, threshold, persistence interval, and referenced action, using the existing
      The Software alarm trigger/action configuration pattern as defined in section 3.1.
FR-018 [MRS-010] The product shall provide a single canonical CLI ‘show‘ command (‘show alarm storm-detect‘)
      that displays, for every configured ‘storm-detect‘ trigger: trigger ID, configured port(s), threshold,
      persistence interval, referenced action, current state (armed / fired / recovering), and the UTC timestamp
      of the most recent fire or recovery event [see ASSUMPTION-18].
FR-019 [MRS-008] The product shall validate the configuration of any ‘storm-detect‘ trigger and reject, with a
      descriptive CLI error message and a non-zero exit status, any of the following: action reference that does
      not exist, trigger without an action reference, port identifier not present on the device, and overlapping
      port lists across multiple triggers [see ASSUMPTION-19].

#### 4.6 SFP and Hardware Constraints
FR-020 [MRS-011, MRS-018, MRS-019] The product shall emit a CLI warning message when a ‘storm-detect‘ trigger is
       configured on a port that is currently equipped with the SFP Module X module, and when a SFP Module X
      module is inserted into a port that already has a ‘storm-detect‘ trigger configured. The warning shall
      identify the affected port and shall state that storm control is not compatible with the SFP Module X [see
      ASSUMPTION-11].
FR-021 [MRS-019] The product shall accept ‘storm-detect‘ configuration only on the supported hardware family
      devices; on non-supported hardware platforms the CLI shall reject the configuration with a descriptive
      error message.
FR-022 [MRS-021] When a ‘storm-detect‘ trigger is applied to a port that is administratively disabled, the
      product shall log a warning event identifying the port and the known limitation that the port may
      transition to enabled in the absence of a storm [see ASSUMPTION-15].
```

```
#### 4.7 Security
FR-023 [MRS-SEC-01] [FR1-IAC] The product shall reject any attempt to create, modify, delete, or display the
      runtime state of a ‘storm-detect‘ trigger or its referenced action from an unauthenticated management
      session.
FR-024 [MRS-SEC-02] [FR2-UC] The product shall permit creation, modification, and deletion of ‘storm-detect‘
      triggers and their referenced actions only from a session authenticated to a role with administrative
      privilege, and shall permit display of ‘storm-detect‘ configuration and state from sessions authenticated
      to roles with read-only or higher privilege [see ASSUMPTION-22].
FR-025 [MRS-016] [FR3-SI] Storm control configuration persisted to startup-config shall be covered by the
      platform’s startup-config integrity protection mechanism [TBD - cross-reference The Company internal
      startup-config integrity specification], such that tampering with persisted storm control parameters is
      detectable on the next boot.
FR-026 [MRS-SEC-03] [FR6-TRE] Storm control audit events produced by FR-014, FR-015, and FR-016 shall be made
      available to the platform’s audit log accessibility mechanism, including remote syslog forwarding when
      remote syslog is configured on the device [see ASSUMPTION-17].
FR-027 [MRS-SEC-05] [FR7-RA] The product shall continue to evaluate the ‘storm-detect‘ trigger and shall keep
      the management CLI accepting and processing commands when every configured port is receiving aggregated
      ingress traffic at the device’s specified line rate, with a CLI command-prompt response time not exceeding
      2 s [see ASSUMPTION-25].
FR-028 [MRS-017] [FR7-RA] The product shall, at the time a ‘storm-detect‘ trigger is configured, verify that CPU
       bandwidth limiting is enabled on the device, and shall emit a CLI warning identifying the absence of CPU
      bandwidth limiting if it is not enabled [see ASSUMPTION-12].

#### 4.8 Error Handling and Edge Cases
FR-029 [MRS-008] When an operator modifies the threshold, persistence interval, action reference, or port list
      of a ‘storm-detect‘ trigger that has currently fired and shut down a port, the product shall immediately re
      -enable the affected port to the normal operational state, cancel any in-flight persistence timer, and re-
      arm the trigger with the new parameters [see ASSUMPTION-21].
FR-030 [MRS-008] When an operator deletes a ‘storm-detect‘ trigger that has currently fired and shut down a port
      , the product shall immediately re-enable the affected port to the normal operational state and cancel any
      in-flight persistence timer [see ASSUMPTION-21].
FR-031 [MRS-008] The product shall accept a port range expressed as ‘<low>-<high>‘ and a comma-separated port
      list ‘<p1>,<p2>,...‘ in ‘storm-detect‘ trigger configuration, and shall reject with a descriptive CLI error
       any specification that contains a port identifier not present on the device [see ASSUMPTION-20].
------------------------------------------------------------
MRS coverage check (MUST/SHOULD inventory IDs):
[MRS-001] FR-001, FR-002 [OK]
[MRS-002] FR-003, FR-004, FR-005 [OK]
[MRS-003] FR-006 [OK]
[MRS-004] FR-014 [OK]
[MRS-005] FR-007 [OK]
[MRS-006] FR-009, FR-010, FR-011, FR-012 [OK]
[MRS-007] FR-009 [OK]
[MRS-008] FR-017, FR-029, FR-030, FR-031 [OK]
[MRS-009] FR-002, FR-008 [OK]
[MRS-010] FR-018 [OK]
[MRS-011] FR-020 [OK] (SHOULD)
[MRS-012] FR-015 [OK] (SHOULD)
[MRS-016] FR-013, FR-025 [OK]
[MRS-017] FR-028 [OK]
[MRS-018] FR-020 [OK]
[MRS-019] FR-020, FR-021 [OK]
[MRS-020] FR-004 [OK]
[MRS-021] FR-022 [OK]
[MRS-SEC-01] FR-023 [OK]
[MRS-SEC-02] FR-024 [OK]
[MRS-SEC-03] FR-014, FR-016, FR-026 [OK]
[MRS-SEC-04] FR-014, FR-015 [OK]
[MRS-SEC-05] FR-027 [OK]
[MRS-013, MRS-014, MRS-015] - COULD items, deferred to Section 2 "Not Committed Market Requirements" [OK]
IEC 62443-4-2 coverage check (Partial/Missing FRs from MRS Security Assessment):
FR1-IAC (Partial) - FR-023 [OK]
FR2-UC (Partial) - FR-014, FR-015, FR-016, FR-024 [OK]
FR3-SI (Missing) - FR-025 [OK]
FR6-TRE (Partial) - FR-026 [OK]
FR7-RA (Partial) - FR-027, FR-028 [OK]

### 6. Acceptance Criteria
The following acceptance criteria are written in Gherkin format. Each scenario references the originating FR-ID.
       All numeric thresholds and intervals shown in scenarios reflect the corresponding ASSUMPTION-nn entries in
       section 11 and require PM/PO confirmation before final test plan sign-off.
------------------------------------------------------------
Scenario: FR-001 - per-port aggregated ingress monitoring active
Given a the supported hardware family device with The Software booted and a ‘storm-detect‘ trigger configured on
       port 1 with threshold 500 fps
When the device is in steady state with no storm condition
Then the product shall actively measure the aggregated ingress fps (broadcast + multicast + unknown unicast) on
      port 1
And the trigger status displayed via ‘show alarm‘ shall be reported as "armed"
Scenario: FR-002 - monitoring is per-port and independent
Given ports 1 and 2 both configured with a ‘storm-detect‘ trigger at 500 fps
When port 1 receives sustained ingress at 1000 fps and port 2 receives 100 fps
Then port 1 shall be shut down by the configured action
And port 2 shall remain operationally up and shall continue to forward traffic without interruption
```

```
Scenario: FR-003 - threshold exceeded fires trigger
Given a ‘storm-detect‘ trigger on port 3 with threshold 500 fps and persistent 300
When the aggregated ingress rate on port 3 exceeds 500 fps for the configured measurement window [ASSUMPTION-02,
      ASSUMPTION-03]
Then the trigger shall fire and the configured actions shall be activated within the maximum detection latency [
     ASSUMPTION-22]
Scenario: FR-004 - aggregated traffic class measurement
Given a ‘storm-detect‘ trigger on port 4 with threshold 500 fps
When the port receives 200 fps broadcast, 200 fps multicast, and 200 fps unknown-unicast simultaneously (
     aggregate 600 fps)
Then the trigger shall fire because the aggregated rate exceeds the threshold
Scenario: FR-005 - traffic below threshold does not fire
Given a ‘storm-detect‘ trigger on port 5 with threshold 500 fps
When the aggregated ingress rate stays at 499 fps for 60 seconds
Then the trigger shall not fire and port 5 shall remain operationally up
Scenario: FR-006 - port shutdown action executes
Given a fired ‘storm-detect‘ trigger on port 6 with ‘action port‘ configured
When the action executes
Then port 6 shall be placed in operationally down state
And no frames shall be transmitted or received on port 6 until a recovery condition is met
Scenario: FR-007 - LED indication on storm
Given a fired ‘storm-detect‘ trigger on port 7 with ‘target port led‘ configured
When the action executes
Then the device LED alarm indicator shall activate [ASSUMPTION-08]
And the LED shall remain activated until the port returns to the normal operational state
Scenario: FR-008 - storm on one port does not affect others
Given 8 ports on a Hardware Platform A device, each with ‘storm-detect‘ trigger configured
When port 1 receives a storm exceeding its threshold and is shut down
Then ports 2 through 8 shall continue to forward traffic without measurable disruption (no additional port state
      changes, no MAC table flush events on unaffected ports)
Scenario: FR-009 - persistence timer governs auto-recovery duration
Given a ‘storm-detect‘ trigger on port 9 with ‘persistent 300‘
When the trigger fires at time T0 and shuts down port 9
Then port 9 shall remain in operationally down state for at least 300 seconds from T0 [ASSUMPTION-04]
Scenario: FR-010 - auto-recovery when storm has cleared
Given port 10 was shut down by a fired ‘storm-detect‘ trigger with ‘persistent 300‘
When 300 seconds have elapsed and the ingress rate is below threshold [ASSUMPTION-05]
Then port 10 shall be automatically re-enabled to the normal operational state
And the trigger shall be re-armed
Scenario: FR-011 - persistence timer expiry while storm is still present
Given port 11 was shut down by a fired ‘storm-detect‘ trigger with ‘persistent 60‘
When 60 seconds have elapsed and the ingress rate is still above threshold [ASSUMPTION-06]
Then the port shall remain shut down (or be re-enabled and immediately re-shut down) per the defined recovery-vs
     -storm interaction [ASSUMPTION-06]
And a log entry shall record the failed recovery attempt
Scenario: FR-012 - persistent 0 disables auto-recovery
Given a ‘storm-detect‘ trigger on port 12 with ‘persistent 0‘
When the trigger fires and port 12 is shut down
Then port 12 shall remain operationally down indefinitely until an authenticated operator issues the manual re-
     enable command [ASSUMPTION-07]
Scenario: FR-013 - configuration persists across reboot
Given an operator has configured a ‘storm-detect‘ trigger on port 13 with threshold 500 fps and ‘persistent
     300‘, and saved to startup-config
When the device is rebooted
Then after boot the trigger shall be present in running-config with identical parameters
And port 13 shall start in the normal operational state regardless of pre-reboot triggered state
Scenario: FR-014 - trigger event is logged with timestamp
Given a ‘storm-detect‘ trigger configured on port 14
When the trigger fires
Then a log entry shall be written to the system log containing: UTC timestamp, port identifier ("port 14"),
     trigger type ("storm-detect"), event type ("triggered"), and severity [ASSUMPTION-09]
Scenario: FR-015 - auto-recovery event is logged
Given port 15 was shut down by storm control and is auto-recovered after persistence expiry
When the port is re-enabled by the auto-recovery mechanism
Then a log entry shall be written containing UTC timestamp, port identifier, trigger type, and event type ("
     recovered") [ASSUMPTION-10]
Scenario: FR-016 - configuration change is logged with user identity
Given an authenticated administrator user "admin1"
When the user creates, modifies, or deletes a ‘storm-detect‘ trigger or its associated action
Then an audit log entry shall be written containing UTC timestamp, user identity "admin1", source (CLI session),
      the configuration command, and before/after values [ASSUMPTION-23]
Scenario: FR-017 - CLI configuration pattern accepted
Given an authenticated administrator at the CLI
When the user configures: (1) ‘action <id>‘ with ‘target port led‘, and (2) ‘trigger <id> storm-detect‘
     referencing the action with port selection, persistence, and threshold parameters [SYNTAX PROPOSAL - verify
      against The Software CLI conventions]
Then both commands shall be accepted and reflected in running-config
Scenario: FR-017 NEGATIVE - trigger references non-existent action
Given no action with id 5 has been configured
When the user enters ‘trigger 1 storm-detect action 5 ...‘ [SYNTAX PROPOSAL - verify against The Software CLI
     conventions]
Then the CLI shall reject the command with a descriptive error referencing the missing action [ASSUMPTION-19]
Scenario: FR-018 - show alarm displays storm-detect status
Given a ‘storm-detect‘ trigger configured on port 18
```

```
When the operator runs ‘show alarm‘ [SYNTAX PROPOSAL - verify against The Software CLI conventions]
Then the output shall include for the storm-detect trigger: trigger id, port(s), threshold, persistence interval
     , current state (armed/triggered), last-trigger UTC timestamp [ASSUMPTION-18]
Scenario: FR-020 - warning when configuring storm control on a port using SFP SFP Module X
Given port 20 is equipped with an SFP module identified as SFP Module X
When the operator configures a ‘storm-detect‘ trigger on port 20
Then the CLI shall display a warning message indicating that storm control is not compatible with SFP SFP Module
      X [ASSUMPTION-11]
And the configuration shall still be accepted (warning, not blocking)
Scenario: FR-020 NEGATIVE - SFP SFP Module X inserted after configuration
Given a ‘storm-detect‘ trigger has been configured on port 20 while no SFP was inserted
When an SFP module identified as SFP Module X is subsequently inserted into port 20
Then a warning log entry and CLI banner (on the next interactive session) shall be emitted indicating storm
     control is not compatible with this SFP [ASSUMPTION-11]
Scenario: FR-021 - feature limited to the supported hardware family
Given a non-the supported hardware family device running The Software
When the operator attempts to configure a ‘storm-detect‘ trigger
Then the CLI shall reject the command with an error indicating the feature is supported only on the supported
     hardware family devices [MRS-019]
Scenario: FR-022 - disabled port re-enabled by trigger configuration (known issue)
Given port 22 is administratively disabled and no storm condition is present
When the operator configures a ‘storm-detect‘ trigger on port 22
Then the documented behaviour from [MRS-021] shall apply: the port becomes enabled in the absence of a storm
And the CLI shall display a warning informing the operator of this side-effect [ASSUMPTION-15]
Scenario: FR-023 - unauthenticated CLI rejected (security)
Given an unauthenticated CLI session
When the session attempts to enter ‘trigger <id> storm-detect ...‘ [SYNTAX PROPOSAL - verify against The
     Software CLI conventions]
Then the command shall be rejected and no change shall be applied to running-config or trigger state
And no audit log entry shall record a successful change
Scenario: FR-024 - read-only role cannot modify configuration (security)
Given an authenticated user assigned the read-only role [ASSUMPTION-23]
When the user attempts to create or modify a ‘storm-detect‘ trigger
Then the CLI shall reject the command with an authorization error
And the user shall still be able to execute ‘show alarm‘ successfully
Scenario: FR-025 - startup-config integrity covers storm control parameters
Given storm control configuration has been saved to startup-config
When the platform startup-config integrity verification runs at next boot
Then any tampering with storm control parameters in startup-config shall be detected by the platform integrity
     mechanism (FR3-SI / CR 3.04) [TBD - verify with The Company doc team: startup-config integrity mechanism]
Scenario: FR-026 - audit events available for remote forwarding
Given remote syslog forwarding is configured on the device
When a storm-detect trigger fires, recovers, or its configuration is changed
Then the corresponding audit log entry shall be forwarded to the configured remote syslog destination (FR6-TRE /
      CR 6.01)
Scenario: FR-027 - CLI remains accessible during storm
Given a ‘storm-detect‘ trigger configured on at least one port
When sustained ingress at line rate is applied to all configured ports simultaneously
Then the management CLI shall remain accessible and respond to a ‘show alarm‘ command within the defined
     response-time bound [ASSUMPTION-25]
And the storm-detect trigger shall still fire within the maximum detection latency [ASSUMPTION-22]
Scenario: FR-028 - CPU bandwidth limiting prerequisite warning
Given CPU bandwidth limiting has not been configured on the device
When the operator configures a ‘storm-detect‘ trigger
Then the CLI shall emit a warning advising that CPU bandwidth limiting is required for correct storm-control
     operation per [MRS-017] [ASSUMPTION-12]
Scenario: FR-029 - modifying parameters of a triggered (shut-down) port
Given port 29 is currently shut down due to a fired ‘storm-detect‘ trigger
When an authenticated administrator modifies the trigger threshold or persistence
Then port 29 shall be immediately re-enabled to the normal operational state
And the in-flight persistence timer shall be cancelled
And the trigger shall be re-armed with the new parameters [ASSUMPTION-21]
Scenario: FR-030 - deleting a trigger that has shut down a port
Given port 30 is currently shut down due to a fired ‘storm-detect‘ trigger
When an authenticated administrator deletes the trigger
Then port 30 shall be immediately re-enabled to the normal operational state
And the in-flight persistence timer shall be cancelled
And no trigger shall remain associated with port 30
Scenario: FR-031 - port range syntax accepted
Given an authenticated administrator at the CLI
When the user enters ‘trigger 1 storm-detect ... port 1-4‘ [SYNTAX PROPOSAL - verify against The Software CLI
     conventions]
Then the trigger shall be applied independently to ports 1, 2, 3, and 4
Scenario: FR-031 NEGATIVE - invalid port in range
Given a Hardware Platform A device with ports 1-10 only
When the user enters ‘trigger 1 storm-detect ... port 1-99‘ [SYNTAX PROPOSAL - verify against The Software CLI
     conventions]
Then the CLI shall reject the command with a descriptive error identifying the invalid port(s)
And no trigger shall be created [ASSUMPTION-19, ASSUMPTION-20]
Scenario: FR-003/FR-004 NEGATIVE - threshold value out of range
Given an authenticated administrator at the CLI
When the user enters a ‘threshold‘ value of 0 or a value above the documented maximum [ASSUMPTION-01]
Then the CLI shall reject the command with a descriptive error identifying the valid range
And no trigger shall be created or modified
```

```
Scenario: FR-009 NEGATIVE - persistence value out of range
Given an authenticated administrator at the CLI
When the user enters a ‘persistent‘ value above the documented maximum [ASSUMPTION-04]
Then the CLI shall reject the command with a descriptive error identifying the valid range
Scenario: FR-012 - manual re-enable after ‘persistent 0‘ shutdown
Given port 12 is shut down by storm control with ‘persistent 0‘ and the storm has cleared
When an authenticated administrator issues the manual re-enable command [ASSUMPTION-07]
Then port 12 shall be re-enabled to the normal operational state
And a recovery log entry shall be written with UTC timestamp and user identity [ASSUMPTION-10, ASSUMPTION-23]
-------------------------------------------------------------

### 7. Risks

[TABLE]
ID | Description | Likelihood | Impact | Mitigation
RISK-01 | **False positives from short legitimate traffic bursts.** Legitimate industrial protocols (e.g. PTP
      bursts, multicast discovery, RSTP topology changes) may briefly exceed the configured fps threshold and
      cause unintended port shutdown, disrupting plant operations. | Medium | High | Define a deterministic
      measurement window and hysteresis/debounce policy [ASSUMPTION-02, ASSUMPTION-03]; document recommended
      threshold ranges per use case in Reference Documents [TBD - verify with The Company doc team]; require
      operators to validate thresholds against measured baseline traffic before enabling ‘action port‘; provide ‘
      persistent 0‘ + manual re-enable for mission-critical links to force human review.
RISK-02 | **False negatives during sustained low-rate but high-impact storms.** A storm slightly below threshold
       can still degrade network performance without firing the trigger. | Medium | Medium | Encourage
      conservative threshold values in documentation; recommend layered protection (CPU bandwidth limiting per [
      MRS-017] / FR-028, RSTP/the Ring Protocol loop prevention); provide observability via ‘show alarm‘ (FR-018)
       so operators can review near-miss events.
RISK-03 | **Performance impact of measurement on device CPU under high load.** Per-port fps measurement on every
       configured port could consume CPU resources and degrade other control-plane functions during a real storm.
       | Medium | High | Implement measurement in hardware/datapath where the platform supports it [TBD - verify
      with The Company doc team: Hardware Platform A datapath capabilities]; mandate CPU bandwidth limiting
      prerequisite (FR-028); test against the DoS-resilience criteria of FR-027 with quantified CPU and CLI-
      response bounds [ASSUMPTION-25].
RISK-04 | **Hardware-platform variability across the supported hardware family models.** Different Hardware
      Platform A variants (e.g. Hardware Platform A vs. other the supported hardware family models) may differ in
       port counts, SFP cages, and datapath capabilities, leading to inconsistent feature behaviour. | Medium |
      Medium | Restrict initial release to the supported hardware family per [MRS-019] / FR-021; define and
      enforce per-model concurrency limits [ASSUMPTION-13]; run the full acceptance test suite on each supported
      Hardware Platform A variant before release; document supported models explicitly in Reference Documents.
RISK-05 | **SFP SFP Module X incompatibility produces silent protection gap.** If the operator misses the
      warning, storm control may appear configured but provide no protection on that port. | Medium | High |
      Implement warning at both configuration time and SFP insertion time (FR-020) [ASSUMPTION-11]; emit a
      periodic warning log entry as long as the incompatible SFP remains inserted on a port with ‘storm-detect‘
      configured; document prominently in user manual [TBD - verify with The Company doc team: user manual
      section]; consider elevating the SHOULD/MUST conflict resolution to MUST in a future release.
RISK-06 | **Operator misconfiguration causing legitimate-traffic outage.** A wrongly low threshold combined with
       ‘action port‘ and ‘persistent 0‘ can shut down a critical port until manual intervention. | High | High |
      Require CLI confirmation prompt or explicit acknowledgement when ‘persistent 0‘ is configured [TBD - PM/PO
      confirmation required]; provide clear ‘show alarm‘ output identifying triggered state and cause (FR-018);
      audit logging of configuration change with user identity and before/after values (FR-016) supports post-
      incident analysis; document recommended commissioning procedure in Reference Documents.
RISK-07 | **Disabled-port side-effect ([MRS-021] / FR-022) causes unexpected port enable.** Configuring a ‘storm
      -detect‘ trigger on an administratively disabled port enables the port, which can violate operator intent (
      e.g. unused ports left disabled for security). | Medium | Medium | Display a CLI warning at configuration
      time (FR-022) [ASSUMPTION-15]; document the limitation in user manual and release notes; track for
      resolution in a future release.
RISK-08 | **Interaction with redundancy protocols (RSTP/the Ring Protocol) and link aggregation undefined.**
      Storm-induced port shutdown may trigger redundancy failover loops or break LAG semantics if the interaction
       is not specified. | Medium | High | Define interaction rules with each port-state controller [ASSUMPTION
      -14]; run integration tests covering RSTP, the Ring Protocol, LAG, PoE, and 802.1X co-existence before
      release; document precedence in Reference Documents.
RISK-09 | **Auto-recovery flapping when storm persists past persistence expiry.** Repeated re-enable / re-shut-
      down cycles can flood logs and destabilise neighbouring devices’ MAC tables. | Medium | Medium | Define
      explicit recovery-vs-storm interaction including back-off if applicable [ASSUMPTION-06]; rate-limit
      recovery-failure log entries; expose the cycle count via ‘show alarm‘ for operator awareness.
RISK-10 | **Audit log volume under attack scenario.** A sustained attack causing repeated trigger/recovery
      cycles may flood the local log and hinder forensic review. | Low | Medium | Rely on platform audit log
      rotation [TBD - verify with The Company doc team]; ensure remote syslog forwarding (FR-026) is functional
      so logs survive local rotation; consider rate-limiting duplicate event entries [TBD - PM/PO confirmation
      required].
RISK-11 | **Quantitative DoS-resilience criteria undefined at FS time.** Without numeric bounds for CPU usage
      and CLI response time, FR-027 cannot be objectively verified at acceptance. | High | Medium | Resolve [
      ASSUMPTION-25] before test plan sign-off; engineering to define line-rate test conditions and measurable
      response-time ceilings; gate release on quantified test results.
RISK-12 | **Reliance on platform-wide controls that may not be fully verified for storm control scope.** FR3-SI
      (startup-config integrity) and FR2-UC session lock/non-repudiation are assumed to be inherited from The
      Software platform. If those platform controls regress, storm control’s IEC 62443-4-2 SL2 posture is
      impacted. | Low | High | Add platform-control verification steps to the storm-control acceptance test plan;
       coordinate with the The Software platform security owner to confirm coverage [TBD - verify with The
      Company doc team]; document the inheritance explicitly in Reference Documents.
[/TABLE]


### 8. Revision History
```

```
[TABLE]
Version | Date | Author | Description
0.1 (draft) | 2025-01-22 | Multi-Agent FS Pipeline | Initial draft generated from approved MRS analysis
[/TABLE]


### 9. Reference Documents

[TABLE]
Ref-ID | Title | Source | Version / Date
[R1] | Software Market Requirement Specification - Storm Control - The Software (Document No: MRS-SC-001) | [MRS
     ] | 1.0
[R2] | IEC 62443-4-2 - Security for industrial automation and control systems - Technical security requirements
     for IACS components | [RAG] | [TBD]
[R3] | [TBD - verify with The Company doc team: The Software Alarm Framework / trigger-action specification] | [
     TBD] | [TBD]
[R4] | [TBD - verify with The Company doc team: The Software CLI Reference Manual] | [TBD] | [TBD]
[R5] | [TBD - verify with The Company doc team: The Software Role-Based Access Control / user role specification
     ] | [TBD] | [TBD]
[R6] | [TBD - verify with The Company doc team: The Software Audit Logging / system log schema specification] |
     [TBD] | [TBD]
[R7] | [TBD - verify with The Company doc team: The Software Startup-Config Integrity Protection specification]
     | [TBD] | [TBD]
[R8] | [TBD - verify with The Company doc team: The Software Release Notes vX.Y, section N (Storm Control)] | [
     TBD] | [TBD]
[R9] | [TBD - verify with The Company doc team: IEC 62443-4-2 SL2 certificate [Internal Certificate Ref]
     referenced by the MRS] | [TBD] | [TBD]
[R10] | [TBD - verify with The Company doc team: the supported hardware family hardware specification (incl. LED
      mapping)] | [TBD] | [TBD]
[R11] | [TBD - verify with The Company doc team: SFP module compatibility list, including SFP Module X] | [TBD]
     | [TBD]
[R12] | [TBD - propose external reference for The Company review: syslog message format / severity convention] |
      [TBD] | [TBD]
[R13] | [TBD - propose external reference for The Company review: SNMP trap / MIB convention applicable to storm
     -detect events] | [TBD] | [TBD]
[/TABLE]


### 10. Terms and Abbreviations

[TABLE]
Term | Definition
FR | Functional Requirement (in IEC 62443-4-2 context: Foundational Requirement)
MRS | Market Requirement Specification
FS | Functional Specification
fps | Frames per second
SFP | Small Form-factor Pluggable transceiver
CLI | Command-Line Interface
PM | Product Manager
PO | Product Owner
IEC 62443 | International standard series for cybersecurity of industrial automation and control systems
SL | Security Level (as defined in IEC 62443-4-2)
SL2 | Security Level 2 (target security level for this feature)
CR | Component Requirement (as used in IEC 62443-4-2)
RBAC | Role-Based Access Control
IAC | Identification and Authentication Control (IEC 62443-4-2 FR1)
UC | Use Control (IEC 62443-4-2 FR2)
SI | System Integrity (IEC 62443-4-2 FR3)
DC | Data Confidentiality (IEC 62443-4-2 FR4)
RDF | Restricted Data Flow (IEC 62443-4-2 FR5)
TRE | Timely Response to Events (IEC 62443-4-2 FR6)
RA | Resource Availability (IEC 62443-4-2 FR7)
MoSCoW | Prioritisation scheme: Must / Should / Could / Won't have
RSTP | Rapid Spanning Tree Protocol
the Ring Protocol | Fast Ring Network Topology (The Company ring redundancy protocol referenced in the MRS
     context)
LAG | Link Aggregation Group
VLAN | Virtual Local Area Network
SNMP | Simple Network Management Protocol
NIC | Network Interface Card
FDB | Forwarding Database
LED | Light-Emitting Diode (front-panel indicator)
PoE | Power over Ethernet
802.1X | Port-based network access control protocol
DoS | Denial of Service
UTC | Coordinated Universal Time
MIB | Management Information Base
TBD | To Be Determined
HITL | Human-In-The-Loop (review checkpoint)
Storm Control | The feature specified by this FS - per-port monitoring of aggregated broadcast/multicast/unknown
     -unicast ingress traffic with configurable threshold and protective actions
storm-detect | Name of the The Software alarm trigger type used by storm control (per [MRS-008])
persistent | The trigger parameter (in seconds) that determines auto-recovery behaviour (per [MRS-006])
```

```
the supported hardware family | The The Company industrial switch product family (e.g. Hardware Platform A)
     supported in this release (per [MRS-019])
SFP Module X | The Company SFP module part number that is not compatible with storm control (per [MRS-011]/[MRS
     -018])
startup-config | Persisted device configuration loaded at boot (per [MRS-016])
[/TABLE]


### 11. Assumptions
ASSUMPTION-01: Threshold value range is assumed to be 1-1,000,000 fps (integer), with no implicit default (the
     parameter shall be mandatory at trigger configuration time); out-of-range values shall be rejected with a
     CLI error - this range covers expected industrial Layer-2 traffic loads at line rate on the supported
     hardware family ports while excluding meaningless values - PM/PO confirmation required before
     implementation.
ASSUMPTION-02: The threshold measurement window is assumed to be a 1-second non-overlapping sampling interval;
     the trigger evaluation uses the frame count accumulated over this window - a 1-second window is a common
     industrial-switch convention that gives deterministic latency without excessive CPU cost - PM/PO
     confirmation required before implementation.
ASSUMPTION-03: Trigger fire hysteresis is assumed to be a single-sample model (the trigger fires the first 1-
     second window in which the measured rate exceeds the threshold); no multi-sample debounce is applied in
     this release - this matches the MRS wording "exceeds the configured threshold" without introducing
     parameters not present in the MRS - PM/PO confirmation required before implementation.
ASSUMPTION-04: Persistence interval range is assumed to be 0-86400 seconds (integer), where 0 disables auto-
     recovery and requires manual re-enable as stated in [MRS-006]/[MRS-007] - 86400 s (24 h) is a practical
     upper bound for operator-driven recovery scheduling - PM/PO confirmation required before implementation.
ASSUMPTION-05: Auto-recovery decision criterion is assumed to be unconditional time-based re-enable: at expiry
     of the persistence timer the port is re-enabled regardless of current measured ingress rate - this matches
     the simplest interpretation of [MRS-006] and avoids dependency on undefined hysteresis - PM/PO confirmation
      required before implementation.
ASSUMPTION-06: Behaviour when the storm persists past persistence-timer expiry is assumed to be: the port re-
     enables, the storm-detect trigger continues to evaluate, and if the threshold is again exceeded the trigger
      fires again, restarting the persistence timer (no exponential back-off in this release) - this is
     consistent with ASSUMPTION-05 and the existing alarm framework semantics - PM/PO confirmation required
     before implementation.
ASSUMPTION-07: The manual re-enable CLI command is assumed to be the standard The Software interface-
     administrative-enable command (conceptually ‘no shutdown‘ on the affected port); no storm-control-specific
     clear command is introduced in this release - this re-uses existing The Software port-administration
     semantics - [SYNTAX PROPOSAL - verify against The Software CLI conventions] - PM/PO confirmation required
     before implementation.
ASSUMPTION-08: The "device LED alarm indicator" referenced in [MRS-005] is assumed to be the the supported
     hardware family front-panel alarm LED (single shared alarm indicator); when multiple alarm sources are
     simultaneously active the LED remains active until all sources clear; the LED is on (non-blinking) while
     any alarm condition is active - this preserves existing LED arbitration in the alarm framework - PM/PO
     confirmation required before implementation.
ASSUMPTION-09: The system log entry for a storm-detect trigger is assumed to use severity "warning", facility
     consistent with other The Software alarm framework events, and a structured message including: UTC
     timestamp, port identifier, trigger type ("storm-detect"), measured rate (fps), configured threshold (fps),
      and resulting action(s) - this provides the context required by CR 2.08/CR 2.11 - PM/PO confirmation
     required before implementation.
ASSUMPTION-10: The auto-recovery log entry is assumed to use severity "notice", facility consistent with other
     The Software alarm framework events, and include: UTC timestamp, port identifier, event type ("storm-detect
     -recovery"), and elapsed persistence interval - this satisfies [MRS-012] - PM/PO confirmation required
     before implementation.
ASSUMPTION-11: SFP SFP Module X incompatibility detection is assumed to fire at three points: (a) at trigger
     configuration time if the SFP is currently inserted on the configured port, (b) at SFP insertion time on a
     port already carrying a storm-detect trigger, and (c) at boot when starting up with a configuration that
     combines storm-detect and a SFP Module X - the warning text shall be "Warning: SFP SFP Module X on port <id
     > is not compatible with storm control; storm-control protection on this port is not effective." [ASSUMED -
      PM/PO confirmation required] - this guarantees the operator is informed in every realistic deployment
     sequence - PM/PO confirmation required before implementation.
ASSUMPTION-12: CPU bandwidth limiting prerequisite ([MRS-017]) is assumed to be a documentation-only
     prerequisite in this release: storm-control configuration shall not block or warn on absence of CPU
     bandwidth limiting; it is the operator’s responsibility per the MRS - this matches the MRS phrasing without
      expanding scope - PM/PO confirmation required before implementation.
ASSUMPTION-13: Concurrency limits are assumed to be: up to one storm-detect trigger per port, with the maximum
     number of monitored ports equal to the total port count of the the supported hardware family device; up to
     one alarm action per trigger of each action type (port-shutdown, log, LED) - this matches the MRS examples
     and avoids scaling claims that cannot be verified at FS time - PM/PO confirmation required before
     implementation.
ASSUMPTION-14: Interaction with other port-affecting features is assumed to follow this precedence: storm-
     control port-shutdown takes effect by setting the port operationally down via the same mechanism as
     administrative shutdown; therefore RSTP/the Ring Protocol, LAG, PoE and 802.1X observe the port as down and
      react per their existing semantics; storm-control does not introduce new precedence rules - this re-uses
     existing platform behaviour - PM/PO confirmation required before implementation.
ASSUMPTION-15: Behaviour on disabled ports ([MRS-021]) is assumed to be accepted as a known limitation in this
     release: the FS documents the behaviour, acceptance tests verify it, and a documentation note warns the
     operator; no source-code change to block the configuration on disabled ports is in scope - this matches the
      MRS labelling of [MRS-021] as a known issue - PM/PO confirmation required before implementation.
ASSUMPTION-16: Statistics and counters exposed via ‘show alarm‘ for storm-detect triggers are assumed to include
     : configured threshold (fps), persistence interval (s), current operational state (normal / triggered /
     persisting), last trigger UTC timestamp, last recovery UTC timestamp, and trigger-cycle count since boot -
     this provides minimum operator observability without expanding the show-command scope beyond [MRS-010] - PM
     /PO confirmation required before implementation.
ASSUMPTION-17: SNMP/management-plane integration is assumed to be out of scope for this release; storm-detect
```

```
      events generate local syslog entries only, plus any platform-wide remote syslog forwarding the operator has
       configured - this keeps scope aligned with the MRS, which lists only port/log/LED actions - PM/PO
      confirmation required before implementation.
ASSUMPTION-18: The canonical show command is assumed to be ‘show alarm‘ (no separate storm-control show command
      introduced); output for storm-detect triggers shall include the fields enumerated in ASSUMPTION-16 - this
      resolves the [MRS-010] "or equivalent" ambiguity - [SYNTAX PROPOSAL - verify against The Software CLI
      conventions] - PM/PO confirmation required before implementation.
ASSUMPTION-19: Configuration validation rules are assumed to be: (a) a trigger without an action reference is
      rejected, (b) an action without a target is rejected, (c) overlapping port lists across multiple storm-
      detect triggers are rejected, (d) port identifiers not present on the device are rejected, (e) out-of-range
       threshold or persistence values are rejected - each rejection produces a CLI error message identifying the
       invalid parameter - this aligns with general The Software CLI input-validation practice - PM/PO
      confirmation required before implementation.
ASSUMPTION-20: Port range/list semantics are assumed to follow the same convention as other The Software port-
      list CLI commands: hyphenated ranges (‘1-2‘), comma-separated lists (‘1,3,5‘) and combinations are accepted
      ; references to non-existent ports are rejected; an "all" keyword is not introduced specifically for storm
      control in this release - this re-uses existing The Software CLI conventions - [SYNTAX PROPOSAL - verify
      against The Software CLI conventions] - PM/PO confirmation required before implementation.
ASSUMPTION-21: Behaviour on configuration change while triggered is assumed to be: deleting the trigger
      immediately re-enables a port that was shut down by that trigger and clears the LED contribution; modifying
       threshold or persistence on an existing trigger leaves the current shut-down state intact and applies the
      new parameters going forward - this provides predictable operator semantics - PM/PO confirmation required
      before implementation.
ASSUMPTION-22: Maximum detection latency from threshold breach to port shutdown is assumed to be <= 2 seconds (i
      .e. one full 1-second measurement window per ASSUMPTION-02 plus up to 1 second for action propagation)
      under nominal CPU load - this is a conservative bound consistent with ASSUMPTION-02 - PM/PO confirmation
      required before implementation.
ASSUMPTION-23: Authenticated CLI scope is assumed to map as follows: The Software administrator-equivalent role
      may configure and view storm control; The Software operator/read-only-equivalent role may only view storm
      control status via ‘show alarm‘; the exact role names align with existing The Software RBAC roles - this
      satisfies [MRS-SEC-01]/[MRS-SEC-02] without inventing new roles - PM/PO confirmation required before
      implementation; see [R5].
ASSUMPTION-24: Audit log fields for storm-control configuration changes are assumed to include: UTC timestamp,
      authenticated user identity, source (CLI session / management interface), command or parameter changed,
      before-value, after-value - this satisfies CR 2.08, CR 2.11 and CR 2.12 - PM/PO confirmation required
      before implementation.
ASSUMPTION-25: DoS-resilience quantitative criteria are assumed to be: under sustained ingress at line rate on
      all configured ports of a the supported hardware family device, (a) the storm-detect trigger fires within
      the latency bound of ASSUMPTION-22, and (b) interactive ‘show‘ CLI commands return their first output
      within <= 5 seconds - these provide testable acceptance criteria for CR 7.01/7.02 - PM/PO confirmation
      required before implementation.
ASSUMPTION-26: The truncated [MRS-SEC-03] requirement text is assumed to read in full: "All storm control
      configuration changes and all storm-control-triggered port shutdown events shall be logged with timestamp
      and sufficient context (port, user, action) to identify the event (CR 2.08)." - this restoration is
      consistent with the surrounding requirements and the cited CR - PM/PO confirmation required before
      implementation; MRS author to restore the canonical text in the next MRS revision.
ASSUMPTION-27: Empty MRS back-matter sections (Revision History, Reference Documents, Terms and Abbreviations)
      are assumed to be addressed in this FS by the corresponding sections 8, 9 and 10; the FS does not attempt
      to retrofit the MRS itself - PM/PO confirmation required before implementation; MRS author to populate the
      equivalent sections in the next MRS revision.
ASSUMPTION-28: Customer Representatives section completeness in the MRS is assumed to be out of scope for this
      FS; absence of external customer representatives in the MRS does not block FS generation - this is a
      process matter, not a functional matter - PM/PO confirmation required before implementation.
```